SQL VE VERİ MÜHENDİSLİĞİ

Teknik Mülakat Hazırlık Notları

SQL • Python • Veri Modelleme • Data Engineering • Orkestrasyon • Doküman Zekası

Mülakat Soruları ve İdeal Cevaplar

İçindekiler

Giriş: Kapsam ve Çalışma Formatı 5
SQL: Temel Veritabanı Kavramları 6
İlişkisel ve İlişkisel Olmayan Veritabanları (SQL vs NoSQL) 6
Anahtarlar: Primary Key, Foreign Key ve Unique Constraint 8
Normalizasyon ve Denormalizasyon 10
SQL: Sorgulama, Filtreleme ve Join'ler 14
Temel Sorgu Yapısı ve Operatörler 14
Sorgu Çalıştırma Sırası (Logical Execution Order) 16
Join Türleri (INNER, LEFT, RIGHT, FULL OUTER) 18
Self Join ve Cross Join 20
Küme Operatörleri (UNION, UNION ALL, INTERSECT, EXCEPT) 22
SQL: Gruplama ve İleri Teknikler 24
Aggregate Fonksiyonlar ve GROUP BY 24
WHERE ve HAVING Farkı 26
Alt Sorgular (Subquery): Correlated ve Non-Correlated 28
CTE (Common Table Expressions) ve Recursive CTE 30
Window (Pencere) Fonksiyonları 32
Koşullu Mantık: CASE WHEN 35
SQL: Veri Değiştirme ve Performans 37
DML, DDL, DCL ve TCL 37
TRUNCATE, DELETE ve DROP Farkı 39
Index'ler: Clustered ve Non-Clustered 40
Sorgu Optimizasyonu 42
Python: Temeller ve Bellek Mekanizmaları 44
Yorumlanan Dil, Bytecode ve PVM 44
GIL (Global Interpreter Lock) 46
Bellek Yönetimi ve Garbage Collection 48
Mutable ve Immutable Tipler 50
Yerleşik Veri Yapıları ve Zaman Karmaşıklığı 52
Python: Fonksiyonlar ve İleri Sözdizimi 54
*args ve **kwargs 54
Comprehension'lar ve Lambda Fonksiyonları 56
Decorator'lar 58
Generator ve Iterator 60
Python: OOP, Hata Yönetimi ve İyi Pratikler 62
Sınıflar: self, Sınıf ve Örnek Değişkenleri 62
Dunder (Magic) Metotlar 64
Kalıtım ve MRO (Method Resolution Order) 66
Hata Yönetimi: try, except, else ve finally 68

Context Manager ve with Deyimi 70
Sanal Ortamlar, PEP 8 ve Test 72
Veri Mühendisliği: Sistem Tasarımı ve Mimari 74
ETL ve ELT 74
Batch ve Streaming İşleme 75
Data Lake, Data Warehouse ve Lakehouse 77
Medallion Mimarisi (Bronze / Silver / Gold) 80
Idempotency, Backfill ve Hata Modları 81
Veri Mühendisliği: Veri Modelleme 83
Boyutsal Modelleme: Star ve Snowflake Şema 83
Slowly Changing Dimensions (SCD) 85
OLTP ve OLAP Sistemleri 87
Veri Mühendisliği: Büyük Veri İşleme 89
Veri Mühendisliği İçin SQL: Deduplication ve Anti-Join 89
Apache Spark Mimarisi 91
Shuffle, Partitioning ve Data Skew 92
Dosya Formatları: Parquet, Avro ve CSV 94
Orkestrasyon ve Modern Veri Yığını 96
DAG Kavramı ve Apache Airflow 96
SSIS ve Apache Airflow Karşılaştırması 98
Modern Veri Yığını ve dbt 100
Yapılandırılmamış Veri ve Doküman Zekası 102
OCR ve Tesseract 102
LayoutLM ve Doküman Zekası 104
unstructured.io ve LLM/RAG Veri Hatları 106
Veri Ambarı Tasarımı ve Stored Procedure'ler 108
İlişkisel (ER) ve Boyutsal Modelleme, Kardinalite 108
Surrogate Key: Neden Kaynak Sistemin Anahtarını Kullanmayız? 110
Stored Procedure: Transaction Yönetimi ve Hata Yakalama 112
Stored Procedure Performansı: Execution Plan, RBAR ve Parameter Sniffing 114
Temp Table, Table Variable ve CTE Seçimi 116
Microsoft BI Yığını ve Veri Yönetimi 118
SSAS Tabular ve VertiPaq Motoru 118
DAX: Row Context, Filter Context ve CALCULATE 120
Storage Mode: Import ve DirectQuery 122
Master Data Management (MDM) ve Golden Record 123
Veri Yönetişimi ve Row-Level Security (RLS) 125
ETL Sürecinde Veri Kalitesi Yönetimi 127
Veritabanı Operasyonu ve Yüksek Erişilebilirlik 129
Transaction Isolation Seviyeleri ve Eşzamanlılık Anomalileri 129

Kilitleme, Blocking ve Deadlock 131
Partitioning ve Büyük Tablo Yönetimi 133
Yedekleme ve Kurtarma (RPO / RTO) 135
Yüksek Erişilebilirlik ve Felaket Kurtarma (HA / DR) 137
Veri Güvenliği ve Uyum 139
Şifreleme: At-Rest, In-Transit ve TDE 139
Veri Maskeleme ve Anonimleştirme 141
Erişim Yönetimi ve Audit (Denetim İzi) 143
KVKK Uyumu, Veri Saklama ve İmha 145
Ek: Mülakat Öncesi Hızlı Tekrar Listesi 147

Giriş: Kapsam ve Çalışma Formatı

Bu doküman; SQL, Veri Mühendisliği (Data Engineering), orkestrasyon araçları ve yapılandırılmamış veri işleme konularında teknik mülakatlarda en sık karşılaşılan soruları ve bu sorulara verilmesi gereken ideal cevapları içerir.

Mülakatçılar önce kısa, net ve doğrudan soruyu yanıtlayan bir cevap duymak ister. Ardından "Peki bu arka planda nasıl çalışıyor?" veya "Bunu neden böyle yaptın?" diyerek derinleşirler. Bu yüzden her konu iki katmanda işlenmiştir:

- **Mülakat Cevabı:** Doğrudan, net ve 1-2 dakikada sözlü olarak verilebilecek ideal yanıt. Mülakatçının duymak istediği anahtar kelimeleri içerir.
- **Detaylı Arka Plan:** Konunun mimarisi, motorun arka planda ne yaptığı, gerçek hayat senaryoları ve adayların en çok düştüğü tuzaklar.

Kapsanan Konular

- SQL: Temel veritabanı kavramları, sorgulama ve filtreleme, join'ler, gruplama, ileri teknikler (CTE, window fonksiyonları), veri değiştirme ve performans/index optimizasyonu.
- Python: Bytecode ve PVM, GIL, bellek yönetimi ve garbage collection, mutable/immutable tipler, veri yapıları, decorator ve generator, OOP ve MRO, hata yönetimi, context manager, sanal ortam ve test.
- Veri Mühendisliği: Sistem tasarımı (ETL/ELT, batch/streaming, Lakehouse, idempotency), veri modelleme (Kimball, star şema, SCD) ve büyük veri işleme (Spark, dosya formatları).
- Orkestrasyon ve Modern Veri Yığını: DAG kavramı, Apache Airflow, SSIS ile karşılaştırma, dbt ve Modern Data Stack.
- Yapılandırılmamış Veri ve Doküman Zekası: OCR (Tesseract), LayoutLM ve unstructured.io ile LLM/RAG veri hatları.
- Veri Ambarı Tasarımı ve BI Katmanı: ER ve boyutsal modelleme, kardinalite, surrogate key, stored procedure'lerde transaction ve performans, SSAS Tabular ve VertiPaq, DAX bağlamları, MDM, Row-Level Security ve veri kalitesi yönetimi.
- Veritabanı Operasyonu ve Güvenlik: Transaction isolation seviyeleri, kilitleme ve deadlock, partitioning, yedekleme ve kurtarma (RPO/RTO), yüksek erişilebilirlik, şifreleme, maskeleme, audit ve KVKK uyumu.

Nasıl Çalışılmalı?

Her başlıkta önce "Mülakat Cevabı" kısmını yüksek sesle tekrar ederek akıcı hale getir. Ardından "Detaylı Arka Plan" bölümündeki tuzak sorulara odaklan; mülakatlarda adayları eleyen kısım genellikle temel tanım değil, bu tuzak sorulardır.

SQL: Temel Veritabanı Kavramları

İlişkisel ve İlişkisel Olmayan Veritabanları (SQL vs NoSQL)

> **SORU:** *İlişkisel (SQL) ve ilişkisel olmayan (NoSQL) veritabanları arasındaki fark nedir? Bir projede hangisini seçeceğine nasıl karar verirsin?*

1. Mülakat Cevabı

> *"Temel fark şema katılığı, tutarlılık garantileri ve ölçeklenme biçimidir.*
>
> *İlişkisel veritabanları (PostgreSQL, MySQL, SQL Server, Oracle) veriyi önceden tanımlanmış bir şemaya göre satır ve sütunlardan oluşan tablolarda tutar. Tablolar arasında Foreign Key ile ilişki kurulur ve ACID garantileri (Atomicity, Consistency, Isolation, Durability) sayesinde veri tutarlılığı çok güçlüdür. Geleneksel olarak dikey ölçeklenirler (scale-up: daha güçlü makine).*
>
> *NoSQL veritabanları (MongoDB, Cassandra, DynamoDB, Redis, Neo4j) esnek şemalıdır; veriyi doküman, key-value, geniş kolon (wide-column) veya graph modelinde tutar. Yatay ölçeklenmek (scale-out: daha çok makine) ve çok yüksek yazma hacmi için tasarlanmışlardır. Çoğu, ACID yerine BASE modelini benimseyerek nihai tutarlılık (eventual consistency) sunar.*
>
> *Seçim kriterim şudur: Veri yapısı belli, tablolar arası ilişkiler karmaşık ve tutarlılık kritikse (finans, sipariş, muhasebe, envanter) ilişkisel veritabanı kullanırım. Şema sık değişiyorsa, veri yarı yapılandırılmışsa ve devasa yatay ölçek gerekiyorsa (log kayıtları, IoT telemetrisi, ürün kataloğu, oturum verisi) NoSQL kullanırım. Gerçek projelerde ise genellikle ikisi birlikte kullanılır; buna Polyglot Persistence denir."*

2. Detaylı Arka Plan

Mülakatçı bu cevabı aldıktan sonra genellikle ACID, CAP teoremi veya "NoSQL daha mı hızlıdır?" gibi sorularla derinleşir.

ACID ve BASE Modelleri

ACID, ilişkisel veritabanlarının işlem (transaction) garantisidir:

- **Atomicity (Bölünmezlik):** Bir transaction ya tamamen uygulanır ya da hiç uygulanmaz. Para transferinde hesaptan düşüp diğerine yatırmama diye bir ara durum olamaz.
- **Consistency (Tutarlılık):** Transaction sonunda veritabanı tüm kısıtları (constraint, FK, check) sağlayan geçerli bir durumda kalır.
- **Isolation (Yalıtım):** Eşzamanlı çalışan transaction'lar birbirinin ara durumunu görmez. İzolasyon seviyeleri (Read Uncommitted, Read Committed, Repeatable Read, Serializable) bunun ne kadar katı olacağını belirler.
- **Durability (Kalıcılık):** Commit edilen veri, elektrik kesilse bile kaybolmaz (write-ahead log sayesinde).

BASE ise NoSQL dünyasının yaklaşımıdır: Basically Available (temel olarak erişilebilir), Soft state (durum zamanla değişebilir), Eventual consistency (yeterli süre geçerse tüm düğümler aynı veriye yakınsar). Yani bir düğüme yazdığın veriyi başka bir düğümden hemen okuyamayabilirsin.

CAP Teoremi

Dağıtık bir sistemde şu üç özellikten aynı anda en fazla ikisi garanti edilebilir: Consistency (tutarlılık), Availability (erişilebilirlik) ve Partition Tolerance (ağ bölünmesine dayanıklılık).

Ağ bölünmesi (partition) gerçek hayatta kaçınılmaz olduğu için pratikte seçim C ile A arasındadır. Örneğin MongoDB ve HBase CP tarafındadır (tutarlılığı korur, gerekirse erişilemez olur); Cassandra ve DynamoDB ise AP tarafındadır (erişilebilir kalır, tutarlılık gecikir).

Mülakat Tuzağı: "NoSQL daha hızlıdır" demek

Bu cümleyi kurmak junior bir izlenim bırakır. Doğru cevap şudur: NoSQL, tasarlandığı erişim deseni (access pattern) için hızlıdır. Örneğin DynamoDB'de bir partition key ile tek kayıt çekmek milisaniyeler sürer; ancak aynı veritabanında beş tabloyu birleştiren, gruplayan ve sıralayan bir analitik sorgu yazmaya çalışırsan performans felakete döner çünkü bu iş için tasarlanmamıştır.

Kilit ifade şudur: "İlişkisel veritabanlarında önce veriyi modelleriz, sonra sorguyu yazarız. NoSQL'de ise önce sorguyu (erişim desenini) belirleriz, sonra veriyi ona göre modelleriz."

Karşılaştırma Tablosu

Özellik	İlişkisel (SQL)	İlişkisel Olmayan (NoSQL)
Şema	Katı, önceden tanımlı (schema-on-write)	Esnek veya şemasız (schema-on-read)
Tutarlılık	ACID, güçlü tutarlılık	BASE, çoğunlukla nihai tutarlılık
Ölçeklenme	Dikey (scale-up) ağırlıklı	Yatay (scale-out) doğal
İlişki/JOIN	Foreign Key ve JOIN yerleşiktir	JOIN genelde yok; veri gömülür (embed) veya uygulamada birleştirilir
Kullanım Alanı	Finans, sipariş, ERP, muhasebe	Log, IoT, katalog, önbellek, gerçek zamanlı öneri
Örnekler	PostgreSQL, MySQL, SQL Server, Oracle	MongoDB, Cassandra, DynamoDB, Redis, Neo4j

Anahtarlar: Primary Key, Foreign Key ve Unique Constraint

> **SORU:** *Primary Key, Foreign Key ve Unique Constraint arasındaki farkları açıkla. Bir tabloda kaç tane Primary Key olabilir?*

1. Mülakat Cevabı

> *"Bu üçü de veri bütünlüğünü (data integrity) veritabanı seviyesinde garanti eden kısıtlardır (constraint).*
>
> *Primary Key, bir tablodaki her satırı benzersiz şekilde tanımlayan sütun veya sütun grubudur. İki katı kuralı vardır: değerleri benzersizdir ve NULL olamaz. Bir tabloda yalnızca bir tane Primary Key bulunur; birden fazla sütundan oluşuyorsa buna Composite Key denir. Çoğu veritabanında Primary Key otomatik olarak bir index (SQL Server ve InnoDB'de clustered index) oluşturur.*
>
> *Foreign Key, bir tablodaki sütunun başka bir tablonun Primary Key'ine (veya Unique sütununa) işaret etmesidir. Amacı referans bütünlüğünü korumaktır; yani sistemde olmayan bir müşteriye sipariş kaydı girilmesini veritabanı seviyesinde engeller. Bir tabloda birden fazla Foreign Key olabilir ve NULL değer alabilir.*
>
> *Unique Constraint de benzersizliği garanti eder, ancak Primary Key'den iki farkı vardır: bir tabloda birden fazla Unique Constraint tanımlanabilir ve NULL değere izin verir. Örneğin bir kullanıcı tablosunda id sütunu Primary Key iken, email ve tc_no sütunları Unique Constraint olabilir."*

2. Detaylı Arka Plan

Örnek Şema

```SQL
CREATE TABLE musteri (
    musteri_id   INT          PRIMARY KEY,        -- benzersiz + NOT NULL
    email        VARCHAR(255) UNIQUE,             -- benzersiz, NULL olabilir
    tc_no        CHAR(11)     UNIQUE,
    ad_soyad     VARCHAR(200) NOT NULL
);

CREATE TABLE siparis (
    siparis_id   INT PRIMARY KEY,
    musteri_id   INT NOT NULL,
    tutar        DECIMAL(12,2) NOT NULL,
    CONSTRAINT fk_siparis_musteri
        FOREIGN KEY (musteri_id) REFERENCES musteri(musteri_id)
        ON DELETE RESTRICT      -- musterinin siparisi varsa silinmesini engelle
        ON UPDATE CASCADE
);
```

Mülakat Tuzağı: Unique sütuna kaç NULL girilebilir?

Bu, veritabanına göre değişen ve adayları ayıran bir detaydır. Standart SQL'e göre NULL değerler birbirine eşit sayılmaz, dolayısıyla bir Unique sütuna birden fazla NULL girilebilir. PostgreSQL, Oracle ve MySQL bu şekilde davranır. Ancak SQL Server'da klasik unique index yalnızca tek bir NULL değere izin verir; birden fazlasını istiyorsan filtreli index kullanman gerekir.

Mülakatta "Veritabanına göre değişir" diye başlamak ve ardından örnek vermek, konuya gerçekten hakim olduğunu gösterir.

Natural Key ve Surrogate Key

- **Natural Key:** İş dünyasında zaten anlamı olan bir alan (TC kimlik no, ISBN, plaka). Avantajı ek sütun gerektirmemesidir; dezavantajı iş kuralları değişince (örneğin format değişimi) tüm ilişkilerin bozulmasıdır.

- **Surrogate Key:** İş anlamı olmayan, sistem tarafından üretilen yapay anahtar (auto-increment id, IDENTITY, UUID). Veri ambarlarında neredeyse her zaman tercih edilir çünkü kaynak sistemdeki değişikliklerden etkilenmez ve Slowly Changing Dimension yönetimini mümkün kılar.

Foreign Key Silme Davranışları

- **ON DELETE RESTRICT / NO ACTION:** Bağlı kayıt varsa silmeye izin vermez. En güvenli ve en yaygın varsayılan davranıştır.
- **ON DELETE CASCADE:** Ana kayıt silinince bağlı tüm kayıtları da siler. Güçlüdür ama tehlikelidir; tek bir müşteri silmek binlerce siparişi sessizce yok edebilir.
- **ON DELETE SET NULL:** Bağlı kayıtların Foreign Key sütununu NULL yapar. Sütunun NOT NULL olmaması gerekir.

Mülakat Sorusu: Neden bazı veri ambarlarında Foreign Key tanımlanmaz?

Kritik cevap: Foreign Key her INSERT işleminde referans kontrolü yapılmasını gerektirir, bu da yükleme (load) performansını ciddi şekilde düşürür. Veri ambarlarında ve analitik sistemlerde veri zaten kontrollü bir ETL/ELT hattından geçtiği için, bütünlük kontrolü veritabanı yerine dönüşüm katmanına (örneğin dbt testleri) taşınır. Snowflake ve BigQuery gibi bazı sistemlerde Foreign Key tanımlanabilir ama zorlanmaz (informational only); yalnızca sorgu optimizasyonuna ipucu verir.

Normalizasyon ve Denormalizasyon

> **SORU:** *Normalizasyon nedir, hangi problemleri çözer? Ne zaman bilinçli olarak denormalize edersin?*

1. Mülakat Cevabı

> *"Normalizasyon, veriyi tekrarı (redundancy) ve güncelleme anomalilerini ortadan kaldıracak şekilde birden fazla ilişkili tabloya bölme işlemidir. Temel amacı veri bütünlüğüdür: bir bilgi sistemde tek bir yerde tutulur, dolayısıyla tek bir noktadan güncellenir.*
>
> *Mülakatlarda ilk üç normal formu bilmek yeterlidir ve üçü birden tek bir cümlede özetlenebilir: her sütun anahtara, anahtarın tamamına ve yalnızca anahtara bağımlı olmalıdır. Açarsak: 1NF her hücrede bölünemez tek bir değer bulunmasını ve tekrarlayan grupların kaldırılmasını ister. 2NF, bileşik bir birincil anahtarın yalnızca bir parçasına bağımlı olan sütunların ayrı tabloya taşınmasını, yani kısmi bağımlılığın kaldırılmasını ister. 3NF ise anahtar olmayan bir sütunun başka bir anahtar olmayan sütuna bağımlı olmasını yasaklar, yani geçişli bağımlılığı kaldırır.*
>
> *Denormalizasyon ise bilinçli olarak tabloları birleştirip veri tekrarına izin vermektir. Amaç okuma performansıdır: JOIN sayısını azaltarak raporlama ve analitik sorguları hızlandırır. Bedeli ise veri tekrarı, daha fazla depolama ve güncelleme sırasında tutarlılığı elle korumak zorunda kalmaktır.*
>
> *Pratik kuralım şudur: OLTP (işlem) sistemlerinde normalize ederim çünkü orada çok sayıda küçük yazma işlemi vardır ve tutarlılık kritiktir. OLAP (analitik/raporlama) sistemlerinde ise denormalize ederim çünkü orada veri toplu ve kontrollü şekilde yüklenir, asıl maliyet okuma tarafındadır."*

2. Detaylı Arka Plan

Normalizasyon Hangi Anomalileri Çözer?

Normalize edilmemiş tek bir tabloda müşteri ve sipariş bilgilerini birlikte tuttuğunu düşün. Üç tür anomali ortaya çıkar:

- **Güncelleme Anomalisi (Update Anomaly):** Müşterinin adresi değişti. Adres bilgisi o müşterinin 500 siparişinin her satırında tekrar ettiği için 500 satırı da güncellemen gerekir. Biri atlanırsa veri tutarsız hale gelir.
- **Ekleme Anomalisi (Insert Anomaly):** Henüz hiç sipariş vermemiş yeni bir müşteriyi sisteme ekleyemezsin, çünkü tablonun anahtarı siparişe bağlıdır.
- **Silme Anomalisi (Delete Anomaly):** Bir müşterinin tek siparişini sildiğinde, o müşterinin bilgileri de sistemden tamamen kaybolur.

Normal Formlar: Tek Bir Örnek Üzerinde Adım Adım

Normal formları ayrı ayrı ezberlemek yerine, tek bir bozuk tabloyu üç adımda düzelterek öğrenmek çok daha kalıcıdır. Aşağıdaki sipariş tablosuyla başlayalım; bu tablo hiçbir normal forma uymuyor.

```
Başlangıç (normalize edilmemiş)
siparis_ham        (urunler = "urun:adet:fiyat" ciftleri, virgulle ayrilmis)
+------------+------------+--------------+--------+-------+--------------------------+
| siparis_no | musteri_no | musteri_ad   | sehir  | plaka | urunler                  |
+------------+------------+--------------+--------+-------+--------------------------+
| 1001       | 50         | Ahmet Yilmaz | Ankara | 06    | Klavye:2:150, Mouse:1:80 |
| 1002       | 51         | Ayse Demir   | Izmir  | 35    | Monitor:1:3200           |
| 1003       | 50         | Ahmet Yilmaz | Ankara | 06    | Mouse:3:80               |
+------------+------------+--------------+--------+-------+--------------------------+

Sorunlar:
  - "urunler" hucresi birden fazla urunu tek metinde tutuyor.
  - Musteri bilgileri her siparis satirinda tekrar ediyor.
  - Sehir ile plaka arasindaki iliski her satirda gereksizce saklaniyor.
```

1. Normal Form (1NF): Her Hücrede Tek ve Bölünemez Değer

Kural: Her hücre yalnızca tek bir atomik değer içermelidir. Tekrarlayan gruplar (aynı bilgiyi tutan urun1, urun2, urun3 gibi sütunlar veya virgülle ayrılmış listeler) bulunmamalıdır. Ayrıca her satırı benzersiz şekilde tanımlayan bir birincil anahtar olmalıdır.

İhlal: urunler sütunu tek hücrede birden fazla ürünü tutuyor. Bu haliyle "Mouse kaç adet satıldı?" sorusunu SQL ile cevaplamak imkansızdır; metin ayrıştırmak zorunda kalırsın.

Düzeltme: Her ürün kendi satırına taşınır. Artık bir siparişte birden fazla kalem olabileceği için birincil anahtar bileşik hale gelir: (siparis_no, urun_kodu).

```
1NF sonrası
siparis_kalem      PK: (siparis_no, urun_kodu)
   kisaltmalar: sip_no = siparis_no, urun = urun_kodu, mus = musteri_no
+--------+------+---------+------+-------+-----+--------------+--------+-------+
| sip_no | urun | urun_ad | adet | fiyat | mus | musteri_ad   | sehir  | plaka |
+--------+------+---------+------+-------+-----+--------------+--------+-------+
| 1001   | K-01 | Klavye  | 2    | 150   | 50  | Ahmet Yilmaz | Ankara | 06    |
| 1001   | M-07 | Mouse   | 1    | 80    | 50  | Ahmet Yilmaz | Ankara | 06    |
| 1002   | N-22 | Monitor | 1    | 3200  | 51  | Ayse Demir   | Izmir  | 35    |
| 1003   | M-07 | Mouse   | 3    | 80    | 50  | Ahmet Yilmaz | Ankara | 06    |
+--------+------+---------+------+-------+-----+--------------+--------+-------+

Kazanc : Artik SQL ile toplama, gruplama ve filtreleme yapilabilir.
Kalan  : Musteri ve urun bilgileri hala her satirda tekrar ediyor.
```

2. Normal Form (2NF): Kısmi Bağımlılığı Kaldır

Kural: Tablo 1NF'te olmalı ve anahtar olmayan hiçbir sütun, bileşik birincil anahtarın YALNIZCA BİR PARÇASINA bağımlı olmamalıdır. Buna kısmi bağımlılık (partial dependency) denir.

Kritik nokta: 2NF yalnızca birincil anahtar bileşik olduğunda anlamlıdır. Anahtar tek bir sütundan oluşuyorsa kısmi bağımlılık matematiksel olarak mümkün değildir, yani tablo 1NF'teyse zaten 2NF'tedir. Mülakatta bu detayı söylemek konuyu ezberden değil anlayarak bildiğini gösterir.

İhlal analizi: Anahtarımız (siparis_no, urun_kodu). Her sütunun neye bağımlı olduğuna bakalım:

```
Bağımlılık analizi
adet, birim_fiyat  -> (siparis_no + urun_kodu)  : TAM bagimli, kalabilir
                      "Bu siparisteki bu urunden kac adet?" -> iki parca da gerekli

urun_adi           -> (urun_kodu)               : KISMI bagimlilik  [IHLAL]
                      Urunun adi hangi siparise ait oldugundan bagimsizdir.

musteri_no,        -> (siparis_no)              : KISMI bagimlilik  [IHLAL]
musteri_ad,              Musteri bilgisi hangi urun oldugundan bagimsizdir.
sehir, plaka
```

Düzeltme: Kısmi bağımlı sütunlar, bağımlı oldukları anahtar parçasıyla birlikte ayrı tablolara taşınır.

```
2NF sonrası
siparis            PK: siparis_no
+------------+------------+--------------+--------+-------+
| siparis_no | musteri_no | musteri_ad   | sehir  | plaka |
+------------+------------+--------------+--------+-------+
| 1001       | 50         | Ahmet Yilmaz | Ankara | 06    |
| 1002       | 51         | Ayse Demir   | Izmir  | 35    |
| 1003       | 50         | Ahmet Yilmaz | Ankara | 06    |
+------------+------------+--------------+--------+-------+
```

```
urun              PK: urun_kodu
+-----------+----------+          siparis_kalem   PK: (siparis_no, urun_kodu)
| urun_kodu | urun_adi |          +------------+-----------+------+-------------+
+-----------+----------+          | siparis_no | urun_kodu | adet | birim_fiyat |
| K-01      | Klavye   |          +------------+-----------+------+-------------+
| M-07      | Mouse    |          | 1001       | K-01      | 2    | 150         |
| N-22      | Monitor  |          | 1001       | M-07      | 1    | 80          |
+-----------+----------+          | 1002       | N-22      | 1    | 3200        |
                                  | 1003       | M-07      | 3    | 80          |
                                  +------------+-----------+------+-------------+

Kazanc : Urun adi tek yerde. Adi degisirse tek satir guncellenir.
Kalan  : siparis tablosunda musteri bilgileri hala tekrar ediyor.
```

3. Normal Form (3NF): Geçişli Bağımlılığı Kaldır

Kural: Tablo 2NF'te olmalı ve anahtar olmayan hiçbir sütun, anahtar olmayan başka bir sütuna bağımlı olmamalıdır. Buna geçişli (transitive) bağımlılık denir: A anahtardır, B A'ya bağlıdır, C ise B'ye bağlıdır.

İhlal analizi: siparis tablosunda anahtar siparis_no. Ancak iki geçişli bağımlılık var:

```
Bağımlılık analizi
siparis_no -> musteri_no -> musteri_ad     [GECISLI IHLAL]
              Musterinin adi siparise degil, musteri numarasina baglidir.

siparis_no -> sehir      -> plaka          [GECISLI IHLAL]
              Plaka siparise degil, sehre baglidir. (Ankara her zaman 06'dir.)
```

Düzeltme: Geçişli bağımlı sütunlar, bağımlı oldukları sütunun birincil anahtar olduğu yeni tablolara taşınır.

```
3NF sonrası (nihai tasarım)
sehir              PK: sehir_kodu          musteri        PK: musteri_no
+------------+-----------+-------+        +------------+--------------+------------+
| sehir_kodu | sehir_adi | plaka |        | musteri_no | musteri_ad   | sehir_kodu |
+------------+-----------+-------+        +------------+--------------+------------+
| 6          | Ankara    | 06    |        | 50         | Ahmet Yilmaz | 6          |
| 35         | Izmir     | 35    |        | 51         | Ayse Demir   | 35         |
+------------+-----------+-------+        +------------+--------------+------------+

siparis            PK: siparis_no         siparis_kalem  PK: (siparis_no, urun_kodu)
+------------+------------+               +------------+-----------+------+-------------+
| siparis_no | musteri_no |               | siparis_no | urun_kodu | adet | birim_fiyat |
+------------+------------+               +------------+-----------+------+-------------+
| 1001       | 50         |               | 1001       | K-01      | 2    | 150         |
| 1002       | 51         |               | 1001       | M-07      | 1    | 80          |
| 1003       | 50         |               | 1002       | N-22      | 1    | 3200        |
+------------+------------+               | 1003       | M-07      | 3    | 80          |
                                          +------------+-----------+------+-------------+
urun               PK: urun_kodu
+-----------+----------+
| urun_kodu | urun_adi |          Sonuc: Her bilgi sistemde TEK bir yerde tutuluyor.
| K-01      | Klavye   |          Musterinin sehri degisirse tek satir guncellenir.
| M-07      | Mouse    |
| N-22      | Monitor  |
+-----------+----------+
```

Üç Adımın Özeti

Form	Kural	Bu örnekte ne düzeltildi?
1NF	Her hücrede tek ve bölünemez değer; tekrarlayan grup yok; birincil anahtar var	Tek hücredeki ürün listesi ayrı satırlara bölündü

Form	Kural	Bu örnekte ne düzeltildi?
2NF	1NF + bileşik anahtarın yalnızca bir parçasına bağımlılık yok (kısmi bağımlılık)	urun_adi ve müşteri bilgileri kendi tablolarına taşındı
3NF	2NF + anahtar olmayan sütuna bağımlılık yok (geçişli bağımlılık)	musteri_ad ve plaka, bağlı oldukları varlığın tablosuna taşındı

Bu üç kuralı mülakatta tek cümlede özetlemek etkileyicidir: "Her sütun anahtara, anahtarın tamamına ve yalnızca anahtara bağımlı olmalıdır." Cümlenin üç parçası sırasıyla 1NF, 2NF ve 3NF'e karşılık gelir.

Bonus: BCNF (Boyce-Codd Normal Form)

3NF'in biraz daha katı halidir ve nadiren de olsa mülakatlarda sorulur. 3NF, anahtar olmayan sütunların bağımlılıklarıyla ilgilenir; BCNF ise bunu genişleterek her belirleyicinin (determinant) bir aday anahtar olmasını şart koşar. Aradaki fark yalnızca tabloda birden fazla örtüşen aday anahtar bulunduğunda ortaya çıkar; pratikte 3NF'te olan tabloların büyük çoğunluğu zaten BCNF'tedir. Mülakatta "3NF genellikle yeterlidir, BCNF'e ihtiyaç duyulan durumlar özel ve nadirdir" demek doğru ve yeterli bir cevaptır.

Mülakat Tuzağı: "Her zaman 3NF'e kadar normalize etmeli miyiz?"

Kesinlikle hayır. Bu, junior ile senior arasındaki ayrımın görüldüğü sorulardan biridir.

Veri ambarlarında kullanılan Star Schema (Yıldız Şema) bilinçli olarak denormalize edilmiştir: boyut (dimension) tabloları 3NF'e uymaz, çünkü analistin 6 tabloyu birleştirmek yerine tek bir boyut tablosuna JOIN yapması istenir. Kimball metodolojisi bunu açıkça savunur.

Benzer şekilde, sık hesaplanan bir toplam değeri (örneğin siparis_toplam_tutar) sipariş tablosunda tutmak da denormalizasyondur; her seferinde sipariş kalemlerini toplamaktan çok daha hızlıdır. Buna türetilmiş sütun (derived column) denir ve trigger, materialized view veya ETL ile senkron tutulur.

Karşılaştırma Tablosu

Kriter	Normalizasyon	Denormalizasyon
Temel Amaç	Veri bütünlüğü ve tutarlılık	Okuma (sorgu) performansı
Veri Tekrarı	Minimum	Bilinçli olarak var
JOIN Sayısı	Fazla	Az
Yazma Performansı	Yüksek (tek noktadan güncelleme)	Düşük (birden çok yerde güncelleme)
Depolama	Daha az	Daha fazla
Tipik Kullanım	OLTP: sipariş, banka, ERP	OLAP: veri ambarı, raporlama, star şema

SQL: Sorgulama, Filtreleme ve Join'ler

Temel Sorgu Yapısı ve Operatörler

> **SORU:** *SELECT, FROM, WHERE yapısını ve sık kullanılan operatörleri anlat. NULL değerlerle çalışırken nelere dikkat edilmeli?*

1. Mülakat Cevabı

> *"SELECT hangi sütunların döneceğini, FROM verinin hangi tablodan okunacağını, WHERE ise hangi satırların filtreleneceğini belirtir.*
>
> *Sık kullandığımız operatörler üç grupta toplanır: karşılaştırma operatörleri (=, <>, <, >, <=, >=), mantıksal operatörler (AND, OR, NOT) ve özel filtreleme operatörleri (LIKE ile desen eşleştirme, IN ile liste içinde arama, BETWEEN ile aralık sorgulama, IS NULL ile boş değer kontrolü).*
>
> *NULL konusunda en kritik nokta şudur: NULL bir değer değil, 'bilinmeyen' anlamına gelen bir işaretçidir. Bu yüzden SQL üç değerli mantık (three-valued logic) kullanır: TRUE, FALSE ve UNKNOWN. NULL hiçbir şeye, hatta kendisine bile eşit değildir; dolayısıyla 'sutun = NULL' yazmak asla sonuç döndürmez, mutlaka IS NULL veya IS NOT NULL kullanılmalıdır. En tehlikeli tuzak ise NOT IN operatörünün içinde NULL bulunmasıdır; bu durumda sorgu hiç satır döndürmez."*

2. Detaylı Arka Plan

Üç Değerli Mantık ve NULL Karşılaştırmaları

```sql
SELECT NULL = NULL;      -- sonuc: NULL (TRUE degil!)
SELECT NULL <> 5;        -- sonuc: NULL
SELECT NULL IS NULL;     -- sonuc: TRUE  <-- dogru kullanim

-- WHERE yalnizca TRUE donen satirlari alir.
-- UNKNOWN donen satirlar da elenir, tipki FALSE gibi.
SELECT * FROM calisan WHERE bonus = NULL;   -- HER ZAMAN bos sonuc
SELECT * FROM calisan WHERE bonus IS NULL;  -- dogru
```

Mülakatın Klasik Tuzağı: NOT IN ve NULL

Bu soru, SQL bilgisinin derinliğini ölçmek için çok sık sorulur. Alt sorgudan dönen listede tek bir NULL bile varsa, NOT IN hiçbir satır döndürmez.

```sql
-- yonetici_id sutununda NULL degerler var (ornegin CEO'nun yoneticisi yok)

-- YANLIS: hicbir satir donmez!
SELECT ad FROM calisan
WHERE calisan_id NOT IN (SELECT yonetici_id FROM calisan);

-- Neden? "x NOT IN (1, 2, NULL)" ifadesi
-- "x <> 1 AND x <> 2 AND x <> NULL" seklinde acilir.
-- Son kosul her zaman UNKNOWN doner, AND zinciri asla TRUE olamaz.

-- DOGRU COZUM 1: NULL'lari disla
SELECT ad FROM calisan
WHERE calisan_id NOT IN (
    SELECT yonetici_id FROM calisan WHERE yonetici_id IS NOT NULL
);

-- DOGRU COZUM 2 (tercih edilen): NOT EXISTS kullan
SELECT c.ad FROM calisan c
```

```
WHERE NOT EXISTS (
    SELECT 1 FROM calisan y WHERE y.yonetici_id = c.calisan_id
);
```

NOT EXISTS bu tuzaktan doğal olarak korunur çünkü satır varlığını kontrol eder, değer karşılaştırması yapmaz. Mülakatta "Ben alışkanlık olarak NOT IN yerine NOT EXISTS kullanırım" demek güçlü bir sinyaldir.

Aggregate Fonksiyonlar ve NULL

Aggregate fonksiyonlar NULL değerleri sessizce yok sayar. Bu, fark edilmesi zor hatalara yol açar:

```SQL
-- 5 satir var, bunlardan 2'sinde bonus NULL, digerlerinde 100, 200, 300

SELECT COUNT(*)      FROM calisan;  -- 5  (tum satirlar)
SELECT COUNT(bonus)  FROM calisan;  -- 3  (NULL'lar sayilmaz)
SELECT SUM(bonus)    FROM calisan;  -- 600
SELECT AVG(bonus)    FROM calisan;  -- 200  (600/3, 600/5 DEGIL!)

-- NULL'lari 0 kabul ederek ortalama almak istiyorsan:
SELECT AVG(COALESCE(bonus, 0)) FROM calisan;  -- 120 (600/5)
```

Operatör Detayları ve Performans Notları

- **LIKE:** % sıfır veya daha fazla karakteri, _ tam olarak bir karakteri temsil eder. Kritik performans notu: LIKE 'ahmet%' index kullanabilir, ancak LIKE '%ahmet' baştaki joker karakter yüzünden index kullanamaz ve tam tablo taramasına (full scan) yol açar.
- **BETWEEN:** Her iki uç değeri de kapsar (inclusive). Tarih sorgularında tuzaktır: BETWEEN '2024-01-01' AND '2024-01-31' ifadesi, sütun DATETIME ise 31 Ocak saat 00:00'dan sonraki kayıtları kaçırır. Doğrusu tarih >= '2024-01-01' AND tarih < '2024-02-01' şeklinde yarı açık aralık kullanmaktır.
- **IN:** Kısa listelerde okunabilirdir. Çok büyük listelerde veya alt sorgu içeriyorsa EXISTS ya da JOIN genellikle daha iyi plan üretir.
- **COALESCE:** İlk NULL olmayan değeri döndürür (COALESCE(bonus, prim, 0)). NULLIF ise iki değer eşitse NULL döndürür; sıfıra bölme hatasını engellemek için sık kullanılır: tutar / NULLIF(adet, 0).

Sorgu Çalıştırma Sırası (Logical Execution Order)

> **SORU:** *Bir SQL sorgusunun mantıksal çalıştırma sırası nedir? Neden SELECT içinde tanımladığın bir alias'ı WHERE'de kullanamazsın?*

1. Mülakat Cevabı

> *"SQL'de sorguyu yazdığımız sıra ile motorun onu değerlendirdiği sıra farklıdır. Mantıksal çalıştırma sırası şudur:*
>
> *FROM ve JOIN (kaynak tablolar belirlenir ve birleştirilir) > WHERE (satırlar filtrelenir) > GROUP BY (satırlar gruplanır) > HAVING (gruplar filtrelenir) > SELECT (sütunlar ve alias'lar hesaplanır) > DISTINCT > ORDER BY (sonuç sıralanır) > LIMIT / OFFSET.*
>
> *Bu sıra iki pratik sonucu doğrudan açıklar. Birincisi, SELECT adımı WHERE'den sonra çalıştığı için, SELECT içinde tanımladığın bir alias WHERE aşamasında henüz mevcut değildir ve kullanılamaz; ancak ORDER BY, SELECT'ten sonra çalıştığı için alias'ı orada kullanabilirsin. İkincisi, WHERE gruplamadan önce çalıştığı için aggregate fonksiyon içeremez; grup üzerinde filtreleme yapmak istiyorsan HAVING kullanman gerekir."*

2. Detaylı Arka Plan

Yazım Sırası ve Çalıştırma Sırası

Adım	Yazım Sırası (bizim yazdığımız)	Mantıksal Çalıştırma Sırası
1	SELECT	FROM / JOIN
2	FROM	ON
3	JOIN / ON	WHERE
4	WHERE	GROUP BY
5	GROUP BY	HAVING
6	HAVING	SELECT
7	ORDER BY	DISTINCT
8	LIMIT	ORDER BY, sonra LIMIT / OFFSET

Alias Kuralının Kod Üzerinde Kanıtı

```sql
-- HATA: 'toplam_tutar' alias'i WHERE asamasinda henuz olusmamistir
SELECT musteri_id, SUM(tutar) AS toplam_tutar
FROM siparis
WHERE toplam_tutar > 1000        -- HATA!
GROUP BY musteri_id;

-- DOGRU: grup uzerinde filtreleme HAVING ile yapilir
SELECT musteri_id, SUM(tutar) AS toplam_tutar
FROM siparis
GROUP BY musteri_id
HAVING SUM(tutar) > 1000
ORDER BY toplam_tutar DESC;      -- ORDER BY'da alias GECERLIDIR
```

Not: PostgreSQL ve MySQL, GROUP BY ve ORDER BY içinde alias kullanımına izin verir; SQL Server ise GROUP BY'da izin vermez. Standart davranış olarak alias'ın yalnızca ORDER BY'da güvenli olduğunu söylemek en doğrusudur.

Mülakatın En Öğretici Tuzağı: LEFT JOIN ve WHERE Birlikte Kullanımı

Bu, çalıştırma sırasını gerçekten anlayıp anlamadığını ölçen sorudur. WHERE, JOIN'den sonra çalıştığı için sağ tablodaki bir sütuna WHERE'de koşul koymak, LEFT JOIN'i sessizce INNER JOIN'e dönüştürür.

```sql
-- AMAC: Tum musterileri listele, 2024'te siparisi olanlarin siparisini de goster.

-- YANLIS: siparisi olmayan musteriler kaybolur!
SELECT m.ad, s.siparis_id
FROM musteri m
LEFT JOIN siparis s ON s.musteri_id = m.musteri_id
WHERE s.siparis_tarihi >= '2024-01-01';
-- Cunku siparisi olmayan musteride s.siparis_tarihi NULL'dir,
-- NULL >= '2024-01-01' ifadesi UNKNOWN doner ve satir elenir.

-- DOGRU: kosulu ON icine tasi (birlestirme aninda uygulanir)
SELECT m.ad, s.siparis_id
FROM musteri m
LEFT JOIN siparis s
       ON s.musteri_id = m.musteri_id
      AND s.siparis_tarihi >= '2024-01-01';

-- Alternatif: NULL'lara acikca izin ver
-- WHERE (s.siparis_tarihi >= '2024-01-01' OR s.siparis_id IS NULL)
```

Kural olarak şunu söyleyebilirsin: "LEFT JOIN'de sağ tabloyu filtreleyen koşullar ON içine, sol tabloyu filtreleyen koşullar WHERE içine yazılır."

Join Türleri (INNER, LEFT, RIGHT, FULL OUTER)

> **SORU:** *INNER, LEFT, RIGHT ve FULL OUTER JOIN arasındaki farkları açıkla. LEFT JOIN kullanmanın sana sağladığı garanti nedir?*

1. Mülakat Cevabı

> *"JOIN, iki veya daha fazla tabloyu ortak bir sütun üzerinden birleştirme işlemidir. Dört temel türü vardır:*
>
> *INNER JOIN: Yalnızca her iki tabloda da eşleşen satırları döndürür. Eşleşmeyen kayıtlar tamamen elenir.*
>
> *LEFT JOIN (LEFT OUTER JOIN): Sol tablodaki tüm satırları korur; sağ tabloda eşleşme yoksa o sütunlar NULL gelir. Sağladığı garanti şudur: ana tablomdan hiçbir kaydı kaybetmem. Bu yüzden 'tüm müşteriler ve varsa siparişleri' gibi raporlarda ve eksik veri tespitinde kullanılır.*
>
> *RIGHT JOIN: LEFT JOIN'in aynadaki yansımasıdır; sağ tablodaki tüm satırları korur. Pratikte nadiren kullanılır çünkü tabloların yerini değiştirip LEFT JOIN yazmak daha okunabilirdir.*
>
> *FULL OUTER JOIN: Her iki tablodaki tüm satırları döndürür; eşleşme olmayan taraflar NULL ile doldurulur. İki kaynak sistem arasındaki farkları karşılaştırmak (mutabakat/reconciliation) için idealdir."*

2. Detaylı Arka Plan

Anti-Join: Eksik Kayıtları Bulmak

LEFT JOIN'in en değerli kullanımlarından biri, "A tablosunda olup B tablosunda olmayan" kayıtları bulmaktır. Buna Anti-Join denir ve veri kalitesi kontrollerinin temelidir.

```SQL
-- Hic siparis vermemis musteriler
SELECT m.musteri_id, m.ad
FROM musteri m
LEFT JOIN siparis s ON s.musteri_id = m.musteri_id
WHERE s.musteri_id IS NULL;      -- eslesme olmadigini gosteren isaret

-- Ayni sonucun NOT EXISTS ile yazimi (genelde daha okunabilir ve NULL-guvenli)
SELECT m.musteri_id, m.ad
FROM musteri m
WHERE NOT EXISTS (
    SELECT 1 FROM siparis s WHERE s.musteri_id = m.musteri_id
);
```

Mülakatın En Sinsi Tuzağı: Satır Çoğalması (Fan-out)

Adayların en sık düştüğü ve raporlarda sessizce yanlış sayılar üreten hata budur. JOIN yaptığın sütun sağ tabloda benzersiz değilse, sol tablodaki satırlar çoğalır.

```SQL
-- musteri: 1 satir (Ahmet)
-- siparis: Ahmet'in 3 siparisi var

SELECT m.musteri_id, m.kredi_limiti
FROM musteri m
JOIN siparis s ON s.musteri_id = m.musteri_id;
-- Sonuc: Ahmet 3 KEZ doner, kredi_limiti 3 kez tekrarlanir.

-- TEHLIKE: simdi toplam alalim
SELECT SUM(m.kredi_limiti)
FROM musteri m
JOIN siparis s ON s.musteri_id = m.musteri_id;
-- Kredi limiti 3 katina cikti! Rapor sessizce yanlis.

-- COZUM: once toplulastir, sonra birlestir
```

```sql
SELECT m.musteri_id, m.kredi_limiti, s.siparis_adedi
FROM musteri m
LEFT JOIN (
    SELECT musteri_id, COUNT(*) AS siparis_adedi
    FROM siparis
    GROUP BY musteri_id
) s ON s.musteri_id = m.musteri_id;
```

Mülakatta şunu söylemek çok değerlidir: "JOIN yazmadan önce her zaman birleştirme anahtarının hangi tarafta benzersiz olduğunu, yani ilişkinin 1-1 mi, 1-N mi yoksa N-N mi olduğunu kontrol ederim. Bir fact tablosuna birden fazla 1-N tablo bağlamak, sayıları sessizce şişiren en yaygın hatadır."

Veritabanı Motoru JOIN'i Nasıl Uygular?

Senior seviye için, mantıksal JOIN türü ile fiziksel uygulama algoritmasının farklı şeyler olduğunu bilmek gerekir. Optimizer, execution plan'da şu üç algoritmadan birini seçer:

- **Nested Loop Join:** Dış tablodaki her satır için iç tabloda arama yapar. Küçük veri setlerinde ve iç tarafta uygun index varsa çok hızlıdır.
- **Hash Join:** Küçük tablodan bellekte bir hash tablosu kurar, büyük tabloyu üzerinden geçirir. Büyük, indexlenmemiş eşitlik birleştirmelerinde en yaygın seçimdir.
- **Merge Join:** Her iki taraf da birleştirme anahtarına göre sıralıysa, iki sıralı listeyi yan yana yürüterek birleştirir. Çok verimlidir ama sıralama maliyeti gerektirebilir.

Karşılaştırma Tablosu

Join Türü	Dönen Satırlar	Tipik Kullanım
INNER JOIN	Yalnızca her iki tarafta eşleşenler	Kesin eşleşme gereken raporlar
LEFT JOIN	Soldakilerin tamamı + eşleşen sağdakiler	Ana listeyi koruma, eksik veri tespiti
RIGHT JOIN	Sağdakilerin tamamı + eşleşen soldakiler	Nadir; genelde LEFT'e çevrilir
FULL OUTER JOIN	Her iki taraftaki tüm satırlar	İki kaynak arasında mutabakat
CROSS JOIN	Kartezyen çarpım (N × M)	Tarih/senaryo matrisi üretme

Self Join ve Cross Join

> **SORU:** *Self Join ve Cross Join nedir? Hangi gerçek senaryolarda kullanırsın?*

1. Mülakat Cevabı

> *"Self Join, bir tablonun kendisiyle birleştirilmesidir. Teknik olarak özel bir JOIN türü değildir; aynı tabloya iki farklı alias verip normal bir JOIN yazarız. Aynı tablo içinde satırlar arasında ilişki olduğunda kullanılır. En klasik örneği çalışan-yönetici hiyerarşisidir: her çalışanın yonetici_id sütunu yine aynı tablodaki başka bir çalışanı işaret eder.*
>
> *Cross Join ise iki tablo arasında kartezyen çarpım üretir; soldaki her satırı sağdaki her satırla eşleştirir. N satırlı ve M satırlı iki tablodan N×M satır döner. Bilinçli kullanımı, tüm olası kombinasyonları üretmek gerektiğinde ortaya çıkar: örneğin her mağaza için her günü içeren eksiksiz bir takvim iskeleti (date spine) oluşturmak ve satış olmayan günleri sıfırla raporlamak.*
>
> *Pratikte Cross Join çoğu zaman kazara oluşur: JOIN yazıp ON koşulunu unutmak veya eksik yazmak, milyonlarca satırlık bir patlamaya yol açar. Bu yüzden bilinçli kullanımlarda niyetimi belli etmek için açıkça CROSS JOIN yazarım."*

2. Detaylı Arka Plan

Self Join: Çalışan-Yönetici Hiyerarşisi

```SQL
-- Her calisani ve yoneticisinin adini listele.
-- Yoneticisi olmayanlar (CEO) da listede kalsin diye LEFT JOIN kullaniyoruz.
SELECT
    c.ad          AS calisan,
    y.ad          AS yonetici
FROM calisan c
LEFT JOIN calisan y ON y.calisan_id = c.yonetici_id;
```

Bir diğer klasik self join senaryosu, aynı tablodaki ardışık satırları karşılaştırmaktır (örneğin bir önceki günün satışıyla kıyaslama). Ancak modern SQL'de bu iş için self join yerine LAG() gibi window fonksiyonları çok daha verimlidir; mülakatta bunu belirtmek artı puandır.

Cross Join: Eksiksiz Takvim İskeleti (Date Spine)

Veri mühendisliğinde sık karşılaşılan bir problem: bir mağazanın satış yapmadığı günler satış tablosunda hiç satır olarak bulunmaz, dolayısıyla rapor o günleri atlar. Çözüm, tüm gün-mağaza kombinasyonlarını üretip satışları buna LEFT JOIN yapmaktır.

```SQL
-- Her magaza icin her gunu ureten iskelet, satis olmayan gunler 0 gorunur
SELECT
    t.tarih,
    m.magaza_id,
    COALESCE(SUM(s.tutar), 0) AS gunluk_ciro
FROM takvim t
CROSS JOIN magaza m
LEFT JOIN satis s
       ON s.tarih = t.tarih
      AND s.magaza_id = m.magaza_id
WHERE t.tarih BETWEEN '2024-01-01' AND '2024-01-31'
GROUP BY t.tarih, m.magaza_id
ORDER BY t.tarih, m.magaza_id;
```

Mülakat Notu: Kazara Cross Join

Bir sorgunun beklenenden çok daha uzun sürdüğü ve devasa satır döndürdüğü durumlarda ilk kontrol edilmesi gereken şey, JOIN koşulunun eksik olup olmadığıdır. Üç tabloyu birleştirirken ikisi arasında ON yazmayı unutmak sessiz bir kartezyen çarpım yaratır. Execution plan'da beklenmedik bir "Nested Loop (no join predicate)" görürsen sebebi budur.

Küme Operatörleri (UNION, UNION ALL, INTERSECT, EXCEPT)

> **SORU:** *UNION ile UNION ALL arasındaki fark nedir? Hangisi daha hızlıdır ve neden? INTERSECT ve EXCEPT ne işe yarar?*

1. Mülakat Cevabı

> *"Küme operatörleri, iki ayrı sorgunun sonuç kümesini alt alta veya kesişimli şekilde birleştirir. Kullanılabilmeleri için her iki sorgunun sütun sayısının aynı ve karşılıklı sütun tiplerinin uyumlu olması gerekir.*
>
> *UNION iki sonucu birleştirir ve tekrar eden satırları eler. UNION ALL ise birleştirir ama tekrarları elemez.*
>
> *Aradaki performans farkı önemlidir: UNION, tekrarları elemek için arka planda tüm sonucu sıralamak veya hash tablosu kurmak zorundadır; bu ek CPU ve bellek maliyeti demektir. UNION ALL ise sadece iki sonucu peş peşe ekler. Bu yüzden verinin zaten tekrarsız olduğunu biliyorsam veya tekrarların elenmesi gerekmiyorsa her zaman UNION ALL kullanırım; bu, veri hatlarında farkı hissedilen kolay bir optimizasyondur.*
>
> *INTERSECT her iki sorguda da bulunan ortak satırları, EXCEPT (Oracle'da MINUS) ise birinci sorguda olup ikincisinde olmayan satırları döndürür. EXCEPT özellikle iki tablo arasındaki farkı bulmak, yani veri mutabakatı yapmak için çok kullanışlıdır."*

2. Detaylı Arka Plan

Kullanım Örnekleri

```SQL
-- UNION ALL: iki yilin siparislerini birlestir (tekrar zaten yok, eleme gereksiz)
SELECT siparis_id, tutar FROM siparis_2023
UNION ALL
SELECT siparis_id, tutar FROM siparis_2024;

-- INTERSECT: hem 2023'te hem 2024'te alisveris yapan musteriler
SELECT musteri_id FROM siparis_2023
INTERSECT
SELECT musteri_id FROM siparis_2024;

-- EXCEPT: 2023'te alisveris yapip 2024'te yapmayanlar (churn analizi)
SELECT musteri_id FROM siparis_2023
EXCEPT
SELECT musteri_id FROM siparis_2024;
```

Veri Mutabakatı (Reconciliation) Deseni

İki sistem arasında verinin birebir aynı olup olmadığını kontrol etmek, veri mühendisliğinde günlük bir iştir. EXCEPT bunun için iki yönlü kullanılır:

```SQL
-- Kaynakta olup hedefte olmayanlar (eksik yuklenenler)
SELECT musteri_id, tutar FROM kaynak_tablo
EXCEPT
SELECT musteri_id, tutar FROM hedef_tablo

UNION ALL

-- Hedefte olup kaynakta olmayanlar (fazladan/bozuk kayitlar)
SELECT musteri_id, tutar FROM hedef_tablo
EXCEPT
SELECT musteri_id, tutar FROM kaynak_tablo;
```

```
-- Sonuc bos donuyorsa iki tablo birebir ayni demektir.
```

Mülakat Tuzağı: INTERSECT/EXCEPT de Tekrarları Eler

Aday genellikle yalnızca UNION'ın tekrar elediğini bilir. Oysa standart SQL'de INTERSECT ve EXCEPT de varsayılan olarak DISTINCT davranır. Tekrarları korumak isteniyorsa INTERSECT ALL / EXCEPT ALL kullanılır (PostgreSQL destekler, birçok motor desteklemez).

Bir diğer detay: SQL Server ve MySQL'de EXCEPT/INTERSECT desteği farklılık gösterir; MySQL bunları ancak 8.0.31 sonrası sürümlerde destekler. Daha taşınabilir bir çözüm NOT EXISTS ile anti-join yazmaktır.

JOIN ile Küme Operatörlerinin Farkı

Kavramsal olarak çok net bir ayrım vardır ve mülakatta bunu tek cümleyle ifade etmek etkilidir: "JOIN tabloları yatay olarak (sütun ekleyerek) birleştirir; küme operatörleri ise dikey olarak (satır ekleyerek) birleştirir."

Operatör	Davranış	Tekrarları Eler mi?
UNION	İki sonucu alt alta ekler	Evet (maliyetli)
UNION ALL	İki sonucu alt alta ekler	Hayır (hızlı)
INTERSECT	Her iki sonuçta da olan satırlar	Evet
EXCEPT / MINUS	Birincide olup ikincide olmayanlar	Evet

SQL: Gruplama ve İleri Teknikler

Aggregate Fonksiyonlar ve GROUP BY

> **SORU:** *Aggregate fonksiyonları ve GROUP BY'ı anlat. COUNT(*), COUNT(sütun) ve COUNT(DISTINCT sütun) arasındaki fark nedir?*

1. Mülakat Cevabı

> *"Aggregate (toplulaştırma) fonksiyonları birden fazla satırı tek bir özet değere indirger: COUNT satır sayar, SUM toplar, AVG ortalama alır, MIN ve MAX uç değerleri bulur.*
>
> *GROUP BY ise satırları belirtilen sütunlardaki değerlere göre gruplar ve her grup için aggregate fonksiyonların ayrı ayrı hesaplanmasını sağlar. Temel kural şudur: SELECT listesinde bulunan ve aggregate fonksiyon içinde olmayan her sütun, GROUP BY listesinde de yer almak zorundadır.*
>
> *COUNT'un üç varyantı arasındaki fark NULL davranışıdır. COUNT(*) tablodaki tüm satırları sayar, NULL olup olmadığına bakmaz. COUNT(sutun) yalnızca o sütunun NULL olmadığı satırları sayar. COUNT(DISTINCT sutun) ise o sütundaki NULL olmayan benzersiz değerlerin sayısını verir. Bu ayrım, raporlarda fark edilmesi zor hatalara yol açtığı için mülakatlarda sıkça sorulur."*

2. Detaylı Arka Plan

COUNT Varyantları Kod Üzerinde

```SQL
-- siparis tablosu: 10 satir, bunlarin 3'unde kupon_kodu NULL,
-- dolu olan 7 satirda ise yalnizca 4 farkli kupon kodu var.

SELECT
    COUNT(*)                    AS toplam_satir,      -- 10
    COUNT(kupon_kodu)           AS kuponlu_siparis,   -- 7
    COUNT(DISTINCT kupon_kodu)  AS farkli_kupon,      -- 4
    SUM(tutar)                  AS ciro,
    AVG(tutar)                  AS ortalama_sepet
FROM siparis;
```

Koşullu Toplulaştırma (Conditional Aggregation)

Mülakatlarda çok sevilen bir tekniktir: tek bir tablo taramasıyla birden fazla koşullu metrik hesaplamak. CASE WHEN ile aggregate fonksiyonlarını birleştiririz.

```SQL
-- Her musteri icin durum bazli siparis sayilarini TEK sorguda cikar
SELECT
    musteri_id,
    COUNT(*)                                                    AS toplam_siparis,
    SUM(CASE WHEN durum = 'TAMAMLANDI' THEN 1 ELSE 0 END)       AS tamamlanan,
    SUM(CASE WHEN durum = 'IPTAL'      THEN 1 ELSE 0 END)       AS iptal,
    SUM(CASE WHEN durum = 'TAMAMLANDI' THEN tutar ELSE 0 END)   AS gecerli_ciro,
    -- COUNT ile de yazilabilir: COUNT NULL saymaz
    COUNT(CASE WHEN durum = 'IPTAL' THEN 1 END)                 AS iptal_alternatif
FROM siparis
GROUP BY musteri_id;
```

Mülakat Tuzağı: GROUP BY ve Gizli Sütunlar

MySQL, tarihsel olarak (ONLY_FULL_GROUP_BY modu kapalıyken) GROUP BY listesinde olmayan sütunları SELECT'te kullanmaya izin verirdi ve rastgele bir satırın değerini döndürürdü. PostgreSQL, SQL Server ve Oracle bunu hata olarak reddeder.

Mülakatta "MySQL'de çalışıyordu ama PostgreSQL'de hata verdi" senaryosu sorulursa cevap budur: MySQL'in gevşek davranışı standart dışıdır ve deterministik olmayan sonuç ürettiği için tehlikelidir. Doğru çözüm sütunu ya GROUP BY'a eklemek ya da bir aggregate (MIN, MAX, ANY_VALUE) içine almaktır.

GROUPING SETS, ROLLUP ve CUBE

Raporlamada aynı sorguda hem detay hem ara toplam hem genel toplam istendiğinde, aynı tabloyu defalarca UNION ALL ile taramak yerine bu yapılar kullanılır. Bilmek senior seviye bir sinyaldir.

```SQL
-- Kategori bazinda, marka bazinda ve genel toplam: tek taramada
SELECT kategori, marka, SUM(tutar) AS ciro
FROM satis
GROUP BY ROLLUP (kategori, marka);
-- Uretilen gruplar: (kategori, marka), (kategori), () -> genel toplam
```

WHERE ve HAVING Farkı

> **SORU:** *WHERE ile HAVING arasındaki fark nedir? Performans açısından hangisini tercih etmelisin?*

1. Mülakat Cevabı

> *"İkisi de filtreleme yapar ancak sorgunun farklı aşamalarında devreye girerler.*
>
> *WHERE, gruplama gerçekleşmeden ÖNCE tek tek satırları filtreler. Bu yüzden aggregate fonksiyon içeremez; çünkü o aşamada henüz gruplar ve dolayısıyla toplamlar oluşmamıştır.*
>
> *HAVING ise GROUP BY tamamlandıktan SONRA, oluşan grupları filtreler. Aggregate fonksiyon kullanabilir; zaten var oluş amacı budur.*
>
> *Performans açısından altın kural şudur: Bir koşulu WHERE ile ifade edebiliyorsan, kesinlikle WHERE kullan. Çünkü WHERE satırları gruplama işleminden önce eler; bu da motorun daha az satırı sıralaması, gruplaması ve toplaması demektir. HAVING'e sadece gerçekten toplulaştırılmış bir değeri filtrelemen gerektiğinde başvurmalısın. İkisini aynı sorguda birlikte kullanmak da çok yaygındır ve genellikle en verimli yazımdır."*

2. Detaylı Arka Plan

İkisinin Birlikte Kullanımı

```SQL
-- 2024 yilinda 5'ten fazla siparis vermis musteriler
SELECT
    musteri_id,
    COUNT(*)   AS siparis_adedi,
    SUM(tutar) AS toplam_tutar
FROM siparis
WHERE siparis_tarihi >= '2024-01-01'      -- once satirlari daralt (ucuz)
  AND siparis_tarihi <  '2025-01-01'
GROUP BY musteri_id
HAVING COUNT(*) > 5                        -- sonra gruplari filtrele
ORDER BY toplam_tutar DESC;
```

Yaygın Hata: Satır Koşulunu HAVING'e Yazmak

```SQL
-- CALISIR ama VERIMSIZ: tum yillar gruplanir, sonra elenir
SELECT musteri_id, COUNT(*)
FROM siparis
GROUP BY musteri_id, YEAR(siparis_tarihi)
HAVING YEAR(siparis_tarihi) = 2024;

-- DOGRU: filtreyi WHERE'e al, gruplanacak veri en basta azalsin
SELECT musteri_id, COUNT(*)
FROM siparis
WHERE siparis_tarihi >= '2024-01-01'
  AND siparis_tarihi <  '2025-01-01'
GROUP BY musteri_id;
```

İkinci yazım ayrıca SARGable'dır: sütuna fonksiyon uygulanmadığı için siparis_tarihi üzerindeki index kullanılabilir. YEAR(siparis_tarihi) = 2024 yazımı index'i kullanılamaz hale getirir.

Mülakat Sorusu: GROUP BY olmadan HAVING kullanılabilir mi?

Evet, kullanılabilir. GROUP BY yoksa tüm tablo tek bir grup olarak kabul edilir ve HAVING bu tek grubu filtreler. Sonuç ya bir satır döner ya da hiç dönmez.

```SQL
```

```
-- Tablodaki toplam ciro 1 milyonu gecti mi? Gectiyse degeri getir.
SELECT SUM(tutar) AS toplam_ciro
FROM siparis
HAVING SUM(tutar) > 1000000;
```

Karşılaştırma Tablosu

Kriter	WHERE	HAVING
Çalıştığı Aşama	GROUP BY'dan önce	GROUP BY'dan sonra
Neyi Filtreler	Tek tek satırları	Oluşmuş grupları
Aggregate Kullanımı	Kullanamaz	Kullanabilir
Index Kullanımı	Kullanabilir	Kullanamaz (veri zaten işlenmiş)
Performans	Ucuz, önce eler	Pahalı, gruplama sonrası eler

Alt Sorgular (Subquery): Correlated ve Non-Correlated

> **SORU:** *Correlated ve non-correlated subquery arasındaki fark nedir? IN, EXISTS ve JOIN arasında nasıl seçim yaparsın?*

1. Mülakat Cevabı

> *"Subquery, başka bir sorgunun içine yerleştirilmiş sorgudur. İki türü vardır.*
>
> *Non-correlated (bağımsız) subquery, dış sorgudan hiçbir sütuna referans vermez. Tek başına çalıştırılabilir ve motor onu genellikle bir kez hesaplayıp sonucu dış sorguda kullanır.*
>
> *Correlated (bağımlı) subquery ise dış sorgunun bir sütununa referans verir. Bu yüzden tek başına çalıştırılamaz ve kavramsal olarak dış sorgunun her satırı için yeniden değerlendirilir. Bu, büyük tablolarda ciddi performans sorunu yaratabilir.*
>
> *Seçim yaparken şu yaklaşımı izlerim: Sadece varlık kontrolü yapıyorsam EXISTS kullanırım, çünkü ilk eşleşmeyi bulduğunda durur (short-circuit) ve NULL değerlerden etkilenmez. Yokluk kontrolü için NOT IN yerine kesinlikle NOT EXISTS tercih ederim, çünkü NOT IN listesinde tek bir NULL varsa sorgu hiç satır döndürmez. Sağ taraftan sütun da almam gerekiyorsa JOIN yazarım. Modern optimizer'lar bu üçünü çoğu zaman aynı plana dönüştürür, ancak okunabilirlik ve NULL güvenliği açısından tercihim nettir."*

2. Detaylı Arka Plan

İki Türün Kod Üzerinde Farkı

```SQL
-- NON-CORRELATED: ic sorgu bagimsiz, tek basina calisir
SELECT ad, maas
FROM calisan
WHERE maas > (SELECT AVG(maas) FROM calisan);
-- Ortalama bir kez hesaplanir, sonra tum satirlarla karsilastirilir.

-- CORRELATED: ic sorgu disaridaki c.departman_id'ye bagimli
SELECT c.ad, c.maas
FROM calisan c
WHERE c.maas > (
    SELECT AVG(c2.maas)
    FROM calisan c2
    WHERE c2.departman_id = c.departman_id   -- <-- dis sorguya referans
);
-- Her calisan icin kendi departmaninin ortalamasi hesaplanir.
```

Correlated Subquery Yerine Window Fonksiyonu

Yukarıdaki correlated sorgu, tabloyu iki kez taramak zorundadır. Aynı iş bir window fonksiyonuyla tek taramada yapılabilir ve mülakatta bu alternatifi sunmak çok değerlidir:

```SQL
-- Tek taramada, cok daha verimli
SELECT ad, maas, departman_id
FROM (
    SELECT
        ad, maas, departman_id,
        AVG(maas) OVER (PARTITION BY departman_id) AS dept_ortalama
    FROM calisan
) t
WHERE maas > dept_ortalama;
```

Subquery'nin Kullanılabileceği Yerler

- **SELECT içinde (scalar subquery):** Tek bir değer döndürmek zorundadır. Birden fazla satır dönerse sorgu hata verir.
- **FROM içinde (derived table / inline view):** Bir alt sonuç kümesini geçici tablo gibi kullanmak için. Çoğu motorda alias vermek zorunludur.
- **WHERE / HAVING içinde:** IN, EXISTS, ANY, ALL veya karşılaştırma operatörleriyle filtreleme için.

IN, EXISTS ve JOIN Karşılaştırması

Yaklaşım	Ne Zaman Kullanılır	Dikkat Edilecek Nokta
IN (alt sorgu)	Küçük, sabit veya tekrarsız listelerde	NULL içeren listede NOT IN felakete yol açar
EXISTS	Yalnızca varlık/yokluk kontrolünde	NULL güvenlidir; ilk eşleşmede durur
JOIN	Sağ tablodan sütun da gerekiyorsa	Anahtar benzersiz değilse satır çoğalması riski

Mülakat Tuzağı: EXISTS içinde SELECT ne yazmalı?

EXISTS içindeki SELECT listesinin hiçbir önemi yoktur; motor yalnızca satırın var olup olmadığına bakar. SELECT 1, SELECT * veya SELECT NULL yazmak arasında performans farkı yoktur. Alışkanlık olarak SELECT 1 yazılır çünkü niyeti en net ifade eden yazımdır.

CTE (Common Table Expressions) ve Recursive CTE

> **SORU:** *CTE nedir ve subquery yerine neden tercih edersin? Recursive CTE ne işe yarar?*

1. Mülakat Cevabı

> *"CTE, WITH anahtar kelimesiyle tanımlanan, sorgunun başında isimlendirilmiş geçici bir sonuç kümesidir. Yalnızca ait olduğu sorgu boyunca yaşar.*
>
> *Subquery yerine tercih etmemin üç sebebi var. Birincisi okunabilirlik: iç içe geçmiş üç seviyeli subquery yerine, adım adım isimlendirilmiş mantıksal bloklar yazarım ve sorgu yukarıdan aşağıya okunabilir hale gelir. İkincisi modülerlik ve tekrar kullanım: aynı CTE'yi sorgu içinde birden fazla yerde referans verebilirim, aynı subquery'yi kopyalamak zorunda kalmam. Üçüncüsü hata ayıklama kolaylığı: karmaşık bir dönüşümü CTE'lere böldüğümde, her adımı tek tek çalıştırıp nerede bozulduğunu görebilirim.*
>
> *Recursive CTE ise kendi kendine referans veren özel bir CTE türüdür. Hiyerarşik ve ağaç yapısındaki verileri gezmek için kullanılır: organizasyon şeması, ürün ağacı (bill of materials), kategori ağacı veya bir grafikteki yollar. İki parçadan oluşur: başlangıç noktasını veren anchor sorgu ve kendini çağıran recursive parça; ikisi UNION ALL ile birleştirilir."*

2. Detaylı Arka Plan

Okunabilirlik Farkı

```SQL
-- CTE ile: adim adim, okunabilir
WITH aktif_musteri AS (
    SELECT musteri_id, ad
    FROM musteri
    WHERE durum = 'AKTIF'
),
musteri_ciro AS (
    SELECT musteri_id, SUM(tutar) AS toplam_ciro
    FROM siparis
    WHERE siparis_tarihi >= '2024-01-01'
    GROUP BY musteri_id
)
SELECT a.ad, c.toplam_ciro
FROM aktif_musteri a
JOIN musteri_ciro c ON c.musteri_id = a.musteri_id
WHERE c.toplam_ciro > 10000
ORDER BY c.toplam_ciro DESC;
```

Recursive CTE: Organizasyon Hiyerarşisi

```SQL
WITH RECURSIVE hiyerarsi AS (
    -- ANCHOR: baslangic noktasi (yoneticisi olmayan, yani CEO)
    SELECT
        calisan_id,
        ad,
        yonetici_id,
        1 AS seviye,
        CAST(ad AS VARCHAR(1000)) AS yol
    FROM calisan
    WHERE yonetici_id IS NULL

    UNION ALL

    -- RECURSIVE: her seviyede bir alt kademeyi ekle
    SELECT
        c.calisan_id,
```

```
        c.ad,
        c.yonetici_id,
        h.seviye + 1,
        CAST(h.yol || ' > ' || c.ad AS VARCHAR(1000))
    FROM calisan c
    JOIN hiyerarsi h ON c.yonetici_id = h.calisan_id
)
SELECT seviye, ad, yol
FROM hiyerarsi
ORDER BY yol;
```

Not: SQL Server'da RECURSIVE anahtar kelimesi yazılmaz, yalnızca WITH kullanılır. PostgreSQL ve MySQL 8+ ise WITH RECURSIVE bekler. Sonsuz döngü riskine karşı SQL Server'da OPTION (MAXRECURSION n), PostgreSQL'de ise bir seviye sınırı koşulu eklenir.

Mülakat Tuzağı: "CTE performansı artırır mı?"

Adayların çoğu "Evet, sonucu bir kere hesaplayıp saklar" der. Bu yanlıştır ve mülakatçının beklediği cevap değildir.

Kritik cevap: CTE kendi başına bir performans optimizasyonu değildir; öncelikle bir okunabilirlik aracıdır. Çoğu motor CTE'yi inline eder, yani sorguya gömüp tek bir plan üretir. PostgreSQL 12 öncesinde CTE'ler zorunlu olarak materialize edilirdi (optimization fence); 12 ve sonrasında varsayılan olarak inline edilir, materialize edilmesini istiyorsan WITH ... AS MATERIALIZED yazman gerekir.

Bir CTE'yi sorguda birden fazla kez referans verirsen, bazı motorlar onu her referans için yeniden hesaplar. Gerçekten bir kez hesaplanıp saklanması gerekiyorsa doğru araç geçici tablo (temp table) veya materialized view'dir.

CTE, Temp Table ve View Karşılaştırması

Yapı	Yaşam Süresi	Ne Zaman Kullanılır
CTE	Yalnızca o sorgu boyunca	Okunabilirlik, tek sorguda çok adımlı mantık
Temp Table	Oturum/işlem boyunca	Sonuç birden çok sorguda tekrar kullanılacaksa, index gerekiyorsa
View	Kalıcı (tanım olarak)	Tekrar eden iş mantığının merkezi tanımı
Materialized View	Kalıcı (veri olarak)	Pahalı toplulaştırmanın önceden hesaplanması

Window (Pencere) Fonksiyonları

> **SORU:** *Window fonksiyonlarını anlat. ROW_NUMBER, RANK ve DENSE_RANK arasındaki fark nedir? GROUP BY'dan farkı nedir?*

1. Mülakat Cevabı

> *"Window fonksiyonları, geçerli satırla ilişkili bir satır kümesi (pencere) üzerinde hesaplama yapar ancak satırları birleştirip yok etmez. GROUP BY'dan en temel farkı budur: GROUP BY her grup için tek satır döndürür, window fonksiyonu ise tüm satırları korur ve her satıra hesaplanan değeri ek bir sütun olarak ekler.*
>
> *Sözdizimi OVER (PARTITION BY ... ORDER BY ...) şeklindedir. PARTITION BY veriyi gruplara böler, ORDER BY ise grup içindeki sıralamayı belirler.*
>
> *Sıralama fonksiyonlarının farkı, eşit değerlerde ortaya çıkar. ROW_NUMBER her satıra benzersiz ardışık numara verir; eşitlik olsa bile 1, 2, 3, 4 şeklinde gider. RANK eşit değerlere aynı sırayı verir ama sonrasında boşluk bırakır: 1, 2, 2, 4. DENSE_RANK ise aynı sırayı verir ve boşluk bırakmaz: 1, 2, 2, 3.*
>
> *Ayrıca LAG ve LEAD ile bir önceki ve bir sonraki satırın değerine erişebilir, SUM veya AVG'yi OVER ile kullanarak kümülatif toplam ve hareketli ortalama hesaplayabilirim. Veri mühendisliğinde en sık kullandığım kalıp, ROW_NUMBER ile tekrar eden kayıtlardan yalnızca en güncelini tutmaktır."*

2. Detaylı Arka Plan

Üç Sıralama Fonksiyonunun Farkı

```SQL
-- Maaslar: 9000, 7000, 7000, 5000
SELECT
    ad,
    maas,
    ROW_NUMBER() OVER (ORDER BY maas DESC) AS row_num,    -- 1, 2, 3, 4
    RANK()       OVER (ORDER BY maas DESC) AS rank_,      -- 1, 2, 2, 4
    DENSE_RANK() OVER (ORDER BY maas DESC) AS dense_rank_ -- 1, 2, 2, 3
FROM calisan;
```

Seçim kuralı: Kesin olarak tek bir satır seçmen gerekiyorsa (örneğin deduplication) ROW_NUMBER kullan. "En yüksek 3 farklı maaş" gibi eşitlikleri birlikte saymak istiyorsan DENSE_RANK kullan. Yarışma sıralaması mantığı (iki birinci varsa ikincilik yok) istiyorsan RANK kullan.

En Sık Sorulan Görev: Deduplication (Tekrar Eden Kayıtları Temizleme)

Veri mühendisliği mülakatlarının klasik sorusu: "Bir tabloda aynı iş anahtarına sahip birden fazla kayıt var; her anahtar için yalnızca en güncel kaydı tut."

```SQL
WITH sirali AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY musteri_id           -- is anahtari
            ORDER BY guncelleme_tarihi DESC,  -- en guncel once
                     kayit_id DESC            -- esitlik icin belirleyici (deterministik)
        ) AS rn
    FROM musteri_ham
)
SELECT * FROM sirali WHERE rn = 1;
```

Kritik detay: ORDER BY içine ikinci bir belirleyici (tie-breaker) sütun eklemek önemlidir. Aksi halde iki kayıt aynı güncelleme tarihine sahipse hangisinin seçileceği belirsizdir ve hattın her çalışmasında farklı sonuç üretebilir; bu da idempotency'yi bozar.

Snowflake ve BigQuery'de bu kalıp QUALIFY ile daha kısa yazılır:

```sql
-- Snowflake / BigQuery
SELECT *
FROM musteri_ham
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY musteri_id ORDER BY guncelleme_tarihi DESC
) = 1;
```

LAG ve LEAD: Satırlar Arası Karşılaştırma

```sql
-- Her ayin cirosunu bir onceki ayla karsilastir (buyume orani)
SELECT
    ay,
    ciro,
    LAG(ciro) OVER (ORDER BY ay)                        AS onceki_ay_ciro,
    ciro - LAG(ciro) OVER (ORDER BY ay)                 AS fark,
    ROUND(
        100.0 * (ciro - LAG(ciro) OVER (ORDER BY ay))
        / NULLIF(LAG(ciro) OVER (ORDER BY ay), 0), 2
    )                                                   AS buyume_yuzde
FROM aylik_ciro;
```

Kümülatif Toplam ve Frame (Çerçeve) Kavramı

ORDER BY içeren bir window fonksiyonunda varsayılan çerçeve RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW'dur; yani başlangıçtan geçerli satıra kadar. Bu, kümülatif toplamı otomatik verir.

```sql
SELECT
    tarih,
    tutar,
    -- Kumulatif toplam (running total)
    SUM(tutar) OVER (ORDER BY tarih)                        AS kumulatif,
    -- Son 7 satirlik hareketli ortalama
    AVG(tutar) OVER (
        ORDER BY tarih
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    )                                                       AS hareketli_ort_7,
    -- Partition genelinde toplam (ORDER BY yok -> tum pencere)
    SUM(tutar) OVER (PARTITION BY magaza_id)                AS magaza_toplam
FROM satis;
```

Mülakat detayı: ROWS fiziksel satır sayar, RANGE ise değer eşitliğine göre çalışır. Aynı tarihte birden çok kayıt varsa RANGE hepsini tek grup sayar, ROWS ise tek tek ilerler. Bu farkı bilmek ileri seviye bir sinyaldir.

Grup İçinde İlk N Kaydı Getirmek (Top-N per Group)

```sql
-- Her kategorideki en cok satan 3 urun
WITH sirali AS (
    SELECT
        kategori, urun_adi, satis_adedi,
        ROW_NUMBER() OVER (
            PARTITION BY kategori ORDER BY satis_adedi DESC
        ) AS sira
    FROM urun_satis
```

```
)
SELECT kategori, urun_adi, satis_adedi
FROM sirali
WHERE sira <= 3
ORDER BY kategori, sira;
```

Mülakat Tuzağı: Window Fonksiyonu WHERE İçinde Kullanılabilir mi?

Hayır. Çalıştırma sırası hatırlanmalı: window fonksiyonları SELECT aşamasında hesaplanır, WHERE ise ondan önce çalışır. Bu yüzden window fonksiyonunun sonucuna göre filtreleme yapmak için sorguyu bir CTE veya alt sorguya sarmak (ya da QUALIFY kullanmak) gerekir. Yukarıdaki tüm örneklerin neden CTE ile yazıldığının sebebi budur.

Koşullu Mantık: CASE WHEN

> **SORU:** *CASE WHEN nerelerde kullanılır? SQL'de pivot işlemini nasıl yaparsın?*

1. Mülakat Cevabı

> *"CASE WHEN, SQL'in if-else yapısıdır. Koşulları sırayla değerlendirir, ilk sağlanan koşulun sonucunu döndürür ve hiçbiri sağlanmazsa ELSE değerini verir. ELSE yazılmazsa varsayılan olarak NULL döner.*
>
> *Dört ana kullanım alanı vardır. Birincisi değer eşleme ve kategorilendirme: sayısal bir alanı segmentlere ayırmak gibi. İkincisi koşullu toplulaştırma: SUM veya COUNT içine CASE koyarak tek taramada birden fazla metrik hesaplamak. Üçüncüsü pivot işlemi: satırları sütuna çevirmek. Dördüncüsü ise ORDER BY içinde özel sıralama mantığı tanımlamak.*
>
> *SQL'de pivot yapmanın en taşınabilir yolu, GROUP BY ile birlikte her hedef sütun için bir CASE WHEN yazmaktır. SQL Server'daki PIVOT operatörü veya Snowflake'in PIVOT fonksiyonu daha kısadır ama motora özgüdür; CASE yaklaşımı her yerde çalışır."*

2. Detaylı Arka Plan

İki Yazım Biçimi

```SQL
-- Searched CASE (esnek, her turlu kosul yazilabilir)
SELECT
    ad,
    maas,
    CASE
        WHEN maas >= 50000 THEN 'Yuksek'
        WHEN maas >= 30000 THEN 'Orta'
        ELSE 'Baslangic'
    END AS maas_segmenti
FROM calisan;

-- Simple CASE (yalnizca esitlik kontrolu, daha kisa)
SELECT
    siparis_id,
    CASE durum
        WHEN 'P' THEN 'Beklemede'
        WHEN 'C' THEN 'Tamamlandi'
        WHEN 'X' THEN 'Iptal'
        ELSE 'Bilinmiyor'
    END AS durum_adi
FROM siparis;
```

Önemli detay: Koşullar yukarıdan aşağıya sırayla değerlendirilir ve ilk eşleşmede durur. Bu yüzden yukarıdaki örnekte önce 50000, sonra 30000 kontrol edilmelidir. Sıra ters yazılırsa 60000 maaş "Orta" olarak etiketlenir; bu, mülakatta sorulan klasik bir mantık hatasıdır.

Pivot: Satırları Sütuna Çevirmek

```SQL
-- Kaynak: her satirda bir ay ve o ayin cirosu
-- Hedef: her magaza tek satir, aylar sutun

SELECT
    magaza_id,
    SUM(CASE WHEN ay = 1 THEN ciro ELSE 0 END) AS ocak,
    SUM(CASE WHEN ay = 2 THEN ciro ELSE 0 END) AS subat,
    SUM(CASE WHEN ay = 3 THEN ciro ELSE 0 END) AS mart,
    SUM(ciro)                                  AS ceyrek_toplam
FROM aylik_satis
WHERE yil = 2024
```

```
GROUP BY magaza_id;
```

Bu yaklaşımın sınırı, sütun isimlerinin sorguya sabit yazılması gerekmesidir. Sütun sayısı veriye göre değişkense (dinamik pivot), sorguyu string olarak üretip çalıştırmak (dynamic SQL) veya pivot işini raporlama katmanına bırakmak gerekir.

ORDER BY İçinde CASE: Özel Sıralama

```SQL
-- Durumlari alfabetik degil, is mantigina gore sirala
SELECT siparis_id, durum
FROM siparis
ORDER BY
    CASE durum
        WHEN 'ACIL'       THEN 1
        WHEN 'BEKLEMEDE'  THEN 2
        WHEN 'TAMAMLANDI' THEN 3
        ELSE 4
    END,
    siparis_tarihi DESC;
```

CASE ve NULL: Sık Yapılan Hata

```SQL
-- YANLIS: WHEN durum = NULL asla TRUE olmaz
CASE durum WHEN NULL THEN 'Bos' ELSE 'Dolu' END   -- her zaman 'Dolu'

-- DOGRU
CASE WHEN durum IS NULL THEN 'Bos' ELSE 'Dolu' END

-- Sadece NULL degistirmek icin daha okunabilir alternatif:
COALESCE(durum, 'Bos')
```

SQL: Veri Değiştirme ve Performans

DML, DDL, DCL ve TCL

> **SORU:** *DML ve DDL arasındaki fark nedir? SQL komutları hangi kategorilere ayrılır?*

1. Mülakat Cevabı

> *"SQL komutları görevlerine göre dört ana kategoriye ayrılır.*
>
> *DML (Data Manipulation Language), tabloların İÇİNDEKİ veriyi değiştirir: INSERT, UPDATE, DELETE ve okuma tarafında SELECT. Bu komutlar bir transaction içinde çalışır, dolayısıyla COMMIT edilmeden geri alınabilirler (ROLLBACK).*
>
> *DDL (Data Definition Language), veritabanının YAPISINI tanımlar ve değiştirir: CREATE, ALTER, DROP, TRUNCATE. Çoğu veritabanında DDL komutları örtük commit (implicit commit) tetikler; yani çalıştırıldığı anda kalıcı hale gelir ve öncesindeki açık transaction'ı da commit eder.*
>
> *DCL (Data Control Language) yetkilendirme ile ilgilenir: GRANT ve REVOKE. TCL (Transaction Control Language) ise transaction yönetimini sağlar: COMMIT, ROLLBACK ve SAVEPOINT.*
>
> *Mülakatta en çok sorulan ayrım DML ile DDL arasındaki transaction davranışıdır: DELETE bir DML olduğu için geri alınabilirken, TRUNCATE bir DDL olduğu için birçok veritabanında geri alınamaz."*

2. Detaylı Arka Plan

Kategori Tablosu

Kategori	Komutlar	Amaç ve Davranış
DML	INSERT, UPDATE, DELETE, (SELECT)	Veriyi değiştirir; transaction'a tabidir, rollback edilebilir
DDL	CREATE, ALTER, DROP, TRUNCATE	Yapıyı değiştirir; çoğu motorda örtük commit yapar
DCL	GRANT, REVOKE	Yetki verir veya geri alır
TCL	COMMIT, ROLLBACK, SAVEPOINT	Transaction sınırlarını yönetir

MERGE (UPSERT): Veri Mühendisliğinin Vazgeçilmezi

Standart DML komutlarının yanında, veri ambarı yüklemelerinde en kritik komut MERGE'dür. Kaynak ile hedefi karşılaştırıp eşleşen kayıtları günceller, eşleşmeyenleri ekler. Bu, idempotent yükleme yazmanın temel aracıdır.

```sql
MERGE INTO hedef_musteri AS h
USING kaynak_musteri AS k
   ON h.musteri_id = k.musteri_id
WHEN MATCHED AND h.guncelleme_tarihi < k.guncelleme_tarihi THEN
    UPDATE SET h.ad    = k.ad,
               h.email = k.email,
               h.guncelleme_tarihi = k.guncelleme_tarihi
WHEN NOT MATCHED THEN
    INSERT (musteri_id, ad, email, guncelleme_tarihi)
    VALUES (k.musteri_id, k.ad, k.email, k.guncelleme_tarihi);
```

Motora göre sözdizimi değişir: PostgreSQL'de INSERT ... ON CONFLICT DO UPDATE, MySQL'de INSERT ... ON DUPLICATE KEY UPDATE aynı işi görür. Mülakatta "Aynı veriyi iki kez yüklersem ne olur?" sorusuna cevabın MERGE veya partition overwrite olmalıdır.

TRUNCATE, DELETE ve DROP Farkı

> **SORU:** *TRUNCATE, DELETE ve DROP arasındaki farklar nelerdir? TRUNCATE geri alınabilir mi?*

1. Mülakat Cevabı

> *"Üçü de veri siler ama kapsamları ve çalışma biçimleri tamamen farklıdır.*
>
> *DELETE bir DML komutudur. Satırları tek tek siler, her silme işlemini transaction log'una yazar ve WHERE koşulu ile seçici silme yapabilir. Transaction içinde olduğu için geri alınabilir ve tabloya tanımlı DELETE trigger'ları tetiklenir. Yavaştır ama kontrollüdür. Tablonun yapısı ve identity sayacı korunur.*
>
> *TRUNCATE bir DDL komutudur. Satırları tek tek silmez; tablonun veri sayfalarını (data pages) topluca serbest bırakır. Bu yüzden çok daha hızlıdır ve minimum log üretir. WHERE koşulu alamaz, tabloyu tamamen boşaltır. Trigger'ları tetiklemez ve genellikle identity/auto-increment sayacını sıfırlar. Tabloya işaret eden bir Foreign Key varsa çalışmaz.*
>
> *DROP ise tabloyu tamamen yok eder: veriyi, yapıyı, index'leri, kısıtları ve izinleri siler. Tablo artık veritabanında mevcut değildir.*
>
> *Geri alınabilirlik konusu motora bağlıdır: SQL Server ve PostgreSQL'de TRUNCATE bir transaction bloğu içinde çalıştırılırsa ROLLBACK edilebilir. Oracle ve MySQL'de ise TRUNCATE örtük commit yaptığı için geri alınamaz."*

2. Detaylı Arka Plan

Karşılaştırma Tablosu

Kriter	DELETE	TRUNCATE	DROP
Kategori	DML	DDL	DDL
WHERE ile seçici silme	Evet	Hayır	Anlamsız
Hız	Yavaş (satır satır)	Çok hızlı (sayfa serbest bırakır)	Çok hızlı
Transaction Log	Her satır loglanır	Minimum loglama	Minimum loglama
Trigger tetikler mi?	Evet	Hayır	Hayır
Identity sayacı	Korunur	Sıfırlanır	Tablo ile birlikte gider
Tablo yapısı	Kalır	Kalır	Silinir
Geri alınabilir mi?	Evet	Motora bağlı	Genelde hayır

Mülakat Senaryosu: 500 Milyon Satırlık Tabloyu Boşaltmak

Mülakatçı sorar: "500 milyon satırlık bir staging tablosunu her gün boşaltıyorsun. DELETE mi TRUNCATE mi kullanırsın?"

Kritik cevap: "Kesinlikle TRUNCATE. DELETE her satır için transaction log kaydı üretir; bu hem log dosyasını şişirir hem de saatler sürebilir, üstelik tablo boyunca kilit tutar. TRUNCATE ise sayfa tahsislerini serbest bıraktığı için saniyeler içinde tamamlanır. Eğer tabloya Foreign Key referansı varsa TRUNCATE çalışmaz; o durumda ya kısıtı geçici olarak kaldırırım ya da partition kullanıyorsam ilgili partition'ı DROP ederim ki en verimli yöntem budur."

Bir başka profesyonel alternatif de şudur: yeni veriyi ayrı bir tabloya yükleyip tabloları takas etmek (atomic swap / partition switch). Böylece hedef tablo hiçbir an boş görünmez ve kesinti yaşanmaz.

Index'ler: Clustered ve Non-Clustered

> **SORU:** *Index nedir ve nasıl çalışır? Clustered ile Non-Clustered index arasındaki fark nedir? Index'in dezavantajı var mıdır?*

1. Mülakat Cevabı

> *"Index, bir tablodaki veriye hızlı erişim sağlamak için oluşturulan ek bir veri yapısıdır. Kitabın arkasındaki dizine benzer: tüm sayfaları taramak yerine doğrudan ilgili sayfaya gidersin. Çoğu ilişkisel veritabanında B-Tree (dengeli ağaç) yapısıyla tutulur; bu sayede arama, ekleme ve silme işlemleri logaritmik zamanda gerçekleşir.*
>
> *Clustered index, tablodaki satırların fiziksel sıralamasını belirler. Veri, index'in kendisinde yapraklarda saklanır; yani tablonun kendisi index'tir. Bu yüzden bir tabloda yalnızca bir tane clustered index olabilir. SQL Server'da Primary Key varsayılan olarak clustered index oluşturur, InnoDB'de ise Primary Key her zaman clustered'dır.*
>
> *Non-clustered index ise ayrı bir yapıdır; içinde indexlenen sütunun değerlerini ve asıl satıra işaret eden bir işaretçiyi tutar. Bir tabloda birden fazla non-clustered index olabilir. Aranan sütun index'te bulunsa bile SELECT'te istenen diğer sütunlar orada yoksa, motor asıl satıra gitmek zorunda kalır; buna key lookup denir ve maliyetlidir.*
>
> *Dezavantajı ise şudur: Index okumayı hızlandırır ama yazmayı yavaşlatır. Her INSERT, UPDATE ve DELETE işleminde ilgili tüm index'lerin de güncellenmesi gerekir. Ayrıca disk alanı tüketir. Bu yüzden gereğinden fazla index eklemek, özellikle yazma yoğun OLTP sistemlerinde ciddi performans kaybı yaratır."*

2. Detaylı Arka Plan

Covering Index ve Key Lookup

Bir index, sorgunun ihtiyaç duyduğu tüm sütunları içeriyorsa motor hiç asıl tabloya gitmez; buna covering index (kapsayan index) denir ve mümkün olan en hızlı senaryodur.

```SQL
-- Sorgu:
SELECT musteri_id, siparis_tarihi, tutar
FROM siparis
WHERE musteri_id = 42;

-- Yetersiz index: musteri_id bulunur ama tutar icin asil satira gidilir (key lookup)
CREATE INDEX ix_siparis_musteri ON siparis (musteri_id);

-- Covering index: gerekli tum sutunlar index'te -> tabloya hic gidilmez
CREATE INDEX ix_siparis_musteri_cover
    ON siparis (musteri_id)
    INCLUDE (siparis_tarihi, tutar);   -- SQL Server sozdizimi
-- PostgreSQL: CREATE INDEX ... ON siparis (musteri_id) INCLUDE (siparis_tarihi, tutar);
```

Composite Index ve Soldan Başlama (Leftmost Prefix) Kuralı

Birden fazla sütun içeren index'lerde sütun sırası kritiktir. Index yalnızca soldan başlayarak kullanılabilir.

```SQL
CREATE INDEX ix ON siparis (musteri_id, siparis_tarihi, durum);

-- Index KULLANILIR:
WHERE musteri_id = 42
WHERE musteri_id = 42 AND siparis_tarihi >= '2024-01-01'
WHERE musteri_id = 42 AND siparis_tarihi >= '2024-01-01' AND durum = 'X'

-- Index KULLANILMAZ (veya cok verimsiz kullanilir):
```

```
WHERE siparis_tarihi >= '2024-01-01'     -- ilk sutun atlandi
WHERE durum = 'X'                        -- ilk iki sutun atlandi
```

Kural: En seçici (selective) ve eşitlikle filtrelenen sütunlar sola, aralık (range) filtrelenen sütunlar sağa yazılır.

Index'in Kullanılmadığı Durumlar (SARGability)

Bir koşulun index kullanabilmesi için SARGable (Search ARGument able) olması gerekir. Aşağıdaki durumlar index'i devre dışı bırakır ve mülakatta sıkça sorulur:

- **Sütuna fonksiyon uygulamak:** WHERE YEAR(tarih) = 2024 index kullanamaz. Doğrusu: WHERE tarih >= '2024-01-01' AND tarih < '2025-01-01'.
- **Başta joker karakter:** WHERE ad LIKE '%mehmet' index kullanamaz çünkü B-Tree sıralaması baştan başlar. LIKE 'mehmet%' ise kullanabilir.
- **Örtük tip dönüşümü:** Sütun VARCHAR iken WHERE kod = 123 yazmak motoru sütunu dönüştürmeye zorlar ve index'i devre dışı bırakır.
- **Sütun üzerinde aritmetik:** WHERE fiyat * 1.2 > 100 yerine WHERE fiyat > 100 / 1.2 yazılmalıdır.
- **Düşük seçicilik:** Cinsiyet gibi yalnızca iki değer alan bir sütunda index çoğu zaman kullanılmaz; motor tam tarama yapmayı daha ucuz bulur.

Karşılaştırma Tablosu

Özellik	Clustered Index	Non-Clustered Index
Veri saklama	Veri index'in yapraklarındadır (tablo = index)	Ayrı yapı; satıra işaretçi tutar
Tablo başına adet	En fazla 1	Birden fazla olabilir
Fiziksel sıralama	Belirler	Belirlemez
Aralık sorguları	Çok verimli (veri yan yana)	Daha az verimli
Ek maliyet	Ekstra alan gerekmez	Ayrı alan tüketir, key lookup riski

Sorgu Optimizasyonu

> **SORU:** *Üretimde çok yavaş çalışan bir sorgu var. Bunu nasıl analiz eder ve optimize edersin? Adım adım anlat.*

1. Mülakat Cevabı

> *"Tahmin ederek değil, ölçerek başlarım. İzlediğim yöntem şudur:*
>
> *Önce sorgunun yürütme planını (EXPLAIN veya Execution Plan) alırım. Burada aradığım şeyler: tam tablo taraması (full table scan) yapılan yerler, tahmini satır sayısı ile gerçek satır sayısı arasındaki büyük sapmalar (bu genellikle bayat istatistiklere işaret eder), pahalı sıralama ve hash işlemleri, ve beklenmedik nested loop join'ler.*
>
> *Ardından darboğazı belirleyip şu sırayla müdahale ederim: Filtreleri mümkün olduğunca erken uygularım ve SARGable hale getiririm, yani sütuna fonksiyon uygulamaktan kaçınırım. Gereksiz sütunları kaldırırım; SELECT * yerine sadece ihtiyacım olan sütunları yazarım çünkü bu hem I/O'yu azaltır hem de covering index kullanımına imkan verir. Uygun index'in var olup olmadığını kontrol eder, gerekiyorsa composite veya covering index öneririm.*
>
> *Sorgu yapısında ise gereksiz DISTINCT ve UNION kullanımını UNION ALL'a çeviririm, korele alt sorguları JOIN veya window fonksiyonuyla yeniden yazarım. Veri hacmi çok büyükse tarih bazlı partitioning ve materialized view gibi mimari çözümleri değerlendiririm.*
>
> *Son olarak değişikliği ölçerim: aynı yürütme planını ve süreyi tekrar alıp gerçekten iyileşme olduğunu kanıtlarım."*

2. Detaylı Arka Plan

Yürütme Planında Nelere Bakılır?

- **Seek ve Scan farkı:** Index Seek, index üzerinden doğrudan hedefe gitmektir (iyi). Index Scan veya Table Scan tüm yapıyı okumaktır (büyük tablolarda kötü).
- **Tahmini ve gerçek satır sayısı:** Aralarında büyük fark varsa istatistikler güncel değildir. Çözüm istatistikleri yenilemektir (UPDATE STATISTICS / ANALYZE).
- **Join algoritması:** Küçük tabloda Nested Loop beklenir; büyük tablolarda Hash Join normaldir. Büyük iki tabloda Nested Loop görüyorsan genellikle index eksikliği veya kötü kardinalite tahmini vardır.
- **Sort ve Spill:** Sıralama işlemi belleğe sığmayıp diske taşıyorsa (spill to disk) ciddi yavaşlama olur. Gereksiz ORDER BY ve DISTINCT'leri kaldırmak veya index ile sıralı veri sağlamak çözümdür.

```sql
-- PostgreSQL: gercek sureleri ve satir sayilarini da gosterir
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;

-- SQL Server
SET STATISTICS IO, TIME ON;
-- veya Actual Execution Plan'i acarak calistir

-- MySQL
EXPLAIN ANALYZE SELECT ...;
```

Pratik Optimizasyon Kontrol Listesi

- SELECT * kullanma; yalnızca gereken sütunları seç. Bu hem ağ trafiğini hem de covering index şansını doğrudan etkiler.
- WHERE koşullarını SARGable yaz: sütunu fonksiyon veya aritmetik içine sokma.

- LIKE '%metin' aramaları için B-Tree yerine full-text index veya trigram index (PostgreSQL pg_trgm) kullan.
- Tekrar elemesi gerekmiyorsa UNION yerine UNION ALL kullan.
- Gereksiz DISTINCT ekleme; genellikle DISTINCT ihtiyacı, hatalı bir JOIN'in ürettiği satır çoğalmasının semptomudur. Önce JOIN'i düzelt.
- OR ile bağlı farklı sütun koşulları index kullanımını engelleyebilir; gerekirse UNION ALL ile iki ayrı sorguya böl.
- Büyük tablolarda tarih bazlı partitioning uygula; sorgu yalnızca ilgili partition'ı okur (partition pruning).
- Sık tekrarlanan pahalı toplulaştırmaları materialized view veya önceden hesaplanmış özet tabloya taşı.
- Uygulama tarafında N+1 sorgu problemine dikkat et: döngü içinde tek tek sorgu atmak yerine tek bir toplu sorgu yaz.

Mülakat Sorusu: "Index ekledim ama sorgu hâlâ yavaş, neden?"

Olası sebepleri sıralayabilmek güçlü bir sinyaldir: Koşul SARGable olmadığı için index kullanılmıyor olabilir; index'in sütun sırası sorguya uymuyor olabilir; sütunun seçiciliği düşük olduğu için optimizer tam taramayı daha ucuz buluyor olabilir; istatistikler bayat olduğu için yanlış plan seçiliyor olabilir; ya da darboğaz aslında sorguda değil, kilit beklemesinde (lock contention) veya diskte olabilir.

Python: Temeller ve Bellek Mekanizmaları

Yorumlanan Dil, Bytecode ve PVM

> **SORU:** *Python yorumlanan bir dil midir yoksa derlenen bir dil mi? Bir .py dosyasını çalıştırdığında arka planda ne olur?*

1. Mülakat Cevabı

> *"Bu ikisi arasında keskin bir ayrım yapmak yanıltıcıdır; Python aslında her ikisini de yapar.*
>
> *Bir .py dosyasını çalıştırdığında CPython önce kaynak kodu ayrıştırır ve bytecode adı verilen ara bir temsile derler. Bu bytecode makine kodu değildir; platformdan bağımsız, düşük seviyeli komutlardan oluşur ve tekrar tekrar derlememek için __pycache__ klasöründe .pyc dosyası olarak önbelleğe alınır.*
>
> *Ardından bu bytecode, Python Sanal Makinesi (PVM) tarafından komut komut yorumlanarak çalıştırılır. Yani ortada bir derleme adımı vardır, ama çıktısı CPU'nun doğrudan çalıştırdığı makine kodu değil, sanal makinenin çalıştırdığı bytecode'dur.*
>
> *Bu mimarinin pratik sonucu şudur: Python taşınabilirdir ve geliştirme döngüsü hızlıdır, ancak C veya C++ gibi doğrudan makine koduna derlenen dillerden yavaştır; çünkü araya bir yorumlama katmanı girer."*

2. Detaylı Arka Plan

Bytecode'u Görmek: dis Modülü

Mülakatta bu konuyu somutlaştırmak istersen standart kütüphanedeki dis modülünden bahsedebilirsin; bir fonksiyonun hangi bytecode komutlarına çevrildiğini gösterir.

```Python
import dis

def topla(a, b):
    return a + b

dis.dis(topla)
#   LOAD_FAST     a
#   LOAD_FAST     b
#   BINARY_OP     + (add)
#   RETURN_VALUE
```

CPython, PyPy ve Diğer Uygulamalar

"Python" bir dil belirtimidir; CPython ise onun referans uygulamasıdır ve C ile yazılmıştır. Mülakatta bu ayrımı yapmak, dilin kendisiyle uygulamasını karıştırmadığını gösterir.

- **CPython:** Varsayılan ve en yaygın uygulama. Bytecode'u yorumlar. GIL burada bulunur.
- **PyPy:** JIT (Just-In-Time) derleyici kullanır; sık çalışan kod bloklarını çalışma anında makine koduna çevirir. Uzun süren saf Python döngülerinde CPython'a göre kat kat hızlı olabilir.
- **Cython:** Python benzeri kodu C'ye çevirip derler. Performans kritik bölümleri hızlandırmak için kullanılır.
- **Jython / IronPython:** Sırasıyla JVM ve .NET üzerinde çalışan uygulamalar. Bu ortamlarda GIL kısıtı bulunmaz.

Mülakat Tuzağı: "Python yavaş bir dil mi?"

Tek kelimeyle "evet" demek eksik bir cevaptır. Doğru cevap katmanlıdır: Saf Python ile yazılmış, milyonlarca iterasyonlu bir sayısal döngü gerçekten yavaştır; çünkü her adım yorumlayıcıdan geçer ve her değer bir Python nesnesidir.

Ancak veri mühendisliğinde ve bilimsel hesaplamada kullandığımız kütüphanelerin çekirdeği (NumPy, Pandas, PyArrow, scikit-learn) C, C++ veya Rust ile yazılmıştır. Biz Python'u yalnızca bu hızlı çekirdekleri koordine eden bir yapıştırıcı dil (glue language) olarak kullanırız. Bu yüzden doğru yazılmış bir Pandas veya PySpark işi, Python'un yorumlama maliyetinden neredeyse hiç etkilenmez. Asıl yavaşlık, vektörleştirilmiş bir işlem yerine satır satır Python döngüsü yazıldığında ortaya çıkar.

GIL (Global Interpreter Lock)

> **SORU:** *GIL nedir ve neden vardır? Python'da multithreading ne zaman işe yarar, ne zaman yaramaz?*

1. Mülakat Cevabı

> *"GIL, CPython yorumlayıcısındaki global bir kilittir ve aynı anda yalnızca tek bir thread'in Python bytecode'u çalıştırmasına izin verir. Yani sekiz çekirdekli bir makinede sekiz thread başlatsan bile, Python kodu gerçek anlamda paralel çalışmaz.*
>
> *Var oluş sebebi bellek yönetimidir. CPython nesneleri referans sayımıyla yönetir ve bu sayaçların eşzamanlı güncellemelerde bozulmaması gerekir. Her nesne için ayrı kilit koymak hem karmaşık hem de yavaş olacağından, CPython tek bir global kilitle bu sorunu ucuz şekilde çözer.*
>
> *Pratik sonucu şudur: CPU-bound işlerde (matematiksel hesaplama, görüntü işleme, sıkıştırma) multithreading hiçbir hızlanma sağlamaz; hatta bağlam değişimi maliyeti yüzünden tek thread'den daha yavaş çalışabilir. Buna karşılık I/O-bound işlerde (dosya okuma, HTTP isteği, veritabanı sorgusu) çok işe yarar; çünkü bir thread I/O beklerken GIL'i serbest bırakır ve diğer thread'ler bu sırada çalışabilir.*
>
> *Gerçek paralellik gerektiğinde multiprocessing kullanırım; her süreç kendi yorumlayıcısına ve kendi GIL'ine sahip olduğu için çekirdekler gerçekten paralel çalışır. Çok sayıda eşzamanlı ağ isteği söz konusuysa da thread yerine asyncio tercih ederim."*

2. Detaylı Arka Plan

GIL Ne Zaman Serbest Bırakılır?

GIL sürekli tutulmaz. Üç durumda bırakılır ve bu, multithreading'in ne zaman işe yaradığını açıklar:

- **I/O işlemleri sırasında:** Dosya okuma, socket bekleme, veritabanı sorgusu gibi işlemlerde yorumlayıcı GIL'i bırakır; çünkü beklenen şey CPU değil, işletim sistemidir.
- **Belirli aralıklarla:** Yorumlayıcı, tek bir thread'in kilidi süresiz tutmasını engellemek için periyodik olarak (varsayılan 5 ms) GIL'i bırakıp diğer thread'lere şans verir.
- **C uzantılarında:** NumPy gibi kütüphaneler, uzun süren C seviyesindeki hesaplamalar sırasında GIL'i açıkça serbest bırakabilir. Bu yüzden NumPy ağırlıklı bazı işler thread ile gerçekten hızlanabilir.

Doğru Aracı Seçmek

Yaklaşım	Gerçek Paralellik	Uygun İş Türü	Maliyet ve Not
threading	Yok (GIL engeller)	I/O-bound: dosya, ağ, veritabanı	Düşük maliyet; ortak bellek, race condition riski
multiprocessing	Var (her sürecin kendi GIL'i)	CPU-bound: hesaplama, dönüşüm	Yüksek maliyet; veri serileştirilerek kopyalanır
asyncio	Yok (tek thread, event loop)	Çok sayıda eşzamanlı I/O	Çok düşük maliyet; kütüphanenin async olması şart

Kod Üzerinde Fark

```python
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def cpu_bound(n):
    return sum(i * i for i in range(n))

# CPU-bound + thread -> HIZLANMA YOK (GIL yuzunden)
with ThreadPoolExecutor(max_workers=4) as ex:
    list(ex.map(cpu_bound, [10_000_000] * 4))
```

```
# CPU-bound + process -> GERCEK PARALELLIK
with ProcessPoolExecutor(max_workers=4) as ex:
    list(ex.map(cpu_bound, [10_000_000] * 4))

# I/O-bound + thread -> BUYUK KAZANC
import requests

def indir(url):
    return requests.get(url).status_code

with ThreadPoolExecutor(max_workers=20) as ex:
    list(ex.map(indir, urls))   # 20 istek ic ice bekler
```

Veri Mühendisliği Bağlamında GIL

Bu konu bir veri mühendisliği mülakatında genellikle şu soruyla gelir: "Airflow worker'ında büyük bir dönüşümü hızlandırmak için thread kullanır mısın?" Doğru cevap hayırdır. Veri dönüşümü CPU-bound bir iştir ve GIL yüzünden thread ile hızlanmaz. Bu iş ya multiprocessing ile ya da doğru mimariyle Spark'a veya veri ambarına itilerek yapılmalıdır. Thread'i yalnızca çok sayıda API çağrısı veya dosya indirme gibi I/O ağırlıklı adımlarda kullanırım.

Mülakat Bonusu: GIL Kalkıyor mu?

PEP 703 ile birlikte CPython'a GIL'siz (free-threaded) bir derleme seçeneği eklendi ve Python 3.13 ile deneysel olarak sunuldu. Bu, uzun vadede CPU-bound multithreading'i mümkün kılacak. Ancak henüz varsayılan değildir, tek thread performansında bir miktar kayba yol açar ve C uzantı ekosisteminin uyumlu hale gelmesi zaman alacaktır. Bu gelişmeden haberdar olduğunu belirtmek, dili yakından takip ettiğini gösterir.

Bellek Yönetimi ve Garbage Collection

> **SORU:** *Python belleği nasıl yönetir? Referans sayımı nedir ve döngüsel referansları nasıl çözer?*

1. Mülakat Cevabı

> *"Python'un birincil bellek yönetimi mekanizması referans sayımıdır (reference counting). Her nesne, kendisine kaç referansın işaret ettiğini tutan bir sayaç barındırır. Bir referans eklendiğinde sayaç artar, silindiğinde veya kapsam dışına çıktığında azalır. Sayaç sıfıra düştüğü anda nesnenin belleği hemen serbest bırakılır.*
>
> *Bu yaklaşımın avantajı, belleğin deterministik ve anında geri kazanılmasıdır. Ancak tek başına çözemediği bir problem vardır: döngüsel referanslar. A nesnesi B'yi, B de A'yı işaret ediyorsa, dışarıdan hiçbir referans kalmasa bile sayaçları asla sıfıra düşmez ve bellek sızar.*
>
> *Bu yüzden CPython'da ikinci bir mekanizma vardır: nesil bazlı döngüsel çöp toplayıcı (generational garbage collector). Nesneleri üç nesle ayırır ve genç nesli daha sık tarar; çünkü nesnelerin çoğu kısa ömürlüdür. Bu toplayıcı periyodik olarak çalışır, ulaşılamaz hale gelmiş döngüleri tespit eder ve temizler.*
>
> *gc modülü üzerinden bu toplayıcıyı manuel tetikleyebilir, istatistiklerini okuyabilir veya performans açısından kritik bir bölümde geçici olarak devre dışı bırakabilirim."*

2. Detaylı Arka Plan

Referans Sayımını Gözlemlemek

```Python
import sys, gc

liste = []
print(sys.getrefcount(liste))   # 2: 'liste' ismi + getrefcount'un kendi argumani

baska = liste                   # ikinci referans
print(sys.getrefcount(liste))   # 3

del baska
print(sys.getrefcount(liste))   # 2 -> tekrar dustu

# DONGUSEL REFERANS: referans sayimi tek basina yetmez
class Dugum:
    def __init__(self, ad):
        self.ad = ad
        self.komsu = None

a = Dugum("A"); b = Dugum("B")
a.komsu = b
b.komsu = a                     # dongu olustu
del a, b                        # disaridan referans kalmadi ama sayaclar 1

print(gc.collect())             # dongusel toplayici bunlari temizler
```

Nesil Bazlı Toplama (Generational GC)

Çöp toplayıcı, "nesnelerin çoğu kısa ömürlüdür" varsayımına dayanır (weak generational hypothesis). Yeni oluşturulan nesneler 0. nesle konur. Bir toplama turundan sağ çıkan nesne bir üst nesle terfi eder. 0. nesil çok sık, 2. nesil ise nadiren taranır. Böylece toplama maliyeti, uzun ömürlü nesneleri sürekli yeniden taramak zorunda kalmadan düşük tutulur.

```Python
import gc
```

```
print(gc.get_threshold())   # (700, 10, 10) -> nesil esik degerleri
print(gc.get_count())       # her nesildeki mevcut nesne sayisi

# Cok sayida kisa omurlu nesne ureten, performans kritik bolumlerde:
gc.disable()
# ... yogun islem ...
gc.enable()
gc.collect()
```

Mülakat Tuzağı: Küçük Tamsayı Önbelleği ve is Operatörü

CPython, sık kullanılan küçük tamsayıları (-5 ile 256 arası) program başlarken önceden oluşturur ve tekrar tekrar aynı nesneyi kullanır. Kısa string'ler için de benzer bir mekanizma vardır (interning). Bu, mülakatlarda çok sevilen bir tuzak sorusu üretir:

```Python
a = 256
b = 256
print(a is b)      # True  -> onbellekten AYNI nesne

x = 257
y = 257
print(x is y)      # False -> ayri nesneler (baglama gore degisebilir)

print(x == y)      # True  -> deger karsilastirmasi her zaman dogru
```

Alınacak ders: Değer karşılaştırması için her zaman == kullanılır. is operatörü kimlik karşılaştırır, yani "aynı nesne mi" sorusunu sorar ve yalnızca None, True, False gibi tekil nesnelerde kullanılmalıdır. Bu yüzden "x is None" doğru, "x == None" ise stil olarak yanlıştır.

Python'da Bellek Sızıntısı Mümkün mü?

Çöp toplayıcı olmasına rağmen evet, mümkündür. En yaygın sebepler şunlardır: global bir listeye veya sözlüğe sürekli veri eklemek ve hiç temizlememek (özellikle önbellek amaçlı), kapanmayan dosya ve bağlantı tanıtıcıları, ve çok uzun süre yaşayan nesnelerin kısa ömürlü nesnelere referans tutması. Çözüm olarak weakref kullanmak, önbelleklere boyut sınırı koymak (functools.lru_cache içinde maxsize) ve kaynakları context manager ile yönetmek gerekir.

Mutable ve Immutable Tipler

> **SORU:** *Mutable ve immutable tipler arasındaki fark nedir? Bir fonksiyona liste geçirip içinde değiştirirsen dışarıdaki liste etkilenir mi?*

1. Mülakat Cevabı

> *"Immutable tipler oluşturulduktan sonra değiştirilemez: int, float, bool, str, tuple, frozenset ve bytes. Bir değişkene yeni değer atadığında aslında nesneyi değiştirmezsin; o isim bellekte yeni bir nesneye bağlanır, eski nesne olduğu gibi kalır.*
>
> *Mutable tipler ise yerinde değiştirilebilir: list, dict, set ve varsayılan olarak kendi yazdığımız sınıflar.*
>
> *Python'da argüman geçişi ne tam olarak call by value ne de call by reference'tır; buna call by object reference denir. Fonksiyona nesnenin referansı geçirilir. Eğer fonksiyon içinde mutable bir nesneyi yerinde değiştirirsen (append, eleman ataması), dışarıdaki nesne de değişir; çünkü ikisi bellekte aynı nesnedir. Ancak fonksiyon içinde parametreye yeni bir değer atarsan, yalnızca yerel isim yeniden bağlanır ve dışarısı hiç etkilenmez.*
>
> *Bu ayrım Python mülakatlarının en sık sorulan ve en çok hata yapılan konusudur; genellikle mutable varsayılan argüman tuzağıyla birlikte gelir."*

2. Detaylı Arka Plan

Değiştirme ve Yeniden Bağlama Farkı

```python
def degistir(liste):
    liste.append(4)          # YERINDE DEGISIKLIK -> disarisi etkilenir

def yeniden_bagla(liste):
    liste = [9, 9, 9]        # YEREL ISIM yeniden baglandi -> disarisi etkilenmez

x = [1, 2, 3]
degistir(x)
print(x)                     # [1, 2, 3, 4]

y = [1, 2, 3]
yeniden_bagla(y)
print(y)                     # [1, 2, 3]  <- degismedi
```

En Klasik Tuzak: Mutable Varsayılan Argüman

Varsayılan argümanlar fonksiyon çağrıldığında değil, fonksiyon TANIMLANDIĞINDA bir kez oluşturulur. Mutable bir varsayılan değer verirsen, o nesne tüm çağrılar arasında paylaşılır.

```python
# YANLIS
def ekle(oge, liste=[]):     # bu liste YALNIZCA BIR KEZ olusturulur
    liste.append(oge)
    return liste

print(ekle(1))               # [1]
print(ekle(2))               # [1, 2]   <- beklenen [2] idi!
print(ekle(3))               # [1, 2, 3]

# DOGRU
def ekle(oge, liste=None):
    if liste is None:
        liste = []           # her cagrida yeni liste
    liste.append(oge)
    return liste
```

Sığ ve Derin Kopya (Shallow vs Deep Copy)

```python
import copy

orijinal = [[1, 2], [3, 4]]

sig   = copy.copy(orijinal)       # veya orijinal[:] / list(orijinal)
derin = copy.deepcopy(orijinal)

orijinal[0].append(99)

print(sig)     # [[1, 2, 99], [3, 4]]  <- ic listeler PAYLASILIYOR
print(derin)   # [[1, 2], [3, 4]]      <- tamamen bagimsiz
```

Sığ kopya yalnızca dış kabı kopyalar; içindeki nesnelerin referansları aynı kalır. İç içe yapılarda beklenmedik davranışın en yaygın sebebi budur.

Mülakat Tuzağı: Tuple Gerçekten Değişmez mi?

Tuple'ın kendisi değişmezdir; yani içindeki referanslar değiştirilemez. Ancak o referansların işaret ettiği nesneler mutable ise, onların içeriği pekala değişebilir.

```python
t = (1, [2, 3])
# t[1] = [9]      -> TypeError: tuple degistirilemez
t[1].append(4)    # ANCAK BU CALISIR
print(t)          # (1, [2, 3, 4])

# Sonuc: bu tuple hashable DEGILDIR, dict anahtari olamaz
# hash(t)  -> TypeError: unhashable type: 'list'
```

Bu, hashable kavramıyla doğrudan ilgilidir: bir nesnenin dictionary anahtarı veya set elemanı olabilmesi için hashable olması, bunun için de pratikte tamamen immutable olması gerekir.

Yerleşik Veri Yapıları ve Zaman Karmaşıklığı

> **SORU:** *list, tuple, dict ve set arasındaki farkları ve zaman karmaşıklıklarını anlat. Dictionary anahtarı olarak neler kullanılabilir?*

1. Mülakat Cevabı

> *"Dört temel yerleşik veri yapısı, farklı erişim desenleri için optimize edilmiştir.*
>
> *list, dinamik dizi (dynamic array) olarak uygulanmış, sıralı ve mutable bir yapıdır. İndeksle erişim ve sona ekleme O(1), araya ekleme veya silme ise elemanların kaydırılması gerektiği için O(n)'dir. İçinde arama, yani in operatörü de O(n)'dir.*
>
> *tuple, list'in immutable karşılığıdır. Değiştirilemediği için daha az bellek kullanır ve içeriği de hashable ise kendisi de hashable olur; bu sayede dictionary anahtarı veya set elemanı olabilir.*
>
> *dict, hash tablosu olarak uygulanmıştır. Anahtarla erişim, ekleme ve silme ortalamada O(1)'dir. Anahtarların hashable, yani pratikte immutable olması zorunludur. Python 3.7'den itibaren ekleme sırasını koruması dil garantisidir.*
>
> *set de hash tablosuna dayanır; benzersiz ve sırasız elemanlar tutar. Üyelik testi ortalamada O(1) olduğu için tekilleştirme ve 'bu değer koleksiyonda var mı' kontrolleri için listeye göre çok üstündür.*
>
> *Pratikte en sık yaptığım optimizasyon, büyük bir koleksiyonda tekrarlı üyelik kontrolü gerekiyorsa listeyi önce set'e çevirmektir; bu, O(n) bir işlemi O(1)'e indirir."*

2. Detaylı Arka Plan

Zaman Karmaşıklığı Tablosu

Veri Yapısı	Arama (in)	Ekleme	Temel Özellik
list	O(n)	O(1) sona, O(n) araya	Sıralı, mutable, indekslenebilir
tuple	O(n)	Yok (immutable)	Sıralı, immutable, hashable olabilir
dict	O(1) ortalama	O(1) ortalama	Anahtar-değer, anahtar hashable olmalı
set	O(1) ortalama	O(1) ortalama	Benzersiz, sırasız, hashable elemanlar

Mülakat Tuzağı: Listede Tekrarlı Arama

```Python
# YAVAS: her kontrol O(n), toplam O(n*m)
gecerli_idler = [1, 2, 3, ...]          # 100.000 elemanli liste
sonuc = [k for k in kayitlar if k.id in gecerli_idler]

# HIZLI: set'e cevir, her kontrol O(1), toplam O(n+m)
gecerli_idler = set(gecerli_idler)
sonuc = [k for k in kayitlar if k.id in gecerli_idler]
```

Bu, veri işleme kodlarında en sık karşılaşılan performans hatasıdır ve mülakatta "bu kodu nasıl hızlandırırsın?" sorusunun beklenen cevabıdır.

Hashable Olmak Ne Demek?

Bir nesnenin dictionary anahtarı olabilmesi için ömrü boyunca değişmeyen bir hash değerine sahip olması ve eşitlik karşılaştırması yapabilmesi gerekir. Mutable nesneler bu şartı sağlayamaz: bir listeyi anahtar yapıp sonra içeriğini değiştirseydin, hash değeri değişir ve nesneyi hash tablosunda bir daha bulamazdın.

```Python
d = {}
```

```
d[(1, 2)] = "gecerli"          # tuple hashable -> OK
d["metin"] = "gecerli"         # str hashable   -> OK
d[frozenset([1, 2])] = "OK"    # frozenset hashable

# d[[1, 2]] = "hata"           # TypeError: unhashable type: 'list'
# d[{1: 2}] = "hata"           # TypeError: unhashable type: 'dict'
```

collections Modülü: Doğru Aracı Seçmek

- **defaultdict:** Olmayan anahtara erişildiğinde otomatik varsayılan değer üretir; gruplama kodlarındaki if kontrolünü ortadan kaldırır.
- **Counter:** Frekans sayımı için özelleşmiş bir dict. most_common() ile en sık geçen elemanları doğrudan verir.
- **deque:** Çift uçlu kuyruk. Listeden farklı olarak baştan ekleme ve silme O(1)'dir; oysa list.pop(0) O(n)'dir. Kuyruk yapıları için doğru seçimdir.
- **namedtuple ve dataclass:** Anlamlı alan isimleriyle hafif veri taşıyıcıları oluşturur; ham sözlük yerine kullanıldığında kod okunabilirliğini ciddi şekilde artırır.

```Python
from collections import defaultdict, Counter, deque

# Gruplama: if kontrolu gerekmez
gruplar = defaultdict(list)
for kayit in kayitlar:
    gruplar[kayit.kategori].append(kayit)

# Frekans sayimi
sayim = Counter(kelimeler)
print(sayim.most_common(3))

# Kuyruk: bastan silme O(1)
kuyruk = deque([1, 2, 3])
kuyruk.appendleft(0)
kuyruk.popleft()
```

Python: Fonksiyonlar ve İleri Sözdizimi

*args ve **kwargs

> **SORU:** **args ve **kwargs ne işe yarar? Fonksiyon parametrelerinde * ve / işaretleri ne anlama gelir?*

1. Mülakat Cevabı

> *"Bu ikisi, bir fonksiyonun değişken sayıda argüman kabul etmesini sağlar.*
>
> **args, isimsiz (pozisyonel) argümanları bir tuple içinde toplar. **kwargs ise isimli (anahtar kelimeli) argümanları bir sözlükte toplar. İsimlendirme zorunlu değildir; asıl anlamı taşıyan yıldız işaretleridir, ancak args ve kwargs kullanmak yerleşmiş bir gelenektir.*
>
> *Aynı işaretler çağrı tarafında ters yönde çalışır: bir listeyi veya tuple'ı *liste ile açarak pozisyonel argümanlara, bir sözlüğü **sozluk ile açarak anahtar kelimeli argümanlara dönüştürürüz. Buna unpacking denir.*
>
> *Parametre listesinde tek başına duran bir yıldız, kendisinden sonraki tüm parametrelerin yalnızca isimle verilebileceğini (keyword-only) belirtir. Eğik çizgi ise kendisinden öncekilerin yalnızca pozisyonel verilebileceğini (positional-only) belirtir. Bunlar API tasarımında çağrı biçimini netleştirmek ve ileride parametre adını değiştirebilmek için kullanılır.*
>
> *En sık kullandığım yer decorator yazmaktır; sardığım fonksiyonun imzasını bilmediğim için argümanları *args ve **kwargs ile alıp olduğu gibi iletirim."*

2. Detaylı Arka Plan

Toplama ve Açma (Packing / Unpacking)

```python
def bilgi(*args, **kwargs):
    print(args)      # tuple
    print(kwargs)    # dict

bilgi(1, 2, 3, ad="Ahmet", yas=30)
# (1, 2, 3)
# {'ad': 'Ahmet', 'yas': 30}

# CAGRI TARAFINDA ACMA
def topla(a, b, c):
    return a + b + c

sayilar = [1, 2, 3]
print(topla(*sayilar))           # 6

parametreler = {"a": 1, "b": 2, "c": 3}
print(topla(**parametreler))     # 6
```

Positional-only ve Keyword-only Parametreler

```python
def f(a, b, /, c, d, *, e, f):
    ...
#       ^^^^      ^^^^     ^^^^
#   sadece    hem pozisyonel   sadece
# pozisyonel   hem isimli      isimli

f(1, 2, 3, d=4, e=5, f=6)     # gecerli
# f(a=1, b=2, ...)            # HATA: a ve b pozisyonel olmali
```

```
# f(1, 2, 3, 4, 5, 6)         # HATA: e ve f isimle verilmeli
```

Positional-only parametrelerin pratik faydası şudur: parametre adı fonksiyonun genel arayüzünün parçası olmaz, dolayısıyla ileride geriye dönük uyumluluğu bozmadan yeniden adlandırılabilir. Standart kütüphanede birçok yerleşik fonksiyon bu şekilde tanımlıdır.

Argüman İletme (Forwarding) Deseni

```python
def loglu(fn):
    def sarmalayici(*args, **kwargs):     # imzayi bilmeden hepsini al
        print(f"cagriliyor: {fn.__name__}")
        return fn(*args, **kwargs)        # oldugu gibi ilet
    return sarmalayici
```

Comprehension'lar ve Lambda Fonksiyonları

> **SORU:** *List comprehension nedir, ne zaman kullanmamalısın? Lambda fonksiyonunun sınırları nelerdir?*

1. Mülakat Cevabı

> *"Comprehension, bir koleksiyondan yeni bir koleksiyon üretmenin kısa ve okunabilir yoludur. Liste, sözlük ve küme için ayrı biçimleri vardır. Geleneksel for döngüsüne göre hem daha kısadır hem de yorumlayıcı tarafında optimize edildiği için genellikle biraz daha hızlıdır.*
>
> *Ancak her yerde kullanılmamalıdır. İçinde birden fazla iç içe döngü ve karmaşık koşullar varsa okunabilirlik hızla bozulur; böyle durumlarda normal döngü yazmak daha doğrudur. Ayrıca comprehension'ı yan etki için, yani yalnızca bir şeyler yazdırmak veya bir listeye eklemek için kullanmak kötü bir pratiktir; amaç her zaman yeni bir koleksiyon üretmek olmalıdır.*
>
> *Köşeli parantez yerine normal parantez kullanıldığında ise generator expression elde edilir; bu, tüm sonucu bellekte oluşturmaz ve büyük veri kümelerinde bellek açısından kritik bir farktır.*
>
> *Lambda ise tek satırlık, isimsiz bir fonksiyondur. Yalnızca tek bir ifade içerebilir; içine atama, döngü veya birden fazla satır koyamazsın. Bu yüzden onu sadece sorted, min, max gibi fonksiyonlara anahtar (key) fonksiyonu geçerken kullanırım. Bir lambda'yı bir değişkene atıyorsam, aslında normal bir def yazmam gerektiğinin işaretidir; PEP 8 da bunu açıkça önerir."*

2. Detaylı Arka Plan

Comprehension Biçimleri

```python
sayilar = [1, 2, 3, 4, 5, 6]

# Liste
kareler = [x * x for x in sayilar]                    # [1, 4, 9, 16, 25, 36]

# Kosullu filtreleme (sagda if)
ciftler = [x for x in sayilar if x % 2 == 0]          # [2, 4, 6]

# Kosullu deger (solda if-else)
etiket = ["cift" if x % 2 == 0 else "tek" for x in sayilar]

# Sozluk
kare_haritasi = {x: x * x for x in sayilar}

# Kume (otomatik tekillestirme)
benzersiz = {k.kategori for k in kayitlar}

# Generator expression: parantez -> tembel, bellek dostu
toplam = sum(x * x for x in range(10_000_000))        # liste olusturulmaz
```

Bellek Farkı: Liste ve Generator

```python
import sys

liste = [x for x in range(1_000_000)]     # tum degerler bellekte
gen   = (x for x in range(1_000_000))     # yalnizca uretec nesnesi

print(sys.getsizeof(liste))   # ~8 MB
print(sys.getsizeof(gen))     # ~200 byte
```

Veri mühendisliğinde bu fark belirleyicidir: bir dosyayı satır satır işlerken tüm satırları listeye almak yerine generator kullanmak, gigabaytlarca veriyi sabit bellekle işlemeyi mümkün kılar.

Lambda'nın Doğru ve Yanlış Kullanımı

```python
kayitlar = [{"ad": "Ali", "yas": 30}, {"ad": "Zeynep", "yas": 25}]

# DOGRU: gecici anahtar fonksiyonu
sirali = sorted(kayitlar, key=lambda k: k["yas"])
en_yasli = max(kayitlar, key=lambda k: k["yas"])

# YANLIS: lambda'yi isme atamak (PEP 8 ihlali)
# kare = lambda x: x * x

# DOGRU karsiligi
def kare(x):
    return x * x
```

Adlandırılmış bir lambda, hata izlerinde (traceback) anlamlı bir fonksiyon adı göstermez ve dokümantasyon dizesi (docstring) taşıyamaz. Bu yüzden def tercih edilir.

map, filter ve Comprehension Karşılaştırması

map ve filter fonksiyonel programlamadan gelir ve hâlâ kullanılabilir; ancak Python topluluğunda comprehension genellikle daha okunabilir kabul edilir. Hazır bir fonksiyon geçiriyorsan map temiz durur, lambda yazman gerekiyorsa comprehension daha nettir.

```python
# map + hazir fonksiyon: temiz
sayilar_str = list(map(str, sayilar))

# map + lambda: comprehension daha okunabilir
# list(map(lambda x: x * 2, sayilar))
[x * 2 for x in sayilar]
```

Decorator'lar

> **SORU:** *Decorator nedir ve nasıl çalışır? Kendi decorator'ını nasıl yazarsın? functools.wraps neden gereklidir?*

1. Mülakat Cevabı

> *"Decorator, bir fonksiyonu (veya sınıfı) parametre olarak alan, onu saran ve yeni bir fonksiyon döndüren bir fonksiyondur. Amacı, orijinal fonksiyonun kodunu hiç değiştirmeden davranışını genişletmektir.*
>
> *Bunun mümkün olmasının sebebi Python'da fonksiyonların birinci sınıf nesne olmasıdır: değişkene atanabilir, argüman olarak geçirilebilir ve başka bir fonksiyondan döndürülebilirler.*
>
> *@decorator sözdizimi yalnızca sözdizimsel şekerdir; arka planda fonksiyon = decorator(fonksiyon) yazmakla birebir aynıdır.*
>
> *Tipik kullanım alanları loglama, süre ölçümü, yetkilendirme kontrolü, önbellekleme, yeniden deneme ve girdi doğrulamadır. Bunlar iş mantığından bağımsız, kesişen ilgilerdir (cross-cutting concerns) ve her fonksiyona kopyalamak yerine decorator ile merkezileştirilir.*
>
> *Decorator yazarken functools.wraps kullanmak kritiktir; aksi halde sarmalanan fonksiyonun adı, docstring'i ve imza bilgisi kaybolur ve hata ayıklama ile dokümantasyon araçları yanlış bilgi gösterir."*

2. Detaylı Arka Plan

Temel Decorator Yapısı

```python
import time
import functools

def sure_olc(fn):
    @functools.wraps(fn)                  # ad, docstring ve metadata korunur
    def sarmalayici(*args, **kwargs):
        basla = time.perf_counter()
        try:
            return fn(*args, **kwargs)
        finally:
            gecen = time.perf_counter() - basla
            print(f"{fn.__name__} {gecen:.3f} sn surdu")
    return sarmalayici

@sure_olc
def veri_isle(n):
    """Buyuk bir veri kumesini isler."""
    return sum(range(n))

veri_isle(10_000_000)
# veri_isle 0.412 sn surdu

# @sure_olc sozdizimi sunun kisayoludur:
# veri_isle = sure_olc(veri_isle)
```

functools.wraps Neden Şart?

```python
# wraps OLMADAN
print(veri_isle.__name__)   # 'sarmalayici'   <- yanlis!
print(veri_isle.__doc__)    # None            <- docstring kayboldu

# wraps ILE
print(veri_isle.__name__)   # 'veri_isle'
```

```
print(veri_isle.__doc__)    # 'Buyuk bir veri kumesini isler.'
```

Bu detayı mülakatta kendiliğinden söylemek, decorator'ı ezberden değil gerçekten kullanarak öğrendiğini gösterir.

Argüman Alan Decorator

Decorator'ın kendisi parametre alacaksa bir kat daha sarmalama gerekir: dış fonksiyon parametreleri alır ve asıl decorator'ı döndürür.

```python
import functools, time

def yeniden_dene(kere=3, bekleme=1.0):
    def decorator(fn):
        @functools.wraps(fn)
        def sarmalayici(*args, **kwargs):
            son_hata = None
            for deneme in range(1, kere + 1):
                try:
                    return fn(*args, **kwargs)
                except Exception as e:
                    son_hata = e
                    print(f"{deneme}. deneme basarisiz: {e}")
                    time.sleep(bekleme * deneme)   # ustel geri cekilme
            raise son_hata
        return sarmalayici
    return decorator

@yeniden_dene(kere=5, bekleme=2)
def api_cagir(url):
    ...
```

Bu desen veri mühendisliğinde çok işlevseldir; geçici ağ hatalarına karşı dayanıklı veri çekme adımları böyle yazılır.

Standart Kütüphanedeki Yararlı Decorator'lar

- **functools.lru_cache / functools.cache:** Fonksiyonun sonucunu argümanlara göre önbelleğe alır. Pahalı ve saf (deterministik) hesaplamalarda dramatik hızlanma sağlar; maxsize ile bellek sınırı konur.
- **property:** Bir metodu, parantez olmadan erişilen bir öznitelik gibi kullanmayı sağlar. Hesaplanan alanlar için idealdir.
- **staticmethod ve classmethod:** Sırasıyla örnekten bağımsız yardımcı metotlar ve sınıfın kendisiyle çalışan alternatif kurucular tanımlar.
- **dataclasses.dataclass:** Sınıfa __init__, __repr__ ve __eq__ metotlarını otomatik ekler; veri taşıyıcı sınıflarda tekrarlayan kodu ortadan kaldırır.

Generator ve Iterator

> **SORU:** *Iterator protokolü nedir? Generator nedir ve liste yerine ne zaman generator kullanırsın?*

1. Mülakat Cevabı

> *"Iterator, üzerinde tek tek gezilebilen bir nesnedir ve iki metot tanımlar: __iter__ kendisini döndürür, __next__ ise sıradaki değeri verir. Değer kalmadığında StopIteration istisnası fırlatır. for döngüsü aslında arka planda tam olarak bunu yapar.*
>
> *Generator, iterator yazmanın çok daha kısa yoludur. return yerine yield kullanan bir fonksiyondur. Çağrıldığında fonksiyon gövdesi hemen çalışmaz; bir generator nesnesi döner. Her next çağrısında fonksiyon bir sonraki yield'e kadar çalışır, değeri verir ve o anki tüm yerel durumunu koruyarak duraklar.*
>
> *Liste yerine generator kullanmamın temel sebebi bellektir. Liste tüm elemanları aynı anda bellekte tutar; generator ise değerleri talep edildikçe üretir, dolayısıyla bellek kullanımı veri boyutundan bağımsız olarak sabit kalır. Gigabaytlık bir log dosyasını satır satır işlemek buna klasik örnektir.*
>
> *Bunun karşılığında iki kısıt kabul ederim: generator tek geçişliktir, tükendikten sonra baştan gezilemez; ve len() ile uzunluğu alınamaz veya indekslenemez. Sonuca birden fazla kez erişmem gerekiyorsa listeye çeviririm."*

2. Detaylı Arka Plan

Iterator Protokolünü Elle Uygulamak

```python
class GeriSayim:
    def __init__(self, baslangic):
        self.mevcut = baslangic

    def __iter__(self):
        return self

    def __next__(self):
        if self.mevcut <= 0:
            raise StopIteration
        self.mevcut -= 1
        return self.mevcut + 1

for i in GeriSayim(3):
    print(i)          # 3, 2, 1
```

Aynı İşin Generator ile Yazımı

```python
def geri_sayim(baslangic):
    while baslangic > 0:
        yield baslangic
        baslangic -= 1

for i in geri_sayim(3):
    print(i)          # 3, 2, 1
```

On satırlık sınıf, üç satıra indi. Generator'ın gücü budur: durum yönetimini (nerede kalındığı) dilin kendisi üstlenir.

Veri Mühendisliğinde Kritik Kullanım: Büyük Dosya İşleme

```python
# YANLIS: 10 GB'lik dosyanin tamami bellege alinir -> MemoryError
```

```
def satirlari_oku_kotu(yol):
    with open(yol) as f:
        return f.readlines()          # tum satirlar listede

# DOGRU: tembel akis, bellek kullanimi sabit
def satirlari_oku(yol):
    with open(yol) as f:
        for satir in f:               # dosya nesnesi zaten bir iterator
            yield satir.rstrip("\n")

# Generator'lari zincirleme baglamak (pipeline)
def sadece_hatalar(satirlar):
    for s in satirlar:
        if "ERROR" in s:
            yield s

def ayristir(satirlar):
    for s in satirlar:
        yield json.loads(s)

# Hicbir asamada tum veri bellege alinmaz
for kayit in ayristir(sadece_hatalar(satirlari_oku("dev.log"))):
    isle(kayit)
```

yield from ve itertools

Python

```
# yield from: baska bir iterable'i oldugu gibi aktarir
def birlestir(*iterables):
    for it in iterables:
        yield from it

import itertools

# Bellek dostu yardimcilar
ilk_10  = itertools.islice(buyuk_generator, 10)       # dilimleme
gruplar = itertools.groupby(sirali_veri, key=lambda x: x.kategori)
zincir  = itertools.chain(liste1, liste2)             # kopyalamadan birlestir
```

Mülakat Tuzağı: Generator Tek Geçişliktir

Python

```
gen = (x for x in range(3))

print(list(gen))    # [0, 1, 2]
print(list(gen))    # []   <- tukendi, ikinci kez bos doner!

# Birden fazla kez gezmen gerekiyorsa listeye cevir
veriler = list(gen)
```

Python: OOP, Hata Yönetimi ve İyi Pratikler

Sınıflar: self, Sınıf ve Örnek Değişkenleri

> **SORU:** *self nedir ve neden açıkça yazılır? Sınıf değişkeni ile örnek değişkeni arasındaki fark nedir?*

1. Mülakat Cevabı

> *"self, bir metodun üzerinde çalıştığı örneğin (instance) kendisine verilen referanstır. Python'da metotlar aslında sıradan fonksiyonlardır; bir örnek üzerinden çağrıldıklarında o örnek otomatik olarak ilk argüman olarak geçirilir. Python'un felsefesi gereği bu argüman açıkça yazılır, çünkü 'explicit is better than implicit' ilkesi benimsenmiştir. Bu sayede metodun hangi nesneyle çalıştığı okuyucu için hiçbir zaman belirsiz kalmaz.*
>
> *Örnek değişkeni self üzerinden tanımlanır ve her nesneye özeldir. Sınıf değişkeni ise sınıf gövdesinde tanımlanır ve tüm örnekler arasında paylaşılır.*
>
> *Buradaki kritik tuzak şudur: sınıf değişkeni mutable bir nesneyse, bir örnek üzerinde yapılan yerinde değişiklik tüm örnekleri etkiler. Bu yüzden liste veya sözlük gibi mutable varsayılanları asla sınıf değişkeni olarak tanımlamam; __init__ içinde her örnek için ayrı ayrı oluştururum."*

2. Detaylı Arka Plan

Klasik Tuzak: Mutable Sınıf Değişkeni

```python
# YANLIS
class Sepet:
    urunler = []                    # SINIF degiskeni: tum ornekler paylasir

    def ekle(self, u):
        self.urunler.append(u)      # yerinde degisiklik -> herkesi etkiler

a = Sepet(); b = Sepet()
a.ekle("elma")
print(b.urunler)                    # ['elma']  <- b'nin sepetinde de var!

# DOGRU
class Sepet:
    def __init__(self):
        self.urunler = []           # ORNEK degiskeni: her nesneye ozel

    def ekle(self, u):
        self.urunler.append(u)
```

Bu, mutable varsayılan argüman tuzağının sınıflardaki karşılığıdır ve aynı kök sebebe dayanır: nesne bir kez oluşturulup paylaşılmaktadır.

Sınıf Değişkeninin Doğru Kullanımı

Sınıf değişkeni yanlış bir şey değildir; sabitler, sayaçlar ve tüm örnekler için ortak olması gereken yapılandırma değerleri için doğru araçtır.

```python
class Baglanti:
    VARSAYILAN_PORT = 5432          # sabit: paylasilmasi mantikli
    acik_baglanti_sayisi = 0        # sinif seviyesinde sayac

    def __init__(self, host, port=None):
        self.host = host                        # ornek degiskeni
```

```
        self.port = port or self.VARSAYILAN_PORT
        Baglanti.acik_baglanti_sayisi += 1
```

staticmethod ve classmethod

- **Örnek metodu:** İlk parametresi self'tir. Nesnenin durumunu okur veya değiştirir.
- **classmethod:** İlk parametresi cls'tir, yani sınıfın kendisidir. En yaygın kullanımı alternatif kurucu (factory) yazmaktır: farklı bir kaynaktan nesne üretmek.
- **staticmethod:** Ne self ne cls alır. Mantıksal olarak sınıfa ait olan ama nesnenin durumuna ihtiyaç duymayan yardımcı fonksiyonlar içindir.

```Python
class Tarih:
    def __init__(self, gun, ay, yil):
        self.gun, self.ay, self.yil = gun, ay, yil

    @classmethod
    def metinden(cls, metin):           # alternatif kurucu
        g, a, y = map(int, metin.split("."))
        return cls(g, a, y)             # cls sayesinde alt siniflarda da dogru calisir

    @staticmethod
    def artik_yil_mi(yil):              # nesne durumuna ihtiyac yok
        return yil % 4 == 0 and (yil % 100 != 0 or yil % 400 == 0)

t = Tarih.metinden("05.09.2026")
print(Tarih.artik_yil_mi(2026))
```

Dunder (Magic) Metotlar

> **SORU:** *__init__, __str__ ve __repr__ arasındaki fark nedir? __eq__ tanımlarsan başka ne tanımlaman gerekir?*

1. Mülakat Cevabı

> *"Dunder metotlar, adları çift alt çizgiyle çevrili özel metotlardır. Python'un yerleşik operatörlerinin ve fonksiyonlarının kendi sınıflarımızla nasıl çalışacağını tanımlarlar; bu yüzden Python'un veri modelinin temelini oluştururlar.*
>
> *__init__ kurucu değil, başlatıcıdır. Nesne zaten __new__ tarafından oluşturulmuştur; __init__ yalnızca onun başlangıç durumunu ayarlar ve None döndürür.*
>
> *__str__ ve __repr__ farkı sık sorulur. __str__ son kullanıcıya yönelik, okunabilir bir metin üretir ve print ile çağrılır. __repr__ ise geliştiriciye yönelik, mümkünse nesneyi yeniden oluşturacak kadar net bir temsil üretir; hata ayıklarken, REPL'de ve bir koleksiyonun içeriği yazdırılırken kullanılır. Yalnızca birini tanımlayacaksam __repr__ tanımlarım, çünkü __str__ yoksa Python otomatik olarak __repr__ metoduna düşer.*
>
> *__eq__ tanımladığımda mutlaka __hash__ metodunu da düşünmem gerekir. Çünkü __eq__ tanımlayan bir sınıfın __hash__ metodu Python tarafından otomatik olarak None yapılır ve nesne hashable olmaktan çıkar; yani sözlük anahtarı veya küme elemanı olamaz. Eşit nesnelerin aynı hash değerini üretmesi de bir zorunluluktur."*

2. Detaylı Arka Plan

__str__ ve __repr__ Kod Üzerinde

```Python
class Urun:
    def __init__(self, ad, fiyat):
        self.ad, self.fiyat = ad, fiyat

    def __str__(self):
        return f"{self.ad} ({self.fiyat} TL)"        # kullaniciya yonelik

    def __repr__(self):
        return f"Urun(ad={self.ad!r}, fiyat={self.fiyat!r})"   # gelistiriciye

u = Urun("Klavye", 750)

print(u)          # Klavye (750 TL)      -> __str__
print(repr(u))    # Urun(ad='Klavye', fiyat=750)  -> __repr__
print([u])        # [Urun(ad='Klavye', fiyat=750)] -> koleksiyonda __repr__
```

__eq__ ve __hash__ Birlikte

```Python
class Nokta:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        if not isinstance(other, Nokta):
            return NotImplemented
        return (self.x, self.y) == (other.x, other.y)

    def __hash__(self):
        return hash((self.x, self.y))     # esit nesneler AYNI hash'i uretmeli
```

```
a, b = Nokta(1, 2), Nokta(1, 2)
print(a == b)        # True
print(len({a, b}))   # 1  -> kume onlari ayni kabul etti

# __hash__ tanimlanmasaydi: TypeError: unhashable type: 'Nokta'
```

Sık Kullanılan Diğer Dunder Metotlar

- **__len__:** len(nesne) çağrısını mümkün kılar. Ayrıca boşluk kontrolünde (if nesne:) kullanılır.
- **__getitem__ ve __setitem__:** Köşeli parantezle erişimi (nesne[anahtar]) sağlar. __getitem__ tanımlıysa nesne for döngüsünde de gezilebilir hale gelir.
- **__enter__ ve __exit__:** Nesneyi with bloğunda kullanılabilir bir context manager yapar.
- **__call__:** Nesnenin fonksiyon gibi çağrılabilmesini sağlar (nesne()).
- **__add__, __lt__ gibi operatör metotları:** + ve < gibi operatörlerin davranışını tanımlar (operator overloading).

Pratik Kısayol: dataclass

Basit veri taşıyıcı sınıflarda tüm bu metotları elle yazmak yerine dataclass kullanmak hem daha kısadır hem de hata riskini azaltır.

```Python
from dataclasses import dataclass

@dataclass(frozen=True)     # frozen=True -> immutable ve otomatik hashable
class Nokta:
    x: int
    y: int

# __init__, __repr__, __eq__ ve (frozen ile) __hash__ otomatik uretilir
print(Nokta(1, 2))          # Nokta(x=1, y=2)
print(Nokta(1, 2) == Nokta(1, 2))   # True
```

Kalıtım ve MRO (Method Resolution Order)

> **SORU:** *Python çoklu kalıtımdaki elmas problemini nasıl çözer? MRO nedir? super() tam olarak neyi çağırır?*

1. Mülakat Cevabı

> *"Python, C++'ın aksine çoklu kalıtıma izin verir. Aynı metot birden fazla üst sınıfta bulunduğunda hangisinin çağrılacağını MRO yani Method Resolution Order belirler.*
>
> *MRO, C3 linearization adı verilen bir algoritmayla hesaplanır. Bu algoritma iki kuralı garanti eder: bir sınıf her zaman kendi üst sınıflarından önce gelir ve üst sınıfların bildirim sırası korunur. Sonuç, ClassName.mro() veya __mro__ özniteliğiyle görülebilen düz bir listedir.*
>
> *En sık yapılan hata super() fonksiyonunu 'üst sınıfı çağır' diye anlamaktır. Doğrusu şudur: super(), MRO listesinde bir sonraki sınıfa geçer. Çoklu kalıtımda bu, o anki sınıfın doğrudan ebeveyni olmayabilir; kardeş bir sınıf da olabilir. Elmas probleminde ortak atanın yalnızca bir kez çalıştırılmasını sağlayan mekanizma tam olarak budur.*
>
> *Bu yüzden çoklu kalıtım kullanan sınıf hiyerarşilerinde tüm sınıfların işbirlikçi (cooperative) yazılması, yani her birinin super() çağırması gerekir; aksi halde zincirin bir yerinde kopma olur."*

2. Detaylı Arka Plan

Elmas Problemi ve MRO

```Python
class A:
    def selam(self):
        print("A")

class B(A):
    def selam(self):
        print("B")
        super().selam()

class C(A):
    def selam(self):
        print("C")
        super().selam()

class D(B, C):        # elmas: D -> B, C -> A
    def selam(self):
        print("D")
        super().selam()

D().selam()
# D
# B
# C      <- super() B'nin ebeveyni A'ya DEGIL, MRO'daki sonraki olan C'ye gitti
# A      <- A yalnizca BIR KEZ calisti

print([k.__name__ for k in D.mro()])
# ['D', 'B', 'C', 'A', 'object']
```

Dikkat edilmesi gereken nokta: B sınıfındaki super().selam() çağrısı, B'nin ebeveyni olan A'ya değil, D örneği bağlamında MRO'da B'den sonra gelen C'ye gitti. super() statik bir "ebeveyn" değil, dinamik bir "sıradaki" anlamına gelir.

super() ile Kurucu Zinciri

```Python
class Temel:
```

```
    def __init__(self, ad, **kwargs):
        super().__init__(**kwargs)       # zinciri kirmadan devam ettir
        self.ad = ad

class Zaman_Damgali:
    def __init__(self, olusturma, **kwargs):
        super().__init__(**kwargs)
        self.olusturma = olusturma

class Kayit(Temel, Zaman_Damgali):
    pass

k = Kayit(ad="test", olusturma="2026-09-05")
print(k.ad, k.olusturma)
```

**kwargs ile argümanları zincir boyunca iletmek, işbirlikçi çoklu kalıtımın standart desenidir. Her sınıf kendi ihtiyacı olan argümanı alır, gerisini bir sonrakine geçirir.

Mülakat Notu: Çoklu Kalıtım Kullanmalı mı?

Olgun cevap, Python'un buna izin vermesinin her zaman kullanılması gerektiği anlamına gelmediğini belirtmektir. Derin ve karmaşık çoklu kalıtım hiyerarşileri okunması ve hata ayıklanması zor sistemler üretir. Pratikte çoklu kalıtım en çok mixin deseninde kullanılır: küçük, tek sorumluluğu olan ve durum tutmayan sınıfların davranış eklemek için karıştırılması. Bunun dışındaki durumlarda kalıtım yerine kompozisyon tercih edilir.

Hata Yönetimi: try, except, else ve finally

> **SORU:** *try / except / else / finally bloklarının akışını anlat. else ne zaman çalışır? Neden çıplak except kullanılmaz?*

1. Mülakat Cevabı

> *"Akış şöyledir: try bloğu, hata verebilecek kodu içerir. except bloğu, belirtilen türde bir istisna oluşursa çalışır. else bloğu, try içinde hiçbir istisna oluşmadıysa çalışır. finally bloğu ise istisna oluşsun ya da oluşmasın, hatta return ile çıkılsa bile her koşulda çalışır; bu yüzden kaynak temizliği için kullanılır.*
>
> *else bloğunun varlık sebebi şudur: try içine yalnızca gerçekten hata vermesini beklediğim minimum kodu koyarım, başarı durumunda çalışacak kodu else'e alırım. Böylece istemeden yanlış bir satırın istisnasını yakalamış olmam.*
>
> *Çıplak except, yani except: veya except Exception: yazmaktan kaçınırım; çünkü beklemediğim hataları da sessizce yutar, gerçek problemi gizler ve hata ayıklamayı imkansız hale getirir. Hatta çıplak except, KeyboardInterrupt gibi sistem çıkış istisnalarını bile yakalar. Bunun yerine her zaman beklediğim spesifik istisna türünü yakalarım.*
>
> *Python'un genel felsefesi de EAFP'dir: izin istemektense af dilemek daha kolaydır. Yani bir işlemi önce kontrol etmek yerine doğrudan denemek ve oluşan istisnayı yakalamak Pythonic kabul edilir."*

2. Detaylı Arka Plan

Dört Bloğun Akışı

```python
def veri_oku(yol):
    try:
        f = open(yol)
        veri = f.read()          # yalnizca hata verebilecek kod burada
    except FileNotFoundError:
        print("Dosya bulunamadi")
        return None
    except PermissionError as e:
        print(f"Yetki hatasi: {e}")
        raise                    # yeniden firlat: ust katman karar versin
    else:
        print("Okuma basarili")  # YALNIZCA istisna olmadiysa calisir
        return veri
    finally:
        print("Temizlik")        # HER durumda calisir (return'den once bile)
```

Mülakat Tuzağı: finally ve return Etkileşimi

```python
def f():
    try:
        return "try"
    finally:
        print("finally calisti")     # return'den ONCE calisir

print(f())
# finally calisti
# try

def g():
    try:
        return "try"
    finally:
        return "finally"             # KOTU PRATIK: try'in return'unu EZER
```

```
print(g())      # 'finally'  <- 'try' kaybolur
```

finally içinde return kullanmak, try bloğundaki dönüş değerini ve hatta fırlatılmış bir istisnayı sessizce yutar. Bu yüzden finally yalnızca temizlik için kullanılmalı, akışı değiştirmemelidir.

Özel İstisna Sınıfları ve Zincirleme

```Python
class VeriKalitesiHatasi(Exception):
    """Veri kalite kontrolu basarisiz oldugunda firlatilir."""

def dogrula(df):
    try:
        assert df["musteri_id"].notna().all()
    except AssertionError as e:
        # 'raise ... from e' orijinal hatayi zincire ekler,
        # traceback'te "The above exception was the direct cause of..." gorunur
        raise VeriKalitesiHatasi("musteri_id sutununda NULL var") from e
```

Kendi istisna sınıflarını tanımlamak, çağıran kodun hangi hatayı yakalayacağını netleştirir. Veri hatlarında bu, geçici bir ağ hatası ile kalıcı bir veri kalitesi hatasını ayırt edip birine yeniden deneme uygulamak, diğerinde hattı durdurmak için kritiktir.

EAFP ve LBYL

```Python
# LBYL (Look Before You Leap) - diger dillerde yaygin
if "anahtar" in sozluk:
    deger = sozluk["anahtar"]
else:
    deger = varsayilan

# EAFP (Easier to Ask Forgiveness than Permission) - Pythonic
try:
    deger = sozluk["anahtar"]
except KeyError:
    deger = varsayilan

# Bu ozel durumda en temizi:
deger = sozluk.get("anahtar", varsayilan)
```

EAFP'nin bir avantajı da yarış durumlarını (race condition) önlemesidir: dosyanın varlığını kontrol edip sonra açmak arasında dosya silinebilir, ancak doğrudan açmayı deneyip istisnayı yakalamak atomiktir.

Context Manager ve with Deyimi

> **SORU:** *with deyimi ne yapar ve neden try/finally yerine tercih edilir? Kendi context manager'ını nasıl yazarsın?*

1. Mülakat Cevabı

> *"with deyimi, bir kaynağın güvenli şekilde alınmasını ve blok bittiğinde mutlaka serbest bırakılmasını sağlar. Bloktan normal çıkışta da, bir istisna oluştuğunda da temizlik garanti edilir.*
>
> *Arka planda context manager protokolüne dayanır: nesne __enter__ ve __exit__ dunder metotlarını tanımlar. Bloğa girerken __enter__ çağrılır ve döndürdüğü değer as ile belirtilen isme bağlanır. Bloktan çıkarken, istisna olsun olmasın __exit__ çağrılır ve temizlik orada yapılır.*
>
> *try/finally ile aynı işi yapmak mümkündür, ancak with'i tercih ederim çünkü kaynak yönetimi mantığı bir kez sınıfın içine yazılır ve her kullanımda tekrarlanmaz. Bu hem kod tekrarını önler hem de geliştiricinin temizliği unutma ihtimalini ortadan kaldırır.*
>
> *Dosyaların yanında veritabanı bağlantıları, kilitler, geçici dizinler ve transaction yönetimi de tipik kullanım alanlarıdır. Basit durumlarda tam bir sınıf yazmak yerine contextlib.contextmanager decorator'ıyla generator tabanlı kısa bir context manager yazarım."*

2. Detaylı Arka Plan

Sınıf Tabanlı Context Manager

```python
class VeritabaniOturumu:
    def __init__(self, baglanti_dizesi):
        self.baglanti_dizesi = baglanti_dizesi

    def __enter__(self):
        self.baglanti = baglan(self.baglanti_dizesi)
        self.baglanti.begin()
        return self.baglanti            # 'as' ile baglanacak deger

    def __exit__(self, exc_type, exc_value, traceback):
        if exc_type is None:
            self.baglanti.commit()      # hata yoksa onayla
        else:
            self.baglanti.rollback()    # hata varsa geri al
        self.baglanti.close()
        return False                    # False -> istisna yukari yayilsin

with VeritabaniOturumu("postgres://...") as conn:
    conn.execute("INSERT INTO ...")
    # Hata olursa otomatik rollback, olmazsa commit; her durumda close
```

Kritik detay: __exit__ metodunun True döndürmesi, oluşan istisnanın bastırıldığı (yutulduğu) anlamına gelir. Bu nadiren istenir; genelde False döndürülür veya hiçbir şey döndürülmez ki istisna normal şekilde yukarı yayılsın.

Generator Tabanlı Kısa Yazım

```python
from contextlib import contextmanager
import time

@contextmanager
def sure_olc(etiket):
    basla = time.perf_counter()
    try:
```

```
        yield                               # yield ONCESI = __enter__
    finally:
        gecen = time.perf_counter() - basla # yield SONRASI = __exit__
        print(f"{etiket}: {gecen:.3f} sn")

with sure_olc("donusum adimi"):
    agir_islem()
```

Buradaki try/finally şarttır: istisna oluştuğunda da temizlik kodunun çalışmasını garanti eder. Bu olmadan generator tabanlı context manager hatalı durumda temizlik yapmaz.

Birden Fazla Kaynağı Yönetmek

```
Python
# Tek satirda birden cok context manager
with open("girdi.csv") as gir, open("cikti.csv", "w") as cik:
    for satir in gir:
        cik.write(donustur(satir))

# Sayisi calisma aninda belli olan kaynaklar icin:
from contextlib import ExitStack

with ExitStack() as stack:
    dosyalar = [stack.enter_context(open(y)) for y in yollar]
    # blok bitince hepsi otomatik kapanir
```

Mülakat Notu: Neden with Olmadan Dosya Açmamalı?

open() ile açılıp close() çağrılmayan bir dosya, CPython'da referans sayacı sıfırlandığında genellikle kapanır; ancak bu bir dil garantisi değildir ve PyPy gibi uygulamalarda gecikmeli olur. Ayrıca yazma işlemlerinde tampon (buffer) diske boşaltılmayabilir ve veri kaybı yaşanır. Uzun süren süreçlerde açık kalan tanıtıcılar işletim sistemi limitine dayanıp "too many open files" hatasına yol açar. Bu yüzden dosya işlemleri istisnasız with ile yapılmalıdır.

Sanal Ortamlar, PEP 8 ve Test

> **SORU:** *Sanal ortam neden kullanılır? PEP 8 nedir? Python'da testleri nasıl yazar ve organize edersin?*

1. Mülakat Cevabı

> *"Sanal ortam, her projenin kendi bağımlılıklarını izole bir dizinde tutmasını sağlar. Buna ihtiyaç duymamızın sebebi, farklı projelerin aynı kütüphanenin farklı sürümlerine ihtiyaç duyabilmesidir; hepsini sistem genelinde kurmak kaçınılmaz olarak sürüm çakışmalarına yol açar. venv standart kütüphaneyle gelir; conda ayrıca Python sürümünü ve Python dışı ikili bağımlılıkları da yönetir. Üretim ortamlarında sürümlerin birebir tekrarlanabilir olması için bir kilit dosyası (lock file) tutmak da önemlidir.*
>
> *PEP 8, Python'un resmi stil kılavuzudur. Fonksiyon ve değişkenler için snake_case, sınıflar için CamelCase, sabitler için BÜYÜK_HARF; girinti için dört boşluk; satır uzunluğu için 79 karakter önerilir. Günümüzde bu kuralları elle uygulamak yerine black gibi bir biçimlendirici ve ruff veya flake8 gibi bir linter kullanılır; böylece stil tartışmaları kod incelemesinden tamamen çıkar.*
>
> *Test tarafında pytest tercih ederim. unittest standart kütüphanede bulunur ve sınıf tabanlıdır; pytest ise sade assert ifadeleriyle çalıştığı, fixture mekanizmasıyla kurulum ve temizliği zarif şekilde yönettiği ve parametrize ile aynı testi farklı girdilerle kolayca çalıştırabildiği için daha üretkendir.*
>
> *Veri hatlarında testi iki katmanda düşünürüm: kod mantığını doğrulayan birim testleri ve verinin kendisini doğrulayan veri kalite testleri. İkincisi için dbt testleri veya Great Expectations gibi araçlar kullanılır."*

2. Detaylı Arka Plan

Sanal Ortam Kullanımı

```bash
# Olusturma ve etkinlestirme
python -m venv .venv
source .venv/bin/activate        # Linux / macOS
.venv\Scripts\activate           # Windows

# Bagimliliklari dondurma
pip install -r requirements.txt
pip freeze > requirements.txt

# Modern alternatifler: bagimlilik cozumleme ve kilit dosyasi yonetir
# poetry add pandas
# uv pip install pandas
```

Mülakatta requirements.txt ile lock dosyası farkını bilmek değerlidir: pip freeze çıktısı o anki kurulu sürümleri dondurur ama bağımlılıkların bağımlılıklarını çözümlemez. Poetry veya uv gibi araçlar tam bir bağımlılık grafiği çözümleyip kilitleyerek üretimde birebir aynı ortamı garanti eder.

PEP 8 Özet Kuralları

Öğe	Kural	Örnek
Değişken / fonksiyon	snake_case	musteri_sayisi, veri_isle()
Sınıf	CamelCase	VeriYukleyici
Sabit	BÜYÜK_HARF	MAX_DENEME = 3
Dahili kullanım	Tek alt çizgi öneki	_yardimci_fonksiyon()
Girinti	4 boşluk (tab değil)	—
İçe aktarma sırası	standart, üçüncü parti, yerel	os → pandas → myapp

pytest ile Test Yazımı

```python
import pytest
from etl import tutar_normalize

# Basit test: sade assert yeterli
def test_tutar_normalize_virgulu_noktaya_cevirir():
    assert tutar_normalize("1.234,56") == 1234.56

# Parametrize: ayni testi farkli girdilerle calistir
@pytest.mark.parametrize("girdi, beklenen", [
    ("1.234,56", 1234.56),
    ("0,00",        0.00),
    ("999",       999.00),
])
def test_tutar_normalize(girdi, beklenen):
    assert tutar_normalize(girdi) == beklenen

# Istisna beklemek
def test_gecersiz_tutar_hata_firlatir():
    with pytest.raises(ValueError, match="gecersiz"):
        tutar_normalize("abc")

# Fixture: kurulum ve temizlik
@pytest.fixture
def ornek_df():
    df = pd.DataFrame({"musteri_id": [1, 2, None]})
    yield df                 # test bu degeri alir
    # yield sonrasi temizlik kodu buraya yazilir

def test_null_temizleme(ornek_df):
    assert temizle(ornek_df)["musteri_id"].notna().all()
```

Veri Hatlarında Ne Test Edilir?

- **Dönüşüm fonksiyonları:** Saf (yan etkisiz) fonksiyonlar olarak yazıldıklarında birim testi çok kolaydır. Bu yüzden iş mantığını I/O'dan ayırmak iyi bir tasarımdır.
- **Sınır durumları:** Boş girdi, NULL değerler, beklenmeyen tipler, çok büyük sayılar ve karakter kodlaması sorunları.
- **Idempotency:** Aynı işi iki kez çalıştırıp sonucun değişmediğini doğrulayan test, veri hatlarında en değerli testlerden biridir.
- **Veri kalitesi:** Kodun değil verinin doğruluğunu ölçer; benzersizlik, NULL olmama, referans bütünlüğü ve beklenen değer aralığı kontrolleri.

Mülakat Notu: Test Kapsamı (Coverage) Yeterli Bir Ölçüt mü?

Olgun cevap hayırdır. Yüzde yüz satır kapsamı, kodun her satırının çalıştırıldığını gösterir ama doğru davrandığını kanıtlamaz; assert içermeyen bir test bile kapsamı yükseltir. Kapsam, test edilmemiş bölgeleri bulmak için yararlı bir sinyaldir, ancak kalite hedefi olarak tek başına kullanılmamalıdır. Asıl önemli olan kritik iş mantığının ve sınır durumlarının anlamlı biçimde doğrulanmasıdır.

Veri Mühendisliği: Sistem Tasarımı ve Mimari

ETL ve ELT

> **SORU:** *ETL ile ELT arasındaki fark nedir? Modern veri yığınında neden ELT'ye geçildi? Hangi durumda hâlâ ETL tercih edersin?*

1. Mülakat Cevabı

> *"Fark, dönüşümün (Transform) nerede yapıldığıdır.*
>
> *ETL'de veri kaynaktan çekilir, ayrı bir işleme sunucusunda (örneğin SSIS, Informatica veya bir Spark kümesi) dönüştürülür ve ancak temizlenmiş haliyle hedef veri ambarına yüklenir. Bu yaklaşım, veri ambarının işlem gücünün pahalı ve sınırlı olduğu dönemde doğmuştur.*
>
> *ELT'de ise ham veri önce olduğu gibi hedefe yüklenir, dönüşüm de hedefin kendi işlem gücü kullanılarak orada yapılır. Snowflake, BigQuery ve Databricks gibi modern ambarların elastik ve güçlü MPP mimarisi bunu mümkün kıldı.*
>
> *ELT'ye geçişin üç temel sebebi var: Depolama artık çok ucuz olduğu için ham veriyi saklamak sorun değil; depolama ile işlem gücü birbirinden ayrıldığı için ihtiyaç anında ölçeklenebiliyoruz; ve en önemlisi ham veriyi sakladığımız için iş kuralı değiştiğinde kaynağa geri dönmeden geçmişi yeniden işleyebiliyoruz (reprocessing).*
>
> *Buna rağmen ETL'yi hâlâ tercih ettiğim durumlar var: Kişisel veri (PII) içeren kayıtların ambara hiç ham haliyle inmemesi gerekiyorsa, maskeleme veya şifreleme yükleme öncesinde yapılmalıdır. Ayrıca kaynak sistemden çok büyük hacimli veriyi gereksiz yere taşımak yerine, kaynağa yakın bir yerde filtreleyip küçültmek daha mantıklı olabilir."*

2. Detaylı Arka Plan

Karşılaştırma Tablosu

Kriter	ETL	ELT
Dönüşüm yeri	Ayrı işleme sunucusu	Hedef veri ambarının içinde
Hedefe inen veri	Yalnızca işlenmiş veri	Ham veri + türetilmiş katmanlar
Yeniden işleme	Zor; kaynağa dönmek gerekebilir	Kolay; ham veri zaten ambarda
Ölçeklenme	İşleme sunucusuyla sınırlı	Ambarın elastik gücüyle
Tipik araçlar	SSIS, Informatica, Talend	Fivetran/Airbyte + dbt + Snowflake
Uygunluk/PII	Yükleme öncesi maskeleme kolay	Ham PII ambara iner, yönetim gerekir

Mülakat Notu: ELT'nin Gizli Maliyeti

Mülakatçı "ELT'nin dezavantajı yok mu?" diye sorabilir. Olgun cevap şudur: ELT'de dönüşüm maliyeti doğrudan ambar faturasına yansır. Kötü yazılmış bir dbt modeli veya her çalıştırmada tüm tabloyu baştan üreten (full refresh) bir dönüşüm, bulut maliyetlerini hızla patlatır. Bu yüzden incremental (artımlı) model yazmak, gereksiz taramayı önlemek ve partition/cluster anahtarlarını doğru seçmek ELT dünyasında maliyet mühendisliğinin (FinOps) parçasıdır.

Batch ve Streaming İşleme

> **SORU:** *Batch ve streaming işleme arasındaki farklar nelerdir? Streaming'de geç gelen veriyi (late-arriving data) nasıl ele alırsın?*

1. Mülakat Cevabı

> *"Batch işleme, sınırlı (bounded) bir veri kümesini belirli aralıklarla toplu olarak işler; örneğin her gece çalışan bir Airflow DAG'ının Spark job'unu tetiklemesi. Yüksek verimlilik (throughput) sunar, geliştirmesi ve hata ayıklaması kolaydır, ancak gecikme (latency) saatler mertebesindedir.*
>
> *Streaming işleme ise sınırsız (unbounded) bir veri akışını olay geldikçe sürekli işler; Kafka, Flink veya Spark Structured Streaming ile kurulur. Gecikme saniyeler hatta milisaniyeler seviyesindedir ama operasyonel karmaşıklığı çok daha yüksektir.*
>
> *Streaming'in en kritik kavramı olay zamanı (event time) ile işlenme zamanı (processing time) ayrımıdır. Bir mobil cihaz çevrimdışıyken üretilen olay, sisteme saatler sonra ulaşabilir. Bu yüzden pencereleme işlemlerini işlenme zamanına göre değil olay zamanına göre yaparım.*
>
> *Geç gelen veriyi yönetmek için watermark kullanırım: sisteme 'bu zaman damgasından daha eski olayları artık beklemiyorum' sınırını söylerim. Watermark içinde kalan geç olaylar pencereye dahil edilip sonuç güncellenir. Watermark'ı aşan olayları ise sessizce düşürmek yerine ayrı bir kanala (dead-letter veya side output) yazarım; böylece veri kaybı yaşanmaz ve gerekirse batch bir düzeltme işiyle sonradan hesaba katılır."*

2. Detaylı Arka Plan

Pencereleme (Windowing) Türleri

- **Tumbling Window:** Sabit boyutlu, birbiriyle örtüşmeyen ardışık pencereler (her 5 dakika). Her olay tek bir pencereye düşer.
- **Sliding Window:** Sabit boyutlu ama kayan pencereler (son 10 dakika, her 1 dakikada bir hesapla). Bir olay birden fazla pencerede yer alabilir.
- **Session Window:** Sabit boyut yoktur; belirli bir hareketsizlik süresi (gap) geçene kadar süren pencere. Kullanıcı oturumu analizinde kullanılır.

Teslimat Garantileri

- **At-most-once:** Olay en fazla bir kez işlenir. Kayıp olabilir, tekrar olmaz. Nadiren kabul edilebilir.
- **At-least-once:** Olay en az bir kez işlenir. Kayıp olmaz ama tekrar (duplicate) olabilir. En yaygın varsayılan garantidir.
- **Exactly-once:** Olay etkisi tam olarak bir kez yansır. Pratikte çoğu sistemde bu, at-least-once teslimat ile idempotent bir hedefin (dedup anahtarı veya transactional yazma) birleşimiyle sağlanır.

Lambda ve Kappa Mimarileri

Lambda mimarisi, aynı veriyi hem batch hem streaming katmanında işleyip sonuçları serving katmanında birleştirir. Doğruluğu batch katmanı garanti eder, hızı streaming katmanı sağlar. Dezavantajı aynı iş mantığının iki ayrı kod tabanında yazılıp bakımının yapılmasıdır.

Kappa mimarisi ise batch katmanını tamamen kaldırır; her şey streaming olarak işlenir ve geçmişi yeniden işlemek gerektiğinde log (örneğin Kafka topic) baştan tekrar okunur. Tek kod tabanı avantajı sağlar; ancak log'un yeterince uzun süre saklanmasını gerektirir.

Karşılaştırma Tablosu

Kriter	Batch	Streaming
Veri sınırı	Bounded (sınırlı küme)	Unbounded (sürekli akış)
Gecikme	Dakikalar - saatler	Milisaniye - saniyeler
Verimlilik	Çok yüksek throughput	Daha düşük ama sürekli
Karmaşıklık	Düşük; hata ayıklaması kolay	Yüksek; state ve zaman yönetimi gerekir
Yeniden çalıştırma	Kolay (aynı partition'ı tekrar üret)	Zor (offset/checkpoint yönetimi)
Tipik araçlar	Airflow, Spark, dbt	Kafka, Flink, Spark Structured Streaming

Data Lake, Data Warehouse ve Lakehouse

> **SORU:** *Data Lake, Data Warehouse ve Lakehouse arasındaki farkı açıkla. Lakehouse hangi problemi çözüyor?*

1. Mülakat Cevabı

> *"Data Warehouse, veriyi önceden tanımlanmış bir şemaya göre yükleyen (schema-on-write), yapılandırılmış ve iş kurallarından geçmiş bir sistemdir. Güçlü SQL desteği, ACID garantileri ve BI araçlarıyla mükemmel uyumu vardır. Dezavantajı pahalı olması ve yapılandırılmamış veriyi (görüntü, ses, log, PDF) doğal olarak barındıramamasıdır.*
>
> *Data Lake ise ham veriyi olduğu gibi, herhangi bir formatta ve ucuz nesne depolamada (S3, ADLS, GCS) tutar. Şema okuma anında uygulanır (schema-on-read). Çok esnektir ve makine öğrenmesi iş yükleri için idealdir. Ancak yönetişim, şema kontrolü ve ACID garantisi olmadığı için kolayca 'data swamp' yani içinden bir şey çıkarılamayan veri bataklığına dönüşebilir.*
>
> *Lakehouse, bu ikisinin arasındaki tercihi ortadan kaldırmak için doğdu. Delta Lake, Apache Iceberg veya Hudi gibi açık tablo formatları sayesinde, ucuz nesne depolamanın üzerine veri ambarı özelliklerini getirir: ACID transaction, şema zorlama ve şema evrimi, zaman yolculuğu (time travel), satır seviyesinde güncelleme ve silme. Böylece tek bir kopya veri üzerinde hem BI raporlaması hem de makine öğrenmesi çalışabilir; veriyi iki ayrı sistemde çoğaltmak zorunda kalmayız."*

2. Detaylı Arka Plan

Açık Tablo Formatları Nasıl Çalışır?

Delta Lake ve Iceberg'in temel fikri şudur: veri dosyaları (genellikle Parquet) nesne depolamada durur, ancak yanlarında bir işlem günlüğü (transaction log) veya metadata katmanı tutulur. Bu katman, hangi dosyaların tablonun hangi versiyonuna ait olduğunu kaydeder.

Bu sayede: Bir yazma işlemi ya tamamen görünür olur ya da hiç görünmez (atomiklik); okuyucular yazma sırasında tutarlı bir anlık görüntü görür (snapshot isolation); eski versiyonlara dönüp sorgu çalıştırılabilir (time travel); ve nesne depolamada normalde imkansız olan UPDATE/DELETE işlemleri, ilgili dosyaların yeniden yazılmasıyla mümkün hale gelir.

```sql
-- Delta Lake: zaman yolculugu ile dunku veriyi sorgulama
SELECT * FROM satis VERSION AS OF 42;
SELECT * FROM satis TIMESTAMP AS OF '2024-05-01 00:00:00';

-- Nesne depolamada satir seviyesinde silme (GDPR talebi gibi)
DELETE FROM musteri WHERE musteri_id = 12345;
```

Karşılaştırma Tablosu

Kriter	Data Warehouse	Data Lake	Lakehouse
Şema	schema-on-write	schema-on-read	Her ikisi (zorlama + evrim)
Veri tipi	Yapılandırılmış	Her tür (ham)	Her tür
ACID	Var	Yok	Var (tablo formatı ile)
Maliyet	Yüksek	Çok düşük	Düşük
Ana kullanıcı	BI / analist	Veri bilimci / ML	Her ikisi
Risk	Esneklik kaybı	Data swamp	Operasyonel olgunluk gerektirir

Hangi Senaryoda Hangisi? Karar Kriterleri

Mülakatta bu üçlü sorulduğunda tanımları saymak yetmez; "senin projende hangisini seçerdin ve neden" sorusu mutlaka gelir. Kararı beş soruyla veririm:

- Şema ne kadar belli? Kaynak yapısı sabit ve iş kuralları netse Warehouse; şema sık değişiyor veya veriyle ne yapacağın henüz belli değilse Lake.
- Veriyi kim tüketecek? SQL yazan analist ve BI aracı ise Warehouse; Python ile çalışan veri bilimci ve ML hattı ise Lake veya Lakehouse.
- Veri tipi nedir? Satır-sütun yapısındaki işlem verisi Warehouse'a uygundur. Görüntü, ses, PDF, ham log ve JSON gibi yapılandırılmamış veriler Warehouse'a sığmaz; Lake zorunludur.
- ACID ve satır seviyesinde silme gerekiyor mu? Düzeltme, güncelleme veya KVKK kapsamında silme talebi varsa klasik Lake yetmez; Lakehouse (Delta, Iceberg) gerekir.
- Ekip olgunluğu ve bütçe ne durumda? Lakehouse en esnek çözümdür ama tablo formatı yönetimi, dosya sıkıştırma ve versiyon temizliği gibi operasyonel yük getirir. Küçük ekipte bu yük projeyi yavaşlatır.

Somut Senaryolar

Senaryo	Tercih	Gerekçe
Aylık ve günlük finansal raporlama; kaynak sistemler sabit, kullanıcı analist	Data Warehouse	Şema belli, SQL ve BI aracı yeterli. Lake eklemek gereksiz karmaşıklık yaratır.
Mobil uygulamadan gelen ham clickstream ve log; ileride ne analiz edileceği belirsiz	Data Lake	Şema değişken, hacim çok yüksek, ucuz depolama şart. Önce topla, sonra anlamlandır.
Aynı veri hem BI raporunda hem ML modelinde kullanılacak; KVKK silme talepleri geliyor	Lakehouse	Tek kopya veri üzerinde hem SQL hem ML; ACID ve satır seviyesinde silme gerekiyor.
50 GB veri, 3 kaynak sistem, 5 kişilik ekip	Yalnızca Data Warehouse	Bu ölçekte Lake katmanı kurmak fayda değil bakım yükü getirir. Aşırı mühendislikten kaçın.
Tarama görüntüleri ve PDF'lerden bilgi çıkarma	Data Lake (+ Warehouse)	İkili (binary) dosyalar Lake'te durur; çıkarılan yapılandırılmış alanlar Warehouse'a yazılır.
Regülasyon 10 yıl ham veri saklamayı zorunlu kılıyor	Lake arşiv + Warehouse aktif	Ham veri ucuz nesne depolamada arşivlenir; yalnızca son 2 yıl ambarda sıcak tutulur.
Gerçek zamanlı dolandırıcılık tespiti	Lakehouse veya akış deposu	Streaming yazma ile analitik okumanın aynı tablo üzerinde çalışması gerekir.

Maliyet ve Performans Dengesi

Üç mimarinin maliyeti farklı yerlerde birikir ve bu, seçimi doğrudan etkiler.

- **Data Warehouse:** Depolama pahalıdır çünkü genellikle işlem gücüyle birlikte fiyatlanır veya özel donanımda durur. Buna karşılık sorgu performansı öngörülebilirdir ve ek bir optimizasyon çabası gerektirmez.
- **Data Lake:** Depolama çok ucuzdur (nesne depolama), ancak sorgu maliyeti taranan veri miktarıyla doğru orantılıdır. Kötü bölümlenmiş (partition) veya çok sayıda küçük dosyaya bölünmüş bir Lake'te tek bir sorgu beklenmedik şekilde pahalıya gelebilir.

- **Lakehouse:** Depolama Lake kadar ucuzdur ve tablo formatı sayesinde performans Warehouse'a yaklaşır. Bedeli operasyonel bakımdır: düzenli dosya sıkıştırma (compaction), eski versiyonların temizlenmesi (vacuum) ve istatistik yönetimi gerekir. Bu işler yapılmazsa performans zamanla bozulur.

Yaygın Anti-Pattern'ler

- **"Önce Lake kuralım, sonra ne yapacağımıza bakarız":** Kataloglama, şema yönetimi ve sahiplik tanımlanmadan kurulan Lake, birkaç ay içinde kimsenin içinden bir şey çıkaramadığı bir veri bataklığına (data swamp) dönüşür. Lake kurmak ucuzdur; yönetmek değildir.
- **Lake'i Warehouse gibi kullanmaya çalışmak:** Nesne depolamada satır seviyesinde güncelleme yapmaya veya binlerce küçük dosya üzerinden interaktif sorgu çalıştırmaya çalışmak. Bu ihtiyaç varsa doğru cevap Lakehouse'a geçmektir.
- **Warehouse'a ham JSON dökmek:** Ambarın pahalı işlem gücünü ayrıştırma işine harcamak. Ham ve yarı yapılandırılmış veri Lake katmanında durmalı, ambara temizlenmiş haliyle girmelidir.
- **Basit ihtiyaca Lakehouse kurmak:** Üç kaynak ve birkaç yüz gigabaytlık veri için Delta Lake, tablo formatı bakımı ve compaction işleri kurmak, çözdüğünden fazla sorun yaratır.
- **Lake ve Warehouse arasında veri kopyalarının çoğalması:** Aynı tablonun Lake'te, ambarda ve bir de BI aracının önbelleğinde farklı versiyonlarının bulunması. Hangi kopyanın doğru olduğu bilinemez hale gelir; tek yönlü ve tanımlı bir akış kurmak şarttır.

Mülakat Notu: "Hepsini birden kullanır mıyız?"

Evet ve pratikte en yaygın kurulum budur. Ham veri Lake'e iner, Lakehouse tablo formatıyla temizlenip işlenir, iş birimlerinin doğrudan tükettiği özet katman ise ya aynı Lakehouse içinde Gold katmanı olarak ya da ayrı bir Warehouse'ta tutulur. Yani soru genellikle "hangisi" değil, "hangi katmanda hangisi" sorusudur. Bu ayrımı yapabilmek, üçünü birbirinin alternatifi sanan adaylardan seni ayırır.

Medallion Mimarisi (Bronze / Silver / Gold)

> **SORU:** *Medallion mimarisi nedir? Neden veriyi tek adımda temizleyip son haline getirmiyoruz?*

1. Mülakat Cevabı

> *"Medallion mimarisi, bir Lakehouse içindeki veriyi kalite seviyesine göre kademeli olarak iyileştiren üç katmanlı bir düzenleme desenidir.*
>
> *Bronze katmanı ham veriyi kaynaktan geldiği gibi, hiçbir iş kuralı uygulamadan saklar. Yalnızca teknik metadata eklenir: yükleme zamanı, kaynak dosya adı, batch kimliği. Bu katman append-only ve değiştirilemez kabul edilir; sistemin tek gerçek kaynağıdır (source of truth).*
>
> *Silver katmanı temizlenmiş, tipleri düzeltilmiş, tekrarları elenmiş ve farklı kaynaklardan gelen verinin ortak bir modele uyumlandırıldığı (conformed) katmandır. Veri kalite kontrolleri burada uygulanır. Genellikle normalize bir yapıdadır ve varlık (entity) bazlıdır.*
>
> *Gold katmanı ise iş birimlerinin doğrudan tükettiği, toplulaştırılmış ve denormalize edilmiş katmandır. Star şema, KPI tabloları ve makine öğrenmesi için hazırlanmış özellik (feature) tabloları burada yaşar.*
>
> *Tek adımda yapmama sebebim şudur: Bronze katmanı sayesinde bir iş kuralı yanlış uygulandığında veya yeni bir ihtiyaç doğduğunda kaynak sisteme geri dönmeden geçmişin tamamını yeniden işleyebilirim. Ayrıca hata ayıklarken problemin hangi katmanda oluştuğunu izole edebilirim; ham veri mi bozuk, dönüşüm mü hatalı, yoksa toplulaştırma mı yanlış, bunu net görürüm."*

2. Detaylı Arka Plan

Katmanların Sorumlulukları

Katman	İçerik	Tipik İşlemler
Bronze (Raw)	Ham, değiştirilmemiş kaynak verisi	Append-only yazma, teknik metadata ekleme, şema evrimi toleransı
Silver (Cleansed)	Temiz, tekilleştirilmiş, uyumlanmış	Tip dönüşümü, deduplication, NULL yönetimi, veri kalite testleri, birleştirme
Gold (Curated)	İş seviyesinde toplulaştırılmış	Star şema, KPI hesaplama, agregasyon, ML feature üretimi

Mülakat Sorusu: Bronze Katmanına Ne Kadar Dokunulur?

Doğru cevap: Mümkün olan en az. Bronze'a iş mantığı uygulanmaz; sütun yeniden adlandırılmaz, filtre uygulanmaz, hatta bozuk kayıtlar bile silinmez. Amaç, kaynağın o andaki halinin birebir arşivini tutmaktır. Bozuk kayıtları temizlemek Silver katmanının işidir ve orada da silmek yerine karantina (quarantine) tablosuna almak tercih edilir; böylece hiçbir veri sessizce kaybolmaz.

Veri Kalite Kontrolleri Nerede Yapılır?

Silver katmanına geçişte uygulanır ve genellikle şu kontrolleri içerir: birincil anahtarın benzersizliği ve NULL olmaması, referans bütünlüğü (Gold'da beklenen boyut anahtarlarının var olması), kabul edilen değer kümesi kontrolü, sayısal alanlarda mantıklı aralık kontrolü ve beklenen satır sayısı sapması. dbt kullanılıyorsa bunlar unique, not_null, relationships ve accepted_values testleri olarak tanımlanır; hattı kırarak bozuk verinin Gold'a ulaşmasını engeller.

Idempotency, Backfill ve Hata Modları

> **SORU:** *Idempotent bir veri hattı ne demektir? Aynı işi iki kez çalıştırırsan ne olur? Backfill'i nasıl güvenli yaparsın?*

1. Mülakat Cevabı

> *"Bir veri hattı, aynı girdiyle kaç kez çalıştırılırsa çalıştırılsın hedefte hep aynı sonucu üretiyorsa idempotenttir. Bu, veri mühendisliğinin en temel tasarım prensibidir; çünkü hatalar kaçınılmazdır ve bir işi güvenle yeniden çalıştırabilmek zorundayız.*
>
> *Idempotency'yi kıran en yaygın hata, hedefe koşulsuz INSERT yapmaktır; iş iki kez çalışırsa veri ikiye katlanır. Bunu üç yöntemle çözerim.*
>
> *Birincisi partition overwrite: hedefi tarih gibi bir anahtara göre bölerim ve her çalıştırmada yalnızca ilgili partition'ı silip yeniden yazarım. Böylece o günün verisi kaç kez üretilirse üretilsin sonuç aynıdır.*
>
> *İkincisi MERGE veya UPSERT: sabit bir iş anahtarı üzerinden eşleşen kaydı günceller, olmayanı eklerim.*
>
> *Üçüncüsü deterministik kayıt anahtarı: kaydın içeriğinden türetilen bir hash'i birincil anahtar yaparım; aynı kayıt tekrar gelirse aynı anahtarı üretir ve çakışma sayesinde çift kayıt oluşmaz.*
>
> *Backfill'de ise işi tarih parametresiyle çalışacak şekilde yazarım; iş asla 'bugün' gibi çalışma anına bağlı bir değere göre değil, kendisine verilen mantıksal tarihe göre çalışmalıdır. Ayrıca backfill'i üretim yükünü etkilememesi için sınırlı paralellikle ve mümkünse ayrı bir işlem havuzunda çalıştırırım."*

2. Detaylı Arka Plan

Idempotent Yükleme Kalıpları

```SQL
-- KALIP 1: Partition overwrite (delete + insert, tek transaction icinde)
BEGIN;
  DELETE FROM gunluk_satis WHERE islem_tarihi = DATE '2024-05-01';
  INSERT INTO gunluk_satis (islem_tarihi, magaza_id, ciro)
  SELECT islem_tarihi, magaza_id, SUM(tutar)
  FROM ham_satis
  WHERE islem_tarihi = DATE '2024-05-01'
  GROUP BY islem_tarihi, magaza_id;
COMMIT;
-- Kac kez calisirsa calissin sonuc aynidir.

-- KALIP 2: Deterministik anahtar ile cift kayit engelleme
INSERT INTO olay (olay_id, kullanici_id, olay_tipi, olay_zamani)
SELECT
    MD5(kullanici_id || '|' || olay_tipi || '|' || olay_zamani) AS olay_id,
    kullanici_id, olay_tipi, olay_zamani
FROM kaynak_olay
ON CONFLICT (olay_id) DO NOTHING;   -- ayni olay tekrar gelirse yok sayilir
```

Kısmi Yazma (Partial Write) Problemi ve Çözümü

Bir iş, hedefe 10 milyon satırdan 6 milyonunu yazdıktan sonra çökerse, hedef tablo tutarsız bir ara durumda kalır. Raporlar bu sırada yanlış sonuç üretir. Üç yaygın çözüm vardır:

- **Transaction ile atomiklik:** Tüm yazmayı tek transaction'a alıp ya hepsini ya hiçbirini uygulamak. Çok büyük hacimlerde log baskısı yaratabilir.
- **Staging + atomic swap:** Veriyi önce geçici bir tabloya yazmak, başarıyla tamamlandığında tabloları takas etmek (rename veya partition switch). Okuyucular hiçbir an yarım veri görmez.

- **Write-Audit-Publish (WAP):** Veriyi yaz, kalite testlerini çalıştır, yalnızca testler geçerse yayınla. Iceberg ve Delta gibi formatlarda branch/snapshot mekanizmasıyla doğal olarak desteklenir.

Mülakat Senaryosu: "Gece işi 3'te çöktü, sabah fark ettin. Ne yaparsın?"

Beklenen cevabın yapısı şudur: Önce etki alanını belirlerim; hangi tablolar hangi tarih aralığında etkilendi ve bu tabloları tüketen aşağı akış (downstream) işler çalıştı mı? Hattım idempotent yazıldıysa çözüm basittir: aynı işi aynı mantıksal tarih parametresiyle yeniden çalıştırırım, veri kendini düzeltir. İdempotent değilse önce kısmi yazılan veriyi temizlemem gerekir. Ardından aşağı akış işlerini bağımlılık sırasına göre yeniden tetiklerim. Son olarak da olayı kalıcı olarak çözmek için hattı idempotent hale getirir ve bu tür sessiz başarısızlıkları erken yakalayacak izleme/alarm (freshness ve satır sayısı kontrolü) eklerim.

Airflow Bağlamında Idempotency

Airflow'da her DAG çalıştırmasının bir mantıksal tarihi (logical date / data interval) vardır. Doğru yazılmış bir task, veriyi CURRENT_DATE yerine bu parametreye göre işler. Bunun iki faydası vardır: geçmiş bir tarihi yeniden çalıştırdığında (backfill veya clear) doğru veri üretilir ve sonuç her zaman aynı olur.

```Python
# YANLIS: calisma anina bagimli, backfill'de yanlis veri uretir
sql = "SELECT * FROM satis WHERE tarih = CURRENT_DATE - 1"

# DOGRU: mantiksal tarihe bagli, tekrar calistirilabilir
sql = "SELECT * FROM satis WHERE tarih = '{{ ds }}'"
```

Veri Mühendisliği: Veri Modelleme

Boyutsal Modelleme: Star ve Snowflake Şema

> **SORU:** *Kimball'ın boyutsal modellemesini anlat. Fact ve Dimension tablosu nedir? Star ile Snowflake şema arasındaki fark nedir?*

1. Mülakat Cevabı

> *"Boyutsal modelleme, analitik sorguları hem hızlı hem de iş kullanıcısı için anlaşılır kılmayı hedefleyen bir veri ambarı tasarım yaklaşımıdır. Veriyi iki tür tabloya ayırır.*
>
> *Fact (olgu) tablosu ölçülebilir iş olaylarını tutar: satış tutarı, adet, süre gibi sayısal metrikler ve boyut tablolarına işaret eden yabancı anahtarlar. Satır sayısı çok yüksektir ama sütun sayısı azdır.*
>
> *Dimension (boyut) tablosu ise bu olayları tanımlayan betimleyici öznitelikleri tutar: müşteri bilgileri, ürün özellikleri, tarih, mağaza. Satır sayısı görece azdır ama sütun sayısı fazladır; çünkü kullanıcının filtreleyeceği ve grupladığı her şey buradadır.*
>
> *Star şemada boyut tabloları denormalize edilir; her boyut tek bir tablodur ve fact tablosuna doğrudan bağlanır. Sorgular tek seviye JOIN gerektirdiği için hızlı ve anlaşılırdır. Snowflake şemada ise boyutlar normalize edilerek alt tablolara bölünür; depolamadan tasarruf sağlar ama JOIN sayısını ve karmaşıklığı artırır.*
>
> *Pratikte neredeyse her zaman Star şemayı tercih ederim; depolama artık ucuz, asıl maliyet sorgu performansı ve kullanıcının modeli anlayabilmesidir. Ve en önemlisi, modellemeye başlamadan önce mutlaka fact tablosunun taneciğini (grain) yani bir satırın tam olarak neyi temsil ettiğini net şekilde tanımlarım."*

2. Detaylı Arka Plan

Grain (Tanecik): En Kritik Karar

Kimball'ın dört adımlı sürecinde ikinci ve en kritik adım grain'i ilan etmektir: "Bu fact tablosunda bir satır neyi temsil ediyor?" Örneğin "bir satış fişindeki bir ürün kalemi" veya "bir müşterinin bir gündeki toplam harcaması" tamamen farklı iki modeldir.

Grain net tanımlanmazsa, farklı taneciklerdeki veriler aynı tabloya karışır ve toplamlar sessizce yanlış çıkar. Mülakatta modelleme sorusu geldiğinde ilk cümlen "Önce grain'i belirlerim" olmalıdır; bu, deneyimli bir modelleyici olduğunun en güçlü işaretidir.

Örnek Star Şema

```SQL
-- FACT: grain = "bir siparisteki bir urun kalemi"
CREATE TABLE fact_satis (
    tarih_key     INT     NOT NULL,   -- FK -> dim_tarih
    musteri_key   INT     NOT NULL,   -- FK -> dim_musteri (surrogate key!)
    urun_key      INT     NOT NULL,   -- FK -> dim_urun
    magaza_key    INT     NOT NULL,   -- FK -> dim_magaza
    siparis_no    VARCHAR(50),        -- degenerate dimension
    adet          INT           NOT NULL,
    birim_fiyat   DECIMAL(12,2) NOT NULL,
    satir_tutari  DECIMAL(12,2) NOT NULL
);

-- DIMENSION: denormalize, genis, betimleyici
CREATE TABLE dim_urun (
    urun_key        INT PRIMARY KEY,   -- surrogate key
    urun_kodu       VARCHAR(50),       -- natural/business key
```

```
    urun_adi        VARCHAR(200),
    kategori_adi    VARCHAR(100),      -- Star: ayri tabloya bolunmez
    alt_kategori    VARCHAR(100),
    marka_adi       VARCHAR(100),
    tedarikci_adi   VARCHAR(200)
);
```

Ölçü (Measure) Türleri

- **Additive (toplanabilir):** Tüm boyutlar boyunca toplanabilir. Satış tutarı gibi. En kullanışlı ölçü türüdür.
- **Semi-additive (yarı toplanabilir):** Bazı boyutlar boyunca toplanabilir ama zaman boyunca toplanamaz. Stok miktarı veya hesap bakiyesi gibi; üç günün bakiyesini toplamak anlamsızdır, ortalama veya son değer alınır.
- **Non-additive (toplanamaz):** Hiçbir boyut boyunca toplanamaz. Oran ve yüzdeler gibi; bunlar toplanmaz, pay ve payda ayrı ayrı toplanıp oran yeniden hesaplanır.

Fact Tablosu Türleri

- **Transaction Fact:** Her iş olayı için bir satır. En yaygın ve en detaylı türdür.
- **Periodic Snapshot Fact:** Düzenli aralıklarla alınan anlık görüntü (her gün sonu stok, her ay sonu bakiye).
- **Accumulating Snapshot Fact:** Bir sürecin aşamalarını tek satırda takip eder ve süreç ilerledikçe aynı satır güncellenir (sipariş verildi, hazırlandı, kargolandı, teslim edildi tarihleri).

Karşılaştırma Tablosu

Kriter	Star Şema	Snowflake Şema
Boyut yapısı	Denormalize, tek tablo	Normalize, alt tablolara bölünmüş
JOIN sayısı	Az (tek seviye)	Fazla (çok seviye)
Sorgu performansı	Yüksek	Daha düşük
Depolama	Daha fazla (tekrar var)	Daha az
Anlaşılabilirlik	İş kullanıcısı için kolay	Daha karmaşık
Tercih	Analitik ambarlarda varsayılan	Boyut çok büyük/çok değişkense

Slowly Changing Dimensions (SCD)

> **SORU:** *Slowly Changing Dimension nedir? Type 1, Type 2 ve Type 3 arasındaki farkı açıkla. Hangisini ne zaman kullanırsın?*

1. Mülakat Cevabı

> *"Slowly Changing Dimension, boyut tablolarındaki özniteliklerin zaman içinde yavaşça değişmesi durumunu ve bu değişimin nasıl yönetileceğini tanımlar. Örneğin bir müşterinin adresi veya segmenti değiştiğinde, geçmiş satışların hangi adrese ait olduğunu koruyup korumayacağımıza karar vermemiz gerekir.*
>
> *Type 1'de eski değer basitçe üzerine yazılır. Tarihçe tutulmaz. Yalnızca hata düzeltmelerinde veya geçmişi bilmenin iş açısından hiçbir değeri olmadığı durumlarda kullanılır. En basit yöntemdir ama geriye dönük raporlar değişir.*
>
> *Type 2'de değişiklik olduğunda mevcut satır kapatılır ve yeni bir satır eklenir. Her satırda geçerlilik başlangıç ve bitiş tarihleri ile aktif kaydı gösteren bir bayrak bulunur. Bu, tarihçeyi tam olarak koruyan ve endüstride en yaygın kullanılan yöntemdir. Type 2'nin çalışabilmesi için boyut tablosunda mutlaka surrogate key kullanılmalıdır; çünkü aynı iş anahtarına sahip birden fazla satır olacaktır.*
>
> *Type 3'te ise tabloya 'önceki değer' için ek bir sütun eklenir. Yalnızca bir adım geriyi saklar. Örneğin bir yeniden yapılanmada eski ve yeni bölge adını yan yana raporlamak gerektiğinde kullanılır; sınırlı bir kullanım alanı vardır.*
>
> *Varsayılan tercihim Type 2'dir, çünkü veri ambarının temel vaadi geçmişi doğru şekilde yeniden üretebilmektir."*

2. Detaylı Arka Plan

Type 2 Tablo Yapısı

```SQL
CREATE TABLE dim_musteri (
    musteri_key      INT PRIMARY KEY,   -- SURROGATE KEY (her versiyon icin farkli)
    musteri_id       INT NOT NULL,      -- BUSINESS KEY (kaynak sistemdeki id)
    ad_soyad         VARCHAR(200),
    sehir            VARCHAR(100),
    segment          VARCHAR(50),
    gecerli_baslangic DATE NOT NULL,
    gecerli_bitis     DATE,             -- aktif kayitta NULL veya '9999-12-31'
    aktif_mi          BOOLEAN NOT NULL
);

-- Ornek veri: musteri 100'un segmenti degisti
-- key | id  | segment  | baslangic  | bitis      | aktif
--  1  | 100 | Standart | 2023-01-01 | 2024-05-31 | false
--  2  | 100 | Premium  | 2024-06-01 | NULL       | true
```

Kritik nokta: fact_satis tablosu musteri_id'ye değil musteri_key'e (surrogate key) bağlanır. Böylece 2023'te yapılan bir satış, o tarihte müşterinin "Standart" segmentte olduğu satıra işaret eder ve geçmiş rapor doğru kalır. Buna "as-of" raporlama denir ve Type 2'nin asıl amacı budur.

Type 2 Yükleme (MERGE ile)

```SQL
-- 1) Degisen kayitlarin mevcut versiyonunu kapat
UPDATE dim_musteri d
SET gecerli_bitis = CURRENT_DATE - 1,
    aktif_mi      = FALSE
FROM staging_musteri s
```

```
WHERE d.musteri_id = s.musteri_id
  AND d.aktif_mi   = TRUE
  AND (d.segment IS DISTINCT FROM s.segment      -- NULL-guvenli karsilastirma
    OR d.sehir   IS DISTINCT FROM s.sehir);

-- 2) Yeni versiyonlari ekle (degisenler + hic olmayanlar)
INSERT INTO dim_musteri
    (musteri_id, ad_soyad, sehir, segment, gecerli_baslangic, gecerli_bitis, aktif_mi)
SELECT s.musteri_id, s.ad_soyad, s.sehir, s.segment, CURRENT_DATE, NULL, TRUE
FROM staging_musteri s
LEFT JOIN dim_musteri d
       ON d.musteri_id = s.musteri_id AND d.aktif_mi = TRUE
WHERE d.musteri_id IS NULL;
```

Not: IS DISTINCT FROM kullanmak önemlidir. Normal <> operatörü NULL değerlerde UNKNOWN döndürdüğü için, NULL'dan bir değere geçen değişiklikler sessizce atlanır. Bu, SCD2 implementasyonlarındaki en yaygın hatadır.

Karşılaştırma Tablosu

Tür	Davranış	Tarihçe	Kullanım
Type 0	Değer hiç değiştirilmez	Orijinal korunur	Doğum tarihi gibi sabit öznitelikler
Type 1	Üzerine yazar	Yok	Hata düzeltme, geçmişin önemsiz olduğu alanlar
Type 2	Yeni satır ekler, eskisini kapatır	Tam	Endüstri standardı; segment, adres, statü
Type 3	Önceki değer için ek sütun	Bir adım	Yeniden yapılanmada eski/yeni karşılaştırma

Mülakat Tuzağı: "SCD2 tabloyu ne kadar büyütür?"

Olgun cevap: Değişim sıklığına bağlıdır. Nadiren değişen bir öznitelik için SCD2 ideal iken, günde birkaç kez değişen bir alan (örneğin bir sayaç veya skor) boyut tablosunu patlatır. Böyle bir alan aslında boyut değil, ölçüdür ve fact tablosuna ya da ayrı bir mini-dimension tablosuna taşınmalıdır. Kimball bu tekniğe "mini-dimension" veya hızlı değişen öznitelikler için "outrigger" der.

OLTP ve OLAP Sistemleri

> **SORU:** *OLTP ve OLAP sistemlerinin veri modelleme yaklaşımı neden birbirinden farklıdır? Kolon bazlı depolama neden analitikte hızlıdır?*

1. Mülakat Cevabı

> *"İkisi tamamen farklı iş yükleri için optimize edilmiştir.*
>
> *OLTP (Online Transaction Processing) sistemleri, günlük iş operasyonlarını yürütür: sipariş oluşturma, para transferi, stok güncelleme. İş yükü çok sayıda küçük ve eşzamanlı okuma-yazma işleminden oluşur; genellikle tek bir kayda erişilir. Bu yüzden 3NF'e kadar normalize edilir, satır bazlı (row-oriented) depolama kullanır ve anahtar üzerinde index'lerle desteklenir.*
>
> *OLAP (Online Analytical Processing) sistemleri ise analiz ve raporlama içindir: 'geçen çeyrekte bölge bazında ciro nasıl değişti' gibi sorular. İş yükü az sayıda ama çok büyük veri hacmini tarayan sorgulardan oluşur ve genellikle tüm satırların yalnızca birkaç sütununa dokunulur. Bu yüzden star şema ile denormalize edilir ve kolon bazlı (columnar) depolama kullanılır.*
>
> *Kolon bazlı depolamanın hızlı olmasının iki sebebi var. Birincisi, sorgu yalnızca ihtiyaç duyduğu sütunları diskten okur; 200 sütunlu bir tabloda 3 sütun sorguluyorsan diğer 197 sütunu hiç okumazsın. İkincisi, aynı sütundaki değerler aynı tipte ve birbirine benzer olduğu için sıkıştırma oranları çok yüksektir; bu da okunan bayt miktarını dramatik şekilde azaltır."*

2. Detaylı Arka Plan

Satır Bazlı ve Kolon Bazlı Depolama

Aynı veri diske iki farklı şekilde yerleştirilebilir. Satır bazlı depolamada bir satırın tüm sütunları yan yana durur; bu, tek bir kaydın tamamını okumak veya yazmak için idealdir (OLTP). Kolon bazlı depolamada ise bir sütunun tüm değerleri yan yana durur; bu, bir sütunu baştan sona toplamak için idealdir (OLAP).

```
Depolama Düzeni
Satir bazli (row-store):
[1, Ahmet, Ankara, 5000][2, Mehmet, Izmir, 7000][3, Ayse, Bursa, 6000]
-> "Ahmet'in tum bilgilerini getir" cok hizli
-> "Tum maaslarin toplamini al" -> tum satirlari okumak gerekir

Kolon bazli (column-store):
id:     [1, 2, 3]
ad:     [Ahmet, Mehmet, Ayse]
sehir:  [Ankara, Izmir, Bursa]
maas:   [5000, 7000, 6000]
-> "Tum maaslarin toplamini al" -> yalnizca maas blogu okunur
-> Ayni tipteki degerler yan yana oldugu icin cok iyi sikisir
```

Karşılaştırma Tablosu

Kriter	OLTP	OLAP
Amaç	Günlük işlem yürütme	Analiz ve karar destek
Tipik sorgu	Tek kayıt oku/yaz	Milyonlarca satırı tara ve topla
Modelleme	Normalize (3NF)	Denormalize (star şema)
Depolama	Satır bazlı	Kolon bazlı
Eşzamanlılık	Çok yüksek (binlerce kullanıcı)	Düşük (onlarca analist)
Veri güncelliği	Anlık	Genelde gecikmeli (batch)
Örnek sistem	PostgreSQL, MySQL, Oracle	Snowflake, BigQuery, Redshift, ClickHouse

Mülakat Sorusu: "Raporları neden doğrudan üretim veritabanından çekmiyoruz?"

Beklenen cevap birkaç katmanlıdır: Ağır analitik sorgular OLTP sisteminde kilit ve kaynak baskısı yaratıp müşteriye dokunan işlemleri yavaşlatır. OLTP'nin normalize modeli, analitik sorgular için çok sayıda JOIN gerektirir ve verimsizdir. Ayrıca veri ambarı geçmişi saklar; OLTP sistemi ise genellikle yalnızca güncel durumu tutar, kayıt güncellenince eski değer kaybolur. Son olarak veri ambarı farklı kaynak sistemleri tek bir tutarlı modelde birleştirir ki OLTP tek başına bunu sağlayamaz.

Veri Mühendisliği: Büyük Veri İşleme

Veri Mühendisliği İçin SQL: Deduplication ve Anti-Join

> **SORU:** *Bir tabloda tekrar eden kayıtlardan yalnızca en güncelini nasıl tutarsın? Anti-join nedir ve hangi yöntemi tercih edersin?*

1. Mülakat Cevabı

> *"Veri mühendisliği mülakatlarında SQL, temel JOIN'lerin ötesinde iki kalıp üzerinden test edilir: tekilleştirme ve fark bulma.*
>
> *Tekilleştirme için window fonksiyonu kullanırım. İş anahtarına göre PARTITION BY yapar, güncellik sütununa göre azalan sıralar ve ROW_NUMBER ile numaralandırırım; ardından yalnızca birinci sırayı seçerim. Burada kritik detay, sıralamaya belirleyici bir ikinci sütun (tie-breaker) eklemektir; aksi halde iki kayıt aynı zaman damgasına sahipse hangisinin seçileceği belirsiz kalır ve hattın sonucu her çalıştırmada değişebilir, bu da idempotency'yi bozar.*
>
> *Anti-join ise 'A tablosunda olup B tablosunda olmayan kayıtlar' sorusunun cevabıdır. Veri kalitesi kontrollerinde, eksik yükleme tespitinde ve churn analizinde sürekli kullanılır.*
>
> *Üç yazım yolu vardır: NOT EXISTS, LEFT JOIN ile NULL kontrolü ve NOT IN. Tercihim NOT EXISTS'tir çünkü hem NULL değerlere karşı güvenlidir hem de niyeti en net ifade eder. NOT IN'den özellikle kaçınırım; alt sorgudan dönen listede tek bir NULL bile varsa sorgu sessizce hiçbir satır döndürmez ve bu hatanın fark edilmesi çok zordur."*

2. Detaylı Arka Plan

Deduplication Kalıbı

```SQL
-- Her musteri_id icin yalnizca en guncel kaydi tut
WITH sirali AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY musteri_id
            ORDER BY guncelleme_tarihi DESC,
                     kaynak_dosya_sira DESC   -- tie-breaker: deterministik sonuc
        ) AS rn
    FROM bronze_musteri
)
SELECT * EXCEPT (rn)      -- BigQuery/Databricks sozdizimi
FROM sirali
WHERE rn = 1;
```

Tam olarak birebir aynı (tüm sütunları aynı) satırları elemek istiyorsan çok daha ucuz bir yol vardır: SELECT DISTINCT veya GROUP BY. ROW_NUMBER'a yalnızca "aynı anahtar, farklı içerik" durumunda ve hangisinin kazanacağına karar vermen gerektiğinde ihtiyaç duyarsın. Mülakatta bu ayrımı yapmak dikkat çeker.

Anti-Join'in Üç Yazımı

```SQL
-- 1) NOT EXISTS  (tercih edilen: NULL-guvenli, okunabilir)
SELECT k.*
FROM kaynak k
WHERE NOT EXISTS (
    SELECT 1 FROM hedef h WHERE h.id = k.id
);
```

```sql
-- 2) LEFT JOIN + IS NULL  (yaygin, planlari genelde ayni)
SELECT k.*
FROM kaynak k
LEFT JOIN hedef h ON h.id = k.id
WHERE h.id IS NULL;

-- 3) NOT IN  (TEHLIKELI: liste NULL iceriyorsa hic satir donmez)
SELECT k.*
FROM kaynak k
WHERE k.id NOT IN (SELECT id FROM hedef);   -- kacinilmali
```

Veri Kalitesi Kontrolleri İçin SQL Kalıpları

```sql
-- Birincil anahtar gercekten benzersiz mi? (bos donmeli)
SELECT musteri_id, COUNT(*) AS adet
FROM silver_musteri
GROUP BY musteri_id
HAVING COUNT(*) > 1;

-- Yetim kayit var mi? Fact'te olup boyutta olmayan anahtarlar
SELECT DISTINCT f.urun_key
FROM fact_satis f
WHERE NOT EXISTS (SELECT 1 FROM dim_urun d WHERE d.urun_key = f.urun_key);

-- Gunluk satir sayisi anormal mi? (hacim/freshness kontrolu)
SELECT
    yukleme_tarihi,
    COUNT(*) AS satir,
    AVG(COUNT(*)) OVER (ORDER BY yukleme_tarihi
                        ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING) AS onceki_7gun_ort
FROM fact_satis
GROUP BY yukleme_tarihi
ORDER BY yukleme_tarihi DESC;
```

Apache Spark Mimarisi

> **SORU:** *Spark'ın mimarisini anlat. Driver, Executor ve Cluster Manager ne yapar? Transformation ile action arasındaki fark nedir?*

1. Mülakat Cevabı

> *"Spark, dağıtık bir işleme motorudur ve üç ana bileşenden oluşur.*
>
> *Driver, uygulamanın beynidir. Yazdığımız kodu çalıştırır, mantıksal planı ve DAG'ı oluşturur, işi aşamalara (stage) ve görevlere (task) böler, bunları executor'lara dağıtır ve sonuçları toplar. SparkSession burada yaşar.*
>
> *Executor'lar, worker düğümlerinde çalışan süreçlerdir. Kendilerine verilen görevleri gerçekten çalıştırır ve önbelleğe alınan (cache) veriyi bellekte tutarlar. Her executor birden fazla görevi paralel çalıştırabilir; bunu belirleyen çekirdek sayısıdır.*
>
> *Cluster Manager ise kaynak tahsisinden sorumludur: YARN, Kubernetes veya Spark'ın kendi standalone yöneticisi olabilir. Driver'ın talebi üzerine executor'ları başlatır.*
>
> *Spark'ta işlemler tembeldir (lazy). Transformation'lar (filter, select, join, groupBy) hemen çalışmaz; yalnızca hesaplama planına eklenir. Ancak bir action (count, collect, write, show) çağrıldığında Spark tüm planı optimize eder ve işi başlatır. Bu tembellik, Catalyst optimizer'ın gereksiz adımları elemesine ve filtreleri kaynağa yakın itmesine (predicate pushdown) imkan verir."*

2. Detaylı Arka Plan

Job, Stage ve Task Hiyerarşisi

- **Job:** Bir action çağrısıyla tetiklenen tüm iş. Kaç action varsa o kadar job oluşur.
- **Stage:** Job'ın shuffle sınırlarıyla bölünmüş parçası. Shuffle gerektirmeyen ardışık işlemler tek bir stage'de birleştirilir (pipelining).
- **Task:** Bir stage'in tek bir veri partition'ı üzerinde çalışan en küçük iş birimi. Partition sayısı kadar task oluşur ve bunlar executor çekirdeklerinde paralel çalışır.

Narrow ve Wide Transformation

Bu ayrım, Spark performansının temelidir. Narrow transformation'da (map, filter, select) bir çıktı partition'ı yalnızca tek bir girdi partition'ına bağlıdır; veri düğümler arasında hareket etmez ve işlem çok hızlıdır.

Wide transformation'da (groupBy, join, distinct, repartition) ise bir çıktı partition'ı birden fazla girdi partition'ından veri toplamak zorundadır. Bu, verinin ağ üzerinden yeniden dağıtılmasını yani shuffle'ı gerektirir ve yeni bir stage başlatır.

Mülakat Tuzağı: collect() Kullanımı

Mülakatçı "Sonuçları görmek için ne yaparsın?" diye sorabilir. Eğer cevabın collect() ise takip sorusu gelir: collect(), tüm dağıtık veriyi tek bir makineye, yani driver'a çeker. Büyük bir DataFrame'de bu, driver'ın belleğini doldurup uygulamayı çökertir (OutOfMemoryError). Doğru yaklaşım, veriyi dağıtık şekilde hedefe yazmak (write) veya yalnızca küçük bir örneği incelemek için show() ya da take(n) kullanmaktır.

cache() ve persist() Ne Zaman Kullanılır?

Bir DataFrame birden fazla action tarafından tekrar tekrar kullanılıyorsa, Spark onu her seferinde baştan hesaplar çünkü tembel değerlendirme sonucu saklamaz. Böyle durumlarda cache() ile bellekte tutmak ciddi kazanç sağlar. Ancak gereksiz cache bellek baskısı yaratıp diğer işleri yavaşlatır; kullanmadıktan sonra unpersist() ile serbest bırakmak iyi bir alışkanlıktır.

Shuffle, Partitioning ve Data Skew

> **SORU:** *Shuffle nedir ve neden pahalıdır? Nasıl azaltırsın? Data skew problemini nasıl çözersin?*

1. Mülakat Cevabı

> *"Shuffle, verinin küme içindeki düğümler arasında anahtar bazında yeniden dağıtılmasıdır. groupBy, join, distinct ve repartition gibi wide transformation'lar bunu tetikler.*
>
> *Pahalıdır çünkü üç maliyeti aynı anda getirir: veri serileştirilip diske yazılır, ağ üzerinden aktarılır ve karşı tarafta tekrar okunup deserileştirilir. Spark'ta bir işi yavaşlatan darboğazın büyük çoğunluğu shuffle kaynaklıdır.*
>
> *Azaltmak için kullandığım yöntemler şunlar: Filtreleri mümkün olan en erken aşamada uygularım, böylece daha az veri taşınır. Küçük bir tabloyla büyük bir tabloyu birleştiriyorsam broadcast join kullanırım; küçük tablo tüm executor'lara kopyalanır ve shuffle tamamen ortadan kalkar. Aynı anahtar üzerinden defalarca join yapılacaksa tabloları o anahtara göre önceden bölümlerim (bucketing). Ayrıca shuffle partition sayısını veri hacmine göre ayarlarım; varsayılan 200 değeri küçük işlerde gereksiz yere çok sayıda minik task üretir.*
>
> *Data skew ise bir anahtarın diğerlerinden orantısız derecede fazla kayda sahip olmasıdır; örneğin müşteri kimliği NULL olan milyonlarca kayıt tek bir partition'a düşer. Bu durumda 199 task saniyeler içinde biter, bir task saatlerce sürer. Çözüm olarak salting uygularım: anahtara rastgele bir ek değer ekleyerek yükü yapay olarak birden fazla partition'a dağıtırım. Spark 3 ile gelen Adaptive Query Execution ise skew join'i çalışma anında tespit edip otomatik bölebiliyor; onu açık tutmak ilk adımdır."*

2. Detaylı Arka Plan

Broadcast Join

```python
from pyspark.sql.functions import broadcast

# Kucuk boyut tablosu tum executor'lara kopyalanir -> shuffle YOK
sonuc = (
    buyuk_fact
    .join(broadcast(kucuk_dim), on="urun_key", how="left")
)

# Otomatik broadcast esigi (varsayilan 10MB) ayarlanabilir:
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 50 * 1024 * 1024)
```

Salting ile Skew Çözümü

```python
from pyspark.sql import functions as F

N = 10  # tuz araligi

# Buyuk (skew olan) tarafa rastgele tuz ekle
buyuk_salted = buyuk.withColumn(
    "salt", (F.rand() * N).cast("int")
).withColumn("join_key", F.concat_ws("_", F.col("musteri_id"), F.col("salt")))

# Kucuk tarafi N kez cogalt ki her tuzla eslesebilsin
kucuk_exploded = (
    kucuk
    .withColumn("salt", F.explode(F.array([F.lit(i) for i in range(N)])))
    .withColumn("join_key", F.concat_ws("_", F.col("musteri_id"), F.col("salt")))
)
```

```
sonuc = buyuk_salted.join(kucuk_exploded, on="join_key", how="inner")
```

repartition ve coalesce Farkı

- **repartition(n):** Tam shuffle yapar, partition sayısını artırabilir veya azaltabilir ve veriyi dengeli dağıtır. Pahalıdır ama sonuç dengelidir.
- **coalesce(n):** Yalnızca azaltır ve tam shuffle yapmaz; mevcut partition'ları birleştirir. Ucuzdur ama dengesiz partition'lar üretebilir. Yazma öncesi dosya sayısını azaltmak için idealdir.

Küçük Dosya Problemi (Small Files Problem)

Streaming veya sık çalışan batch işler, her çalıştırmada çok sayıda küçük dosya üretir. Binlerce 1 MB'lık dosya, nesne depolamada listeleme ve açma maliyeti yüzünden okuma performansını felç eder. Çözümler: yazmadan önce coalesce ile dosya sayısını azaltmak, periyodik olarak sıkıştırma (compaction) işi çalıştırmak veya Delta Lake'te OPTIMIZE komutunu kullanmak. Hedef dosya boyutu genellikle 128 MB ile 1 GB arasında tutulur.

Partitioning Stratejisi

Disk üzerinde partitioning yaparken (örneğin tarihe göre klasörleme) kritik kural, çok yüksek kardinaliteli bir sütun seçmemektir. Örneğin musteri_id'ye göre partition yapmak milyonlarca minik klasör üretir ve felaket olur. İyi partition anahtarı, sorgularda sürekli filtrelenen ve makul sayıda değer üreten bir sütundur; tarih bunun klasik örneğidir.

Dosya Formatları: Parquet, Avro ve CSV

> **SORU:** *Analitik iş yüklerinde neden CSV yerine Parquet kullanırız? Parquet ile Avro arasındaki fark nedir?*

1. Mülakat Cevabı

> *"Fark, verinin dosya içinde nasıl düzenlendiğidir.*
>
> *Parquet kolon bazlı (columnar) bir formattır. Aynı sütunun değerleri dosya içinde yan yana saklanır. Bu iki büyük avantaj getirir: Birincisi kolon budama (column pruning); 200 sütunlu bir tablodan 3 sütun sorguladığında motor yalnızca o üç sütunun bloklarını okur. İkincisi sıkıştırma verimliliği; aynı tipteki ve birbirine benzeyen değerler yan yana olduğu için sıkıştırma oranları çok yüksektir. Ayrıca Parquet her blok için min-max gibi istatistikler tutar; bu sayede motor filtreye uymayan blokları hiç okumadan atlar (predicate pushdown).*
>
> *Avro ise satır bazlı bir formattır ve şemayı dosyanın içinde taşır. Bir kaydın tamamını yazmak veya okumak gerektiğinde verimlidir ve şema evrimini (schema evolution) çok iyi destekler. Bu yüzden veri aktarımı, olay akışları ve Kafka mesajlarında tercih edilir.*
>
> *CSV ise ne şema ne tip bilgisi taşır, sıkıştırma sunmaz ve iç içe (nested) yapıları temsil edemez. Yalnızca sistemler arası basit veri alışverişi ve insan tarafından okunabilirlik gerektiğinde kullanılır.*
>
> *Pratik kuralım şu: Veri ambarında ve analitik katmanda Parquet, veri taşıma ve streaming katmanında Avro kullanırım; CSV'yi ise yalnızca kaynak sistemden veri alırken kabul eder, Bronze katmanına indirir indirmez Parquet'e dönüştürürüm."*

2. Detaylı Arka Plan

Somut Fark: Aynı Sorgu, Farklı Maliyet

100 sütunlu, 1 TB'lık bir olay tablosunda "geçen ayın toplam cirosunu" hesapladığını düşün.

- CSV ile: Motor 1 TB'ın tamamını okumak zorundadır. Sütunları ayırmak için her satırı ayrıştırması, tipleri dönüştürmesi gerekir. Filtre ancak okuma sonrasında uygulanabilir.
- Parquet ile: Motor yalnızca tarih ve tutar sütunlarının bloklarını okur (belki 15 GB). Blok istatistikleri sayesinde geçen aya ait olmayan blokları hiç açmaz (belki 1 GB'a iner). Sonuç, onlarca kat daha az I/O ve dramatik hız farkıdır.

Şema Evrimi (Schema Evolution)

Kaynak sisteme yeni bir sütun eklendiğinde ne olacağı, veri mühendisliğinde sürekli karşılaşılan bir sorudur. Avro, yazma ve okuma şemalarını ayrı tuttuğu ve varsayılan değer tanımlamaya izin verdiği için bu konuda en esnek formattır. Parquet de sütun ekleme gibi uyumlu değişiklikleri destekler; Delta Lake veya Iceberg kullanılıyorsa şema evrimi tablo seviyesinde kontrollü şekilde yönetilir (mergeSchema).

Karşılaştırma Tablosu

Kriter	Parquet	Avro	CSV
Düzen	Kolon bazlı	Satır bazlı	Satır bazlı, düz metin
Şema	Dosyada gömülü	Dosyada gömülü (JSON)	Yok
Sıkıştırma	Çok iyi	İyi	Zayıf (harici gerekir)
Kolon budama	Var	Yok	Yok
Predicate pushdown	Var (min/max istatistik)	Sınırlı	Yok
İç içe yapı	Destekler	Destekler	Desteklemez

Kriter	Parquet	Avro	CSV
En uygun kullanım	Analitik okuma (OLAP)	Streaming, veri aktarımı	Basit alışveriş, insan okuması

Orkestrasyon ve Modern Veri Yığını

DAG Kavramı ve Apache Airflow

> **SORU:** *DAG nedir? Airflow'un temel bileşenlerini (Operator, Sensor, Scheduler, Executor) anlat. Airflow'da en sık yapılan hata nedir?*

1. Mülakat Cevabı

> *"DAG, Directed Acyclic Graph yani yönlü ve döngüsüz çizge demektir. Veri görevlerini ve aralarındaki bağımlılıkları tanımlar: A tamamlanmadan B ve C başlayamaz. Döngüsüz olması zorunludur, aksi halde iş sonsuza kadar birbirini bekler.*
>
> *Airflow bir orkestratördür; verinin kendisini işlemez, veriyi işleyen sistemleri doğru sırada ve doğru zamanda tetikler. Bir Spark job'ını başlatır, bir Snowflake sorgusunu çalıştırır, bir dbt komutunu koşturur.*
>
> *Temel bileşenleri şunlardır: Operator, tek bir görevi çalıştıran şablondur (PythonOperator, BashOperator veya SnowflakeOperator gibi). Sensor, özel bir operator türüdür ve belirli bir koşul gerçekleşene kadar bekler; örneğin S3'e bir dosyanın düşmesini bekleyen S3KeySensor. Hook ise dış sistemlere bağlantıyı soyutlar. Scheduler, DAG tanımlarını okuyup zamanı gelen görevleri sıraya alır; Executor (Local, Celery veya Kubernetes) bu görevleri fiilen çalıştırır; Metadata Database ise tüm durum bilgisini tutar.*
>
> *En sık yapılan hata, Airflow'u bir işleme motoru gibi kullanmaktır. Milyonlarca satırı bir PythonOperator içinde pandas ile işlemeye çalışmak worker'ı çökertir. Airflow'un görevi işi yapmak değil, işi yaptırmaktır."*

2. Detaylı Arka Plan

Basit Bir DAG Örneği

```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from datetime import datetime

with DAG(
    dag_id="gunluk_satis_hatti",
    start_date=datetime(2024, 1, 1),
    schedule="0 2 * * *",        # her gun 02:00
    catchup=False,
    max_active_runs=1,
    default_args={"retries": 3},
) as dag:

    dosya_bekle = S3KeySensor(
        task_id="kaynak_dosya_bekle",
        bucket_key="s3://ham-veri/satis/{{ ds }}/veri.parquet",
        poke_interval=300,
        timeout=60 * 60,
        mode="reschedule",       # bekleme sirasinda worker slotunu birak
    )

    donustur = PythonOperator(
        task_id="spark_isini_tetikle",
        python_callable=spark_job_gonder,
        op_kwargs={"islem_tarihi": "{{ ds }}"},   # mantiksal tarih parametresi
    )

    dosya_bekle >> donustur      # bagimlilik: once bekle, sonra donustur
```

Mülakat Tuzağı 1: Sensor'ün poke ve reschedule Modu

Varsayılan poke modunda sensor, bekleme boyunca bir worker slotunu işgal eder. Uzun bekleyen çok sayıda sensor, tüm worker havuzunu kilitleyip başka hiçbir görevin çalışamamasına yol açar (sensor deadlock). Çözüm mode="reschedule" kullanmaktır; sensor kontrol arasında slotu serbest bırakır. Bunu bilmek operasyonel deneyim işaretidir.

Mülakat Tuzağı 2: DAG Dosyasındaki Top-Level Kod

Scheduler, DAG dosyalarını sürekli (varsayılan olarak birkaç saniyede bir) yeniden ayrıştırır. DAG dosyasının en üst seviyesine ağır bir işlem koyarsan (veritabanı sorgusu, API çağrısı, büyük dosya okuma) bu kod her ayrıştırmada çalışır ve scheduler'ı felce uğratır. Ağır işler mutlaka task fonksiyonlarının içine konulmalıdır.

```python
# YANLIS: her DAG parse'inda calisir, scheduler'i yavaslatir
df = pd.read_sql("SELECT * FROM buyuk_tablo", conn)   # top-level!

# DOGRU: yalnizca task calistiginda calisir
def isle():
    df = pd.read_sql("SELECT * FROM buyuk_tablo", conn)
    ...
PythonOperator(task_id="isle", python_callable=isle)
```

Mantıksal Tarih, Catchup ve Backfill

Airflow'da her çalıştırmanın bir mantıksal tarihi vardır ve şablon değişkeni olarak {{ ds }} ile erişilir. Görevler bu tarihe göre yazıldığında geçmişi yeniden çalıştırmak (backfill) güvenli hale gelir. catchup=True olarak ayarlanırsa, start_date'ten bugüne kadar çalışmamış tüm aralıklar otomatik olarak çalıştırılır; bu bilinçli yapılmazsa aniden yüzlerce eşzamanlı çalıştırma başlayabilir. Bu yüzden çoğu üretim DAG'ında catchup=False ve max_active_runs=1 tercih edilir.

Idempotency ile Bağlantı

Airflow, bir görev başarısız olduğunda otomatik olarak yeniden dener (retries). Bu, görevlerin idempotent olmasını zorunlu kılar. Görev her denemede hedefe koşulsuz INSERT yapıyorsa, üç deneme sonunda veri üçe katlanır. Bu yüzden Airflow ile idempotency birbirinden ayrı düşünülemez.

SSIS ve Apache Airflow Karşılaştırması

> **SORU:** *SSIS gibi geleneksel bir ETL aracıyla Airflow arasındaki temel fark nedir? SSIS'ten Airflow'a geçişte nelere dikkat edersin?*

1. Mülakat Cevabı

> *"En temel fark, ikisinin farklı kategoride araçlar olmasıdır. SSIS bir ETL aracıdır; veriyi kendisi taşır ve kendi motorunda dönüştürür. Airflow ise bir orkestratördür; veriyi kendisi işlemez, işleme yapan sistemleri koordine eder.*
>
> *SSIS, Microsoft'un geleneksel çözümüdür ve GUI odaklıdır: kutuları sürükleyip bırakarak veri akışı tasarlarsınız. SQL Server ekosistemiyle sıkı entegredir ve şirket içi (on-premise) Windows ortamlarında güçlüdür. Ancak paketleri karmaşık XML dosyaları olarak sakladığı için sürüm kontrolü ve kod incelemesi (code review) pratikte çok zordur; birim test yazmak neredeyse imkansızdır ve bulut ortamında dinamik ölçeklenmesi kısıtlıdır.*
>
> *Airflow ise 'yapılandırma kod olarak' (configuration as code) prensibiyle çalışır; her şey Python'dur. Bu, veri hatlarına yazılım mühendisliği disiplinini getirir: git ile sürüm kontrolü, pull request ile kod incelemesi, birim testi ve CI/CD. Ayrıca DAG'ları programatik olarak üretebilirsiniz; yüz tabloyu yükleyen yüz görevi bir döngüyle oluşturmak mümkündür.*
>
> *Migrasyonda en kritik nokta şudur: SSIS'in data flow bileşenlerinin doğrudan bir Airflow karşılığı yoktur. O dönüşüm mantığını yeniden bir yere yazmak gerekir; genellikle hedef ambarda SQL veya dbt modeli olarak ya da Spark job'ı olarak. Yani bu bir araç değişimi değil, ETL'den ELT'ye mimari geçiştir."*

2. Detaylı Arka Plan

Karşılaştırma Tablosu

Kriter	SSIS	Apache Airflow
Kategori	ETL aracı (veriyi kendisi işler)	Orkestratör (işi başkasına yaptırır)
Tanımlama biçimi	GUI, sürükle-bırak (.dtsx XML)	Python kodu (configuration as code)
Sürüm kontrolü	Zor; XML diff'i okunamaz	Doğal; git ile tam uyumlu
Test edilebilirlik	Çok kısıtlı	Birim testi yazılabilir
Dinamiklik	Statik paketler	DAG'lar programatik üretilebilir
Ekosistem	SQL Server / Windows merkezli	Bulut ve teknoloji bağımsız
Ölçeklenme	Sunucuya bağlı, dikey	Executor ile yatay (Celery/K8s)
Öğrenme eğrisi	Düşük (görsel)	Orta (Python bilgisi gerekir)

Migrasyon Kontrol Listesi

- Dönüşüm mantığının yeni evini belirle: SSIS data flow içindeki lookup, derived column ve conditional split mantığı SQL/dbt modeline mi yoksa Spark'a mı taşınacak?
- Bağlantı ve gizli bilgi yönetimini planla: SSIS'teki bağlantı yöneticileri, Airflow Connections ve bir secret backend (Vault, AWS Secrets Manager) ile değiştirilmeli.
- Zamanlama semantiğini eşle: SQL Server Agent job'larının tetikleme mantığı ile Airflow'un mantıksal tarih/aralık modeli aynı değildir; özellikle ay sonu ve geriye dönük çalıştırmalarda davranış farkına dikkat et.
- Hata yönetimini yeniden tasarla: SSIS'in checkpoint ve event handler yapıları yerine Airflow'un retry, SLA ve callback mekanizmaları kullanılmalı; görevler idempotent hale getirilmeli.

- Aşamalı geçiş yap: Tüm paketleri bir anda taşımak yerine, önce düşük riskli bir hattı taşıyıp iki sistemi paralel çalıştır ve çıktıları karşılaştır (mutabakat).

Mülakat Notu: SSIS Kötü Bir Araç mı?

Dengeli cevap vermek önemlidir. SSIS, kendi bağlamında hâlâ etkilidir: şirket içi SQL Server ortamında, görsel geliştirme hızının önemli olduğu ve ekipte Python yetkinliği bulunmayan durumlarda makul bir seçimdir. Airflow'un avantajı mutlak üstünlük değil, modern bulut mimarilerine ve yazılım mühendisliği pratiklerine uygunluktur. Mülakatta eski teknolojiyi küçümsemek yerine bağlama göre değerlendirme yapmak olgunluk gösterir.

Modern Veri Yığını ve dbt

> **SORU:** *Modern Data Stack nedir? dbt ne işe yarar ve neden bu kadar yaygınlaştı?*

1. Mülakat Cevabı

> *"Modern Data Stack, bulut veri ambarının merkezde olduğu, her katmanı özelleşmiş araçlardan oluşan modüler bir mimaridir. Katmanlar şöyledir: veri alma (Fivetran, Airbyte), depolama ve işlem (Snowflake, BigQuery, Databricks), dönüşüm (dbt), orkestrasyon (Airflow, Dagster), görselleştirme (Looker, Power BI, Tableau) ve veri gözlemlenebilirliği (Monte Carlo, Great Expectations).*
>
> *dbt bu yığında ELT'nin 'T' yani dönüşüm katmanını üstlenir. Temel fikri şudur: her dönüşüm, bir SELECT sorgusu olarak yazılır ve dbt bunu ambarda tablo veya view'e dönüştürür. Yani veriyi taşımaz, ambarın kendi gücünü kullanır.*
>
> *Yaygınlaşmasının sebebi, SQL'e yazılım mühendisliği disiplinini getirmesidir. Modeller arası bağımlılıkları ref fonksiyonuyla tanımlarsınız ve dbt bundan otomatik bir DAG çıkarır; çalıştırma sırasını kendisi belirler. Veri kalitesi testlerini (unique, not_null, relationships, accepted_values) yapılandırma olarak tanımlarsınız ve bunlar hattı kırarak bozuk verinin ilerlemesini engeller. Ayrıca dokümantasyon ve veri soyağacını (lineage) otomatik üretir, Jinja makrolarıyla tekrarlayan SQL'i modülerleştirir ve snapshot özelliğiyle SCD Type 2 tarihçesini hazır olarak sunar.*
>
> *Kısacası dbt, SQL yazan analistlerin git, test ve CI/CD kullanarak yazılım geliştirici gibi çalışmasını mümkün kılar."*

2. Detaylı Arka Plan

Basit Bir dbt Modeli

SQL
```
-- models/silver/stg_siparis.sql
{{ config(materialized='incremental', unique_key='siparis_id') }}

SELECT
    siparis_id,
    musteri_id,
    CAST(tutar AS DECIMAL(12,2)) AS tutar,
    CAST(siparis_tarihi AS DATE)  AS siparis_tarihi,
    guncelleme_zamani
FROM {{ source('bronze', 'ham_siparis') }}

{% if is_incremental() %}
  -- yalnizca son calistirmadan sonra degisenleri isle
  WHERE guncelleme_zamani > (SELECT MAX(guncelleme_zamani) FROM {{ this }})
{% endif %}
```

SQL
```
-- models/gold/fact_musteri_ciro.sql
SELECT
    s.musteri_id,
    DATE_TRUNC('month', s.siparis_tarihi) AS ay,
    COUNT(*)        AS siparis_adedi,
    SUM(s.tutar)    AS toplam_ciro
FROM {{ ref('stg_siparis') }} s     -- ref() bagimliligi ve DAG'i olusturur
GROUP BY 1, 2
```

Materialization Türleri

- **view:** Her sorguda yeniden hesaplanır. Ucuz depolama, pahalı okuma. Küçük ve hafif ara katmanlar için.

- **table:** Her çalıştırmada tablo sıfırdan yeniden oluşturulur. Hızlı okuma, ama büyük tablolarda pahalı yeniden hesaplama.
- **incremental:** Yalnızca yeni veya değişen kayıtlar işlenir. Büyük fact tablolarında maliyeti dramatik şekilde düşürür; doğru unique_key ve filtre tanımı gerektirir.
- **ephemeral:** Fiziksel nesne oluşturmaz, kendisini referans veren modele CTE olarak gömülür. Küçük yardımcı mantıklar için.

dbt Testleri

```yaml
# models/schema.yml
models:
  - name: stg_siparis
    columns:
      - name: siparis_id
        tests:
          - unique
          - not_null
      - name: musteri_id
        tests:
          - not_null
          - relationships:
              to: ref('dim_musteri')
              field: musteri_id
      - name: durum
        tests:
          - accepted_values:
              values: ['BEKLEMEDE', 'TAMAMLANDI', 'IPTAL']
```

Airflow ve dbt Nasıl Birlikte Kullanılır?

Sık sorulan bir sorudur çünkü ikisi de "DAG" kelimesini kullanır ve bu kafa karıştırır. Ayrım nettir: dbt yalnızca ambar içindeki SQL model bağımlılıklarını yönetir. Airflow ise uçtan uca süreci yönetir; kaynaktan veri çekme, dbt'yi tetikleme, sonrasında BI önbelleğini yenileme ve bildirim gönderme gibi ambar dışı adımları da kapsar. Tipik kullanımda Airflow bir görev olarak dbt run ve dbt test komutlarını çalıştırır.

Yapılandırılmamış Veri ve Doküman Zekası

OCR ve Tesseract

> **SORU:** *Tesseract nedir ve nasıl çalışır? Sınırlamaları nelerdir? OCR kalitesini nasıl artırırsın?*

1. Mülakat Cevabı

> *"Tesseract, Google tarafından sürdürülen açık kaynaklı bir OCR yani Optik Karakter Tanıma motorudur. Görüntüdeki piksel desenlerini metne dönüştürür. Sürüm 4'ten itibaren LSTM tabanlı bir sinir ağı kullanır ve satır bazlı tanıma yapar.*
>
> *Çalışma akışı kabaca şudur: önce görüntü ön işlemeden geçer (eğrilik düzeltme, ikilileştirme, gürültü temizleme), ardından sayfa düzeni analiziyle metin blokları ve satırlar tespit edilir, son olarak her satır karakterlere çözümlenir.*
>
> *En önemli sınırlaması düzen körlüğüdür (layout blind). Tesseract metni okur ama o metnin sayfadaki anlamını bilmez. İki sütunlu bir PDF verdiğinizde satırları sayfa boyunca soldan sağa okuyup iki sütunu birbirine karıştırabilir; tablolarda ise hücre yapısını koruyamaz, sonuç mantıksal sırası bozulmuş bir metin yığını olur.*
>
> *Kaliteyi artırmak için önce girdiye odaklanırım: 300 DPI civarında tarama çözünürlüğü, yüksek kontrast, düzgün hizalanmış ve gürültüsüz görüntü. Ardından doğru sayfa segmentasyon modunu (PSM) seçerim; tek bir metin bloğu mu yoksa karmaşık düzen mi olduğunu belirtmek sonucu ciddi şekilde değiştirir. Doğru dil modelini yüklemek de kritiktir; Türkçe için tur veri paketi olmadan ı, ş, ğ gibi karakterlerde hata oranı yükselir."*

2. Detaylı Arka Plan

Ön İşlemenin Etkisi

OCR sonuçlarının kalitesini belirleyen en büyük faktör model değil, girdi görüntüsünün kalitesidir. Veri mühendisi olarak hattın başında uygulanacak adımlar:

- **Çözünürlük:** 300 DPI altındaki taramalarda karakter kenarları bulanıklaşır. Gerekirse yükseltme (upscaling) uygulanır.
- **Deskew (eğrilik düzeltme):** Tarayıcıda eğri duran sayfa, satır tespitini bozar. Küçük bir açı bile hata oranını ciddi artırır.
- **Binarization (ikilileştirme):** Görüntüyü siyah-beyaza indirger. Değişken aydınlatmada Otsu veya adaptive threshold yöntemleri kullanılır.
- **Gürültü temizleme ve leke giderme:** Fotokopi artefaktları karakter olarak yorumlanabilir.

Sayfa Segmentasyon Modu (PSM)

```bash
# PSM 6: tek bir tekduze metin blogu varsay (temiz paragraflarda iyi)
tesseract belge.png cikti -l tur --psm 6

# PSM 3: tam otomatik sayfa duzeni analizi (varsayilan)
tesseract belge.png cikti -l tur --psm 3

# PSM 11: seyrek metin, duzen varsaymadan bulabildigi kadar metin bul
tesseract fatura.png cikti -l tur --psm 11

# Metinle birlikte konum bilgisi (bounding box) uretmek icin:
tesseract belge.png cikti -l tur tsv
```

Son komut önemlidir: TSV çıktısı her kelimenin sayfadaki koordinatlarını da verir. LayoutLM gibi düzen farkındalıklı modeller tam olarak bu koordinat bilgisine ihtiyaç duyar; yani Tesseract çoğu modern hatta tek başına değil, bir ön adım olarak yaşamaya devam eder.

Ne Zaman Tesseract Yeterlidir?

Belge tek sütunlu, temiz taranmış ve düz metin ağırlıklıysa Tesseract fazlasıyla yeterlidir; üstelik ücretsizdir, şirket içinde çalışır ve veriyi dışarı göndermez. Bu son madde, veri gizliliği kısıtı olan kurumlarda (finans, sağlık, kamu) bulut OCR servislerine karşı belirleyici avantajdır. Buna karşılık fatura, form ve tablo gibi düzenin anlam taşıdığı belgelerde tek başına yetersiz kalır.

LayoutLM ve Doküman Zekası

> **SORU:** *LayoutLM klasik OCR'dan nasıl farklıdır? Hangi problemleri çözer ve ne zaman kullanmayı tercih edersin?*

1. Mülakat Cevabı

> *"Klasik OCR yalnızca 'ne yazıyor' sorusunu cevaplar. LayoutLM ise 'ne yazıyor, nerede yazıyor ve nasıl görünüyor' sorularını birlikte cevaplar. Microsoft tarafından geliştirilen çok modlu (multimodal) bir transformer modelidir.*
>
> *Üç tür bilgiyi aynı anda işler: metnin kendisi, metnin sayfadaki iki boyutlu konumu yani sınırlayıcı kutu koordinatları, ve görsel özellikler (yazı tipi büyüklüğü, kalınlık, çizgiler, tablo kenarlıkları). Bu sayede kelimenin anlamını konumundan da çıkarabilir.*
>
> *Çözdüğü asıl problem şudur: Bir faturada 'Toplam' kelimesinin hemen sağındaki veya altındaki sayının o toplamın değeri olduğunu anlamak, tamamen uzamsal bir ilişkidir. Düz metin akışında bu bilgi kaybolur. LayoutLM form anlama, fatura ve fiş üzerinden anahtar-değer çıkarımı, belge sınıflandırma ve tablo yapısı tanıma gibi görevlerde bu yüzden çok başarılıdır.*
>
> *Kullanım kararımda maliyet dengesi belirleyicidir. LayoutLM bir GPU gerektirir, kendi başına OCR yapmaz yani hâlâ önünde Tesseract gibi bir motora ihtiyaç duyar ve en iyi sonucu kendi belge tipinizle ince ayar (fine-tuning) yaptığınızda verir. Bu yüzden belge yapısı basitse klasik OCR ile devam ederim; düzenin anlam taşıdığı, yüksek hacimli ve iş açısından kritik belgelerde (fatura işleme gibi) LayoutLM veya benzeri bir doküman zekası modeline geçerim."*

2. Detaylı Arka Plan

Neden Düzen Bilgisi Bu Kadar Önemli?

Aşağıdaki iki sütunlu fatura parçasını düşün. Tesseract satır satır soldan sağa okuduğunda anlamsal bağ kopar:

```
Metin
GERCEK BELGE DUZENI:
    Fatura No: 2024-8891          Ara Toplam:   1.200,00
    Tarih:     05.09.2026         KDV (%20):      240,00
    Musteri:   ACME A.S.          GENEL TOPLAM: 1.440,00

DUZ OCR CIKTISI (duzen bilgisi kayip):
    "Fatura No: 2024-8891 Ara Toplam: 1.200,00 Tarih: 05.09.2026
     KDV (%20): 240,00 Musteri: ACME A.S. GENEL TOPLAM: 1.440,00"

-> Hangi sayinin hangi etikete ait oldugu yalnizca KONUMDAN anlasilir.
-> LayoutLM her token'in (x0, y0, x1, y1) koordinatini da girdi olarak alir.
```

LayoutLM Sürümlerinin Evrimi

- **LayoutLM (v1):** Metin ve 2B konum gömmelerini (position embedding) birleştirdi. Görsel özellikleri yalnızca son aşamada kullanıyordu.
- **LayoutLMv2:** Görsel özellikleri en baştan metinle birlikte işlemeye başladı ve uzamsal farkındalıklı dikkat (spatial-aware attention) mekanizması ekledi.
- **LayoutLMv3:** Metin ve görüntü için birleşik maskeleme ile ön eğitim yaptı, ayrı bir CNN omurgasına ihtiyacı ortadan kaldırarak mimariyi sadeleştirdi ve doğruluğu artırdı.

Tipik Doküman Zekası Hattı

```
Metin
```

```
PDF / Goruntu
      |
      v
[1] On isleme (deskew, binarize, DPI normalize)
      |
      v
[2] OCR (Tesseract / PaddleOCR)  -> metin + her kelimenin bbox koordinati
      |
      v
[3] LayoutLM  -> token siniflandirma (etiket mi, deger mi, hangi alan?)
      |
      v
[4] Son isleme -> yapilandirilmis JSON  {fatura_no, tarih, toplam, ...}
      |
      v
[5] Dogrulama (toplam = ara toplam + KDV ?) ve dusuk guvenli kayitlari
    insan onayina yonlendirme (human-in-the-loop)
```

Alternatifler ve Karar Kriterleri

- **Donut:** OCR'sız (OCR-free) bir yaklaşımdır; doğrudan görüntüden yapılandırılmış çıktı üretir. OCR hatalarının zincirleme yayılmasını ortadan kaldırır ama eğitimi daha zordur.
- Bulut servisleri (Azure Document Intelligence, AWS Textract, Google Document AI): Hazır, yüksek doğruluklu ve bakım gerektirmezler; fatura ve kimlik gibi yaygın belgeler için önceden eğitilmiş modeller sunarlar. Dezavantajı sayfa başına maliyet ve verinin dışarı çıkmasıdır.
- **Karar kriteri:** Belge hacmi, veri gizliliği kısıtı, belgenin standart mı özel mi olduğu ve ekipte ML yetkinliği bulunup bulunmadığı. Mülakatta "her zaman en gelişmiş modeli kullanırım" demek yerine bu dengeyi kurmak beklenir.

Mülakat Notu: Değerlendirme Metriği

"Bu modelin başarısını nasıl ölçersin?" sorusu gelebilir. OCR seviyesinde karakter hata oranı (CER) ve kelime hata oranı (WER) kullanılır. Alan çıkarımı seviyesinde ise alan bazında precision, recall ve F1 skoru ölçülür. İş tarafında asıl önemli metrik ise otomasyon oranıdır: kaç belge insan müdahalesi olmadan uçtan uca işlendi? Bu metriği söylemek, teknik çözümü iş değeriyle ilişkilendirdiğini gösterir.

unstructured.io ve LLM/RAG Veri Hatları

> **SORU:** *unstructured.io ne işe yarar? Bir RAG hattında veri mühendisi olarak sorumlulukların nelerdir?*

1. Mülakat Cevabı

> *"unstructured.io, dağınık belgeler ile büyük dil modelleri arasındaki boşluğu dolduran bir veri işleme kütüphanesidir. PDF, Word, PowerPoint, HTML, e-posta gibi onlarca farklı dosya tipini alır, her birini uygun ayrıştırma motoruna yönlendirir ve hepsini ortak, yapılandırılmış bir çıktıya dönüştürür.*
>
> *Bunu yaparken belgeyi düz metne indirgemez; anlamlı elemanlara ayırır. Her parçayı Title, NarrativeText, Table, ListItem gibi tiplerle etiketler ve sayfa numarası, kaynak dosya adı gibi metadata ekler. Çıktı temiz JSON olduğu için doğrudan bir sonraki adıma beslenebilir.*
>
> *İkinci önemli işlevi akıllı parçalama yani chunking'dir. Metni kör bir şekilde her 500 karakterde bir kesmek yerine, başlık sınırlarına saygı göstererek anlamsal bütünlüğü koruyan parçalar üretir.*
>
> *RAG hattında veri mühendisi olarak sorumluluklarım klasik veri mühendisliğinden çok farklı değil: kaynak belgeleri güvenilir şekilde toplamak, tekilleştirmek, kişisel verileri maskelemek, doğru chunking stratejisini seçmek, embedding üretimini yönetmek ve vektör veritabanına idempotent şekilde yazmak. Buradaki kritik nokta, her parçaya içerikten türetilmiş kararlı bir kimlik vermektir; böylece hat tekrar çalıştığında aynı belge iki kez indekslenmez. Ayrıca belge güncellendiğinde eski parçaların temizlenmesini sağlayan bir değişiklik yönetimi kurmak gerekir."*

2. Detaylı Arka Plan

Temel Kullanım

```python
from unstructured.partition.auto import partition
from unstructured.chunking.title import chunk_by_title

# Dosya tipini otomatik algilar ve dogru ayristiriciya yonlendirir
elements = partition(
    filename="sozlesme.pdf",
    strategy="hi_res",        # hi_res: goruntu/tablo icin model kullanir
                              # fast: yalnizca metin katmani (cok daha hizli)
                              # ocr_only: taranmis belgeler icin
    languages=["tur"],
)

for el in elements[:5]:
    print(type(el).__name__, "|", el.text[:60])
# Title          | HIZMET SOZLESMESI
# NarrativeText  | Isbu sozlesme, asagida bilgileri yer alan taraflar...
# Table          | Kalem | Tutar | KDV ...

# Anlamsal sinirlara saygi gosteren parcalama
chunks = chunk_by_title(
    elements,
    max_characters=1000,
    overlap=100,              # baglami korumak icin ust uste binme
)
```

Strateji Seçimi: Maliyet ve Kalite Dengesi

- **fast:** PDF'in gömülü metin katmanını okur. Çok hızlı ve ucuzdur, ancak taranmış (görüntü) PDF'lerde hiçbir şey döndürmez.
- **hi_res:** Düzen tespiti için görüntü modeli çalıştırır; tabloları ve karmaşık düzenleri çok daha iyi çıkarır. Yavaş ve maliyetlidir.

- **ocr_only:** Doğrudan OCR uygular. Taranmış belgeler için tek seçenektir.

Üretim hattında yaygın desen, belgenin metin katmanı olup olmadığını tespit edip stratejiyi belge bazında dinamik seçmektir; tüm koleksiyona hi_res uygulamak gereksiz maliyet üretir.

Chunking Neden Bu Kadar Kritik?

RAG sistemlerinde cevap kalitesini belirleyen en büyük etken çoğu zaman model değil, getirilen bağlamın kalitesidir. Kötü chunking iki tür hata üretir: Parça çok küçükse bağlam kopar ve model soruyu cevaplayacak bilgiyi bulamaz. Parça çok büyükse alakasız metin de vektöre karışır, benzerlik araması bulanıklaşır ve yanlış parçalar getirilir.

Ayrıca bir tabloyu ortasından bölmek veya bir başlığı ait olduğu paragraftan ayırmak, o bilgiyi pratikte erişilemez hale getirir. unstructured'ın eleman tiplerini bilmesi tam da bu yüzden değerlidir.

Idempotent Vektör Yazma

```python
import hashlib

def parca_kimligi(kaynak_dosya: str, chunk_index: int, metin: str) -> str:
    """Icerikten turetilen kararli kimlik: ayni girdi -> ayni id."""
    ham = f"{kaynak_dosya}|{chunk_index}|{metin}"
    return hashlib.sha256(ham.encode("utf-8")).hexdigest()

# Vektor veritabanina UPSERT: hat tekrar calisirsa cift kayit olusmaz
vektor_db.upsert([
    {
        "id": parca_kimligi(dosya_adi, i, c.text),
        "values": embed(c.text),
        "metadata": {
            "kaynak": dosya_adi,
            "sayfa": c.metadata.page_number,
            "belge_versiyonu": belge_hash,   # degisiklik tespiti icin
        },
    }
    for i, c in enumerate(chunks)
])

# Belge guncellendiginde eski parcalari temizle
vektor_db.delete(filter={"kaynak": dosya_adi, "belge_versiyonu": {"$ne": belge_hash}})
```

Mülakat Sorusu: "RAG hattı klasik ETL'den farklı mı?"

Güçlü cevap: Prensipler aynıdır, yalnızca dönüşüm adımları değişmiştir. Extract adımında tablo yerine belge toplarsın; Transform adımında SQL yerine ayrıştırma, chunking ve embedding üretimi yaparsın; Load adımında veri ambarı yerine vektör veritabanına yazarsın. Ama idempotency, tekilleştirme, şema/metadata yönetimi, artımlı yükleme, veri soyağacı ve kalite testleri ihtiyacı birebir aynıdır. Hatta RAG'de veri kalitesinin etkisi daha görünürdür; çünkü bozuk veri sessiz bir rapor hatası olarak değil, kullanıcıya yanlış cevap olarak geri döner.

Değerlendirme (Evaluation) Sorumluluğu

Veri mühendisi olarak hattın çıktısını ölçmek de senin sorumluluğundadır. RAG'de iki katman ayrı ayrı ölçülür: Getirme (retrieval) kalitesi, doğru parçanın ilk k sonuç içinde gelip gelmediğiyle ölçülür (recall@k, MRR). Üretim (generation) kalitesi ise cevabın getirilen bağlama sadık olup olmadığıyla ölçülür (faithfulness/groundedness). Chunking stratejisini değiştirdiğinde bu metrikleri yeniden ölçmek, hattı bilimsel şekilde iyileştirmenin tek yoludur.

Veri Ambarı Tasarımı ve Stored Procedure'ler

İlişkisel (ER) ve Boyutsal Modelleme, Kardinalite

> **SORU:** *Bir uygulama veritabanı (OLTP) tasarımıyla veri ambarı (OLAP) tasarımı arasındaki fark nedir? Kardinalite türlerini ve çoka-çok ilişkiyi nasıl çözdüğünü anlat.*

1. Mülakat Cevabı

> *"İki tasarımın hedefi tamamen farklıdır, bu yüzden kuralları da farklıdır.*
>
> *OLTP tarafında Varlık-İlişki (Entity-Relationship) modellemesi kullanırım. Amaç veri tekrarını ortadan kaldırmak ve yüksek hacimli INSERT, UPDATE, DELETE işlemleri sırasında bütünlüğü korumaktır. Tablolar 3NF'e kadar normalize edilir ve şema, birbirine bağlanan çok sayıda tablodan oluşan karmaşık bir ağ görünümü alır.*
>
> *OLAP tarafında ise boyutsal modelleme kullanırım. Amaç ağır SELECT ve toplulaştırma sorgularını hızlandırmak ve modeli iş kullanıcısı için anlaşılır kılmaktır. Veriyi bilinçli olarak denormalize ederim; şema, merkezde bir fact tablosu ve etrafında doğrudan ona bağlanan boyut tablolarından oluşan bir yıldıza dönüşür.*
>
> *Kardinalite, iki varlık arasındaki ilişkinin sayısal doğasıdır. Birebir (1:1) ilişkide her iki taraftaki kayıt en fazla bir kayıtla eşleşir; nadiren kullanılır. Bire-çok (1:N) en yaygın olanıdır ve ilişki, çok tarafına yabancı anahtar konarak kurulur. Çoka-çok (M:N) ilişki ise ilişkisel veritabanında doğrudan kurulamaz; araya bir bağlantı tablosu (junction veya bridge table) koyarak iki adet bire-çok ilişkiye dönüştürülür.*
>
> *Boyutsal modellemede ise en kritik karar taneciktir (grain): fact tablosunda bir satırın tam olarak neyi temsil ettiğini modellemeye başlamadan önce ilan ederim. Grain'i yanlış belirlemek, sonradan düzeltilmesi en pahalı tasarım hatasıdır."*

2. Detaylı Arka Plan

Kardinalite Türleri ve SQL Karşılıkları

- **Birebir (1:**1): Her iki tarafta da en fazla bir eşleşme vardır. Genellikle güvenlik (hassas sütunları ayrı tabloya almak), performans (nadiren okunan geniş sütunları ayırmak) veya opsiyonel öznitelik grupları için kullanılır. Yabancı anahtar taraflardan birine konur ve üzerine UNIQUE kısıtı eklenir.
- **Bire-çok (1:**N): En yaygın ilişkidir. Bir müşterinin çok sayıda siparişi olabilir. Yabancı anahtar her zaman "çok" tarafına konur; sipariş tablosuna musteri_id eklenir.
- **Çoka-çok (M:**N): Bir öğrenci birçok derse kayıt olabilir, bir derse birçok öğrenci kayıt olabilir. İlişkisel model bunu doğrudan ifade edemez; bağlantı tablosu şarttır.

```SQL
-- M:N iliski: araya baglanti (junction) tablosu konur
CREATE TABLE ogrenci (
    ogrenci_id INT PRIMARY KEY,
    ad_soyad   VARCHAR(200) NOT NULL
);

CREATE TABLE ders (
    ders_id INT PRIMARY KEY,
    ders_adi VARCHAR(200) NOT NULL
);

-- Baglanti tablosu: M:N iliskiyi iki adet 1:N iliskiye cevirir
CREATE TABLE ders_kayit (
    ogrenci_id  INT NOT NULL REFERENCES ogrenci(ogrenci_id),
    ders_id     INT NOT NULL REFERENCES ders(ders_id),
```

```
    kayit_tarihi DATE NOT NULL,
    not_ortalamasi DECIMAL(4,2),          -- iliskiye ait oznitelik
    PRIMARY KEY (ogrenci_id, ders_id)     -- bilesik anahtar
);
```

Dikkat edilecek nokta: Bağlantı tablosu yalnızca iki anahtarı taşıyan pasif bir tablo değildir. İlişkinin kendisine ait öznitelikler (kayıt tarihi, not, adet, fiyat) burada yaşar. Veri ambarında bu yapının karşılığı genellikle bir fact tablosudur; zaten fact tabloları özünde birden fazla boyut arasındaki M:N ilişkiyi ölçülerle birlikte tutan yapılardır.

Yapısal Fark: ER Ağı ve Yıldız Şema

```
Şema karşılaştırması
OLTP (ER modeli) - normalize, karmasik ag:

    sehir --- musteri --- siparis --- siparis_kalem --- urun
                |           |                           |
              musteri_tel  kargo                      kategori --- marka
                             |
                         kargo_firma

  -> Tek bir rapor icin 6-7 tablo JOIN edilir.
  -> Yazma hizli ve tutarli, okuma karmasik ve yavas.


OLAP (Yildiz sema) - denormalize, merkezde fact:

              dim_tarih      dim_musteri
                    \           /
                     \         /
        dim_magaza ---- fact_satis ---- dim_urun
                       (olculer:
                        adet, tutar,
                        indirim)

  -> Her boyut fact'e DOGRUDAN baglidir, tek seviye JOIN.
  -> Okuma hizli ve anlasilir, yazma toplu ve kontrollu (ETL/ELT).
```

Ne Zaman Hangisi?

Kriter	ER Modelleme (OLTP)	Boyutsal Modelleme (OLAP)
Amaç	Veri bütünlüğü, hızlı yazma	Hızlı okuma ve toplulaştırma
Normalizasyon	3NF veya üzeri	Bilinçli denormalizasyon
Şema görünümü	Karmaşık ağ	Yıldız (star)
Tipik işlem	Tek kayıt INSERT / UPDATE	Milyonlarca satır SELECT + GROUP BY
Kullanıcı	Uygulama ve son kullanıcı	Analist, BI aracı, veri bilimci
Anahtar tercihi	Natural veya surrogate	Her zaman surrogate key

İlgili bölümler: Fact ve dimension türleri, grain, ölçü türleri ve Star/Snowflake karşılaştırması için "Boyutsal Modelleme: Star ve Snowflake Şema"; normal formların adım adım örneği için "Normalizasyon ve Denormalizasyon"; satır ve kolon bazlı depolama farkı için "OLTP ve OLAP Sistemleri" bölümlerine bakabilirsin.

Surrogate Key: Neden Kaynak Sistemin Anahtarını Kullanmayız?

> **SORU:** *Veri ambarında neden kaynak sistemin doğal anahtarı (TC kimlik no, ürün kodu) yerine kendi ürettiğimiz surrogate key'i kullanırız?*

1. Mülakat Cevabı

> *"Surrogate key, iş anlamı taşımayan, veri ambarının kendisi tarafından üretilen ve genellikle artan bir tamsayı olan yapay anahtardır. Natural key (doğal anahtar) ise kaynak sistemde zaten var olan ve iş anlamı taşıyan anahtardır.*
>
> *Veri ambarı boyut tablolarında surrogate key kullanmamın beş sebebi var.*
>
> *Birincisi ve en önemlisi, Slowly Changing Dimension Type 2'yi mümkün kılmasıdır. Aynı müşterinin zaman içindeki farklı versiyonlarını ayrı satırlarda tutmam gerekir; doğal anahtar bu satırların hepsinde aynı olduğu için birincil anahtar olamaz.*
>
> *İkincisi kaynak sistem değişikliklerinden yalıtımdır. Kaynak sistem ürün kodu formatını değiştirirse veya şirket başka bir uygulamaya geçerse, ambar modeli ve geçmiş veriler etkilenmez.*
>
> *Üçüncüsü birden fazla kaynak sistemi birleştirebilmektir. İki farklı sistemde aynı müşteri farklı numaralarla bulunabilir; daha kötüsü, farklı müşteriler aynı numaraya sahip olabilir.*
>
> *Dördüncüsü performanstır. Dar bir tamsayı üzerinden JOIN yapmak, uzun bir metin veya bileşik anahtara göre çok daha hızlıdır ve milyarlarca satırlık fact tablosunda çok daha az yer kaplar.*
>
> *Beşincisi eksik ve bilinmeyen kayıtları yönetebilmektir. -1 gibi özel bir surrogate key ile 'Bilinmiyor' satırı tanımlarım ve fact tablosunda NULL yabancı anahtar bulundurmaktan kurtulurum."*

2. Detaylı Arka Plan

Surrogate Key SCD Type 2'yi Nasıl Mümkün Kılar?

```SQL
-- dim_musteri: ayni musterinin (natural key = 50) IKI versiyonu var
+-------------+------------+--------------+---------+------------+------------+-------+
| musteri_key | musteri_no | ad_soyad     | sehir   | baslangic  | bitis      | aktif |
|  SURROGATE  |  NATURAL   |              |         |            |            |       |
+-------------+------------+--------------+---------+------------+------------+-------+
| 1001        | 50         | Ahmet Yilmaz | Ankara  | 2023-01-01 | 2025-05-31 | false |
| 1002        | 50         | Ahmet Yilmaz | Istanbul| 2025-06-01 | NULL       | true  |
+-------------+------------+--------------+---------+------------+------------+-------+

-- fact_satis NATURAL key'e degil SURROGATE key'e baglanir:
--   2024'teki satis  -> musteri_key = 1001  -> raporda "Ankara" cikar (DOGRU)
--   2026'daki satis  -> musteri_key = 1002  -> raporda "Istanbul" cikar (DOGRU)
--
-- Eger fact, musteri_no = 50'ye baglansaydi tum gecmis satislar
-- bugunku sehre (Istanbul) kayardi ve gecmis raporlar bozulurdu.
```

Buna "as-of" raporlama denir: olayın gerçekleştiği andaki boyut değerleriyle raporlayabilmek. Veri ambarının temel vaadi budur ve surrogate key olmadan sağlanamaz.

Bilinmeyen Üye (Unknown Member) Deseni

Kaynaktan gelen bir fact kaydının boyutta karşılığı bulunamayabilir; örneğin sipariş kaydı gelmiştir ama müşteri kaydı henüz aktarılmamıştır. Satırı atmak ciroyu eksik gösterir, NULL yabancı anahtar ise JOIN'lerde kaydın kaybolmasına yol açar. Doğru çözüm her boyut tablosunda özel bir satır bulundurmaktır.

```SQL
-- Her boyut tablosuna standart "bilinmeyen" satirlari eklenir
INSERT INTO dim_musteri (musteri_key, musteri_no, ad_soyad, aktif)
VALUES (-1, NULL, 'Bilinmiyor',   TRUE),
```

```
       (-2, NULL, 'Uygulanamaz',  TRUE);

-- ETL sirasinda eslesmeyen kayitlar -1'e baglanir
INSERT INTO fact_satis (musteri_key, urun_key, tutar)
SELECT
    COALESCE(d.musteri_key, -1),      -- eslesme yoksa "Bilinmiyor"
    u.urun_key,
    s.tutar
FROM staging_satis s
LEFT JOIN dim_musteri d ON d.musteri_no = s.musteri_no AND d.aktif = TRUE
LEFT JOIN dim_urun    u ON u.urun_kodu  = s.urun_kodu;
```

Bu desenin faydası şudur: toplamlar her zaman doğru kalır ve sorun raporda "Bilinmiyor" satırı olarak görünür hale gelir. Sessizce kaybolan veri, görünen hatadan çok daha tehlikelidir.

Mülakat Sorusu: Doğal Anahtar Ambarda Hiç Tutulmaz mı?

Tutulur, ancak birincil anahtar olarak değil. Boyut tablosunda doğal anahtar mutlaka bir sütun olarak saklanır; ETL'in kaynakla eşleştirme yapabilmesi, hata ayıklama ve kaynak sisteme geri izlenebilirlik (traceability) için gereklidir. Fact tablosunda ise sipariş numarası gibi doğal anahtarlar bazen boyut tablosu olmadan doğrudan tutulur; buna degenerate dimension denir ve fiş veya sipariş bazında gruplama yapabilmek için kullanılır.

SCD Type 1, 2 ve 3 arasındaki farklar, tablo yapısı ve MERGE ile yükleme örneği için "Slowly Changing Dimensions (SCD)" bölümüne bakabilirsin.

Stored Procedure: Transaction Yönetimi ve Hata Yakalama

> **SORU:** *Bir stored procedure içinde birden fazla DML işlemini nasıl güvenli hale getirirsin? TRY...CATCH ile transaction'ı nasıl birlikte kullanırsın?*

1. Mülakat Cevabı

> *"Birden fazla tabloyu değiştiren bir prosedürde işlemlerin ya tamamının uygulanmasını ya da hiçbirinin uygulanmamasını garanti etmem gerekir. Bunun için BEGIN TRANSACTION ile başlar, tüm DML işlemlerini aynı transaction içine alır ve sorun yoksa COMMIT ederim.*
>
> *Hata yönetimini TRY…CATCH bloğuyla yaparım: TRY içinde transaction'ı açıp işlemleri yaparım, bir hata oluşursa akış CATCH bloğuna düşer ve orada ROLLBACK yaparım.*
>
> *CATCH bloğunda üç şeye dikkat ederim. Birincisi, geri alma yapmadan önce gerçekten açık bir transaction olup olmadığını XACT_STATE veya @@TRANCOUNT ile kontrol ederim; çünkü bazı ağır hatalar transaction'ı zaten geri alınamaz duruma sokar ve kontrolsüz bir ROLLBACK yeni bir hata üretir. İkincisi, hatayı asla sessizce yutmam; ERROR_MESSAGE, ERROR_LINE ve ERROR_PROCEDURE gibi fonksiyonlarla detayları bir log tablosuna yazarım. Üçüncüsü, log yazdıktan sonra THROW ile hatayı yukarı fırlatırım ki çağıran katman, örneğin Airflow veya SSIS, işi başarısız olarak işaretlesin.*
>
> *Ayrıca prosedürün başına SET XACT_ABORT ON yazmayı tercih ederim; bu, çalışma zamanı hatalarında transaction'ın otomatik geri alınmasını sağlar ve yarım kalmış açık transaction riskini ortadan kaldırır. SET NOCOUNT ON ise gereksiz 'x rows affected' mesajlarını kapatarak ağ trafiğini azaltır."*

2. Detaylı Arka Plan

Güvenli Prosedür İskeleti (T-SQL)

```T-SQL
CREATE OR ALTER PROCEDURE dbo.sp_SiparisAktar
    @IslemTarihi DATE
AS
BEGIN
    SET NOCOUNT ON;         -- gereksiz "rows affected" mesajlarini kapat
    SET XACT_ABORT ON;      -- calisma zamani hatasinda otomatik rollback

    BEGIN TRY
        BEGIN TRANSACTION;

        -- 1) Hedef partition'i temizle (idempotency icin)
        DELETE FROM dbo.fact_satis
        WHERE islem_tarihi = @IslemTarihi;

        -- 2) Yeniden yukle
        INSERT INTO dbo.fact_satis (islem_tarihi, musteri_key, urun_key, tutar)
        SELECT s.islem_tarihi, COALESCE(d.musteri_key, -1), u.urun_key, s.tutar
        FROM   dbo.stg_satis s
        LEFT JOIN dbo.dim_musteri d
               ON d.musteri_no = s.musteri_no AND d.aktif = 1
        LEFT JOIN dbo.dim_urun u
               ON u.urun_kodu = s.urun_kodu
        WHERE s.islem_tarihi = @IslemTarihi;

        -- 3) Yukleme kaydini logla
        INSERT INTO dbo.etl_log (prosedur, islem_tarihi, satir_sayisi, durum)
        VALUES ('sp_SiparisAktar', @IslemTarihi, @@ROWCOUNT, 'BASARILI');

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        -- Geri alinabilir bir transaction var mi?
```

```
        IF XACT_STATE() <> 0
            ROLLBACK TRANSACTION;

        -- Hatayi sessizce yutma: once logla
        INSERT INTO dbo.etl_hata_log
            (prosedur, islem_tarihi, hata_no, hata_mesaji, hata_satiri, olusma_zamani)
        VALUES
            ('sp_SiparisAktar', @IslemTarihi, ERROR_NUMBER(),
             ERROR_MESSAGE(), ERROR_LINE(), SYSDATETIME());

        -- Sonra yukari firlat ki cagiran katman basarisiz saysin
        THROW;
    END CATCH
END;
```

XACT_STATE() Değerleri

Değer	Anlamı	Yapılması Gereken
1	Açık ve geri alınabilir (committable) transaction var	COMMIT veya ROLLBACK yapılabilir
0	Açık transaction yok	ROLLBACK çağırma, hata verir
-1	Açık ama geri alınamaz (uncommittable) transaction var	Yalnızca ROLLBACK yapılabilir, COMMIT hata verir

Mülakat Tuzağı: THROW Olmadan Sessiz Başarısızlık

Bu, veri hatlarında en tehlikeli hatalardan biridir. CATCH bloğunda hatayı yakalayıp yalnızca loglayan ve THROW etmeyen bir prosedür, çağıran sisteme başarılı görünür. Airflow görevi yeşile döner, aşağı akış işleri çalışır ve eksik veriyle raporlar üretilir. Kimse bir sorun olduğunu fark etmez.

Kural şudur: Hata yakalamak, hatayı gizlemek değildir. Yakala, bağlamı zenginleştirerek logla, sonra mutlaka yeniden fırlat.

THROW ve RAISERROR Farkı

- **THROW:** SQL Server 2012 ile geldi ve modern tercihtir. Parametresiz kullanıldığında orijinal hatayı numarası ve mesajıyla birlikte olduğu gibi yeniden fırlatır. Kullanımı basittir.
- **RAISERROR:** Eski yöntemdir. Mesajı biçimlendirme ve önem derecesi (severity) belirleme esnekliği sunar, ancak orijinal hata numarasını koruyamaz ve kendi hata numarasını üretir. Yeni kodda THROW tercih edilir.

İç İçe Transaction ve @@TRANCOUNT

SQL Server gerçek anlamda iç içe transaction desteklemez. İçteki BEGIN TRANSACTION yalnızca @@TRANCOUNT sayacını artırır; içteki COMMIT ise sayacı azaltır ama hiçbir şeyi kalıcı yapmaz. Yalnızca en dıştaki COMMIT gerçek commit'i yapar. Buna karşılık herhangi bir seviyedeki ROLLBACK, tüm transaction'ı en baştan geri alır ve @@TRANCOUNT değerini sıfırlar. Bu asimetri, birbirini çağıran prosedürlerde beklenmedik davranışlara yol açar; bu yüzden transaction sınırlarını mümkün olduğunca tek bir yerde, en dış katmanda yönetmek en güvenli yaklaşımdır.

Stored Procedure Performansı: Execution Plan, RBAR ve Parameter Sniffing

> **SORU:** *Bir stored procedure bazen çok hızlı bazen çok yavaş çalışıyor. Nasıl teşhis edersin? Parameter sniffing nedir ve nasıl çözülür?*

1. Mülakat Cevabı

> *"Önce tahmin etmem, ölçerim: gerçek yürütme planını (actual execution plan) alırım. Baktığım şeyler; tahmini ve gerçek satır sayıları arasındaki büyük sapmalar, beklenmedik tablo taramaları, diske taşan sıralama işlemleri ve pahalı key lookup'lardır.*
>
> *'Bazen hızlı bazen yavaş' tarifi klasik olarak parameter sniffing'e işaret eder. SQL Server bir prosedürü ilk kez çalıştırdığında, o anki parametre değerine göre bir plan üretir ve önbelleğe alır. Sonraki çağrılar tamamen farklı seçicilikte parametrelerle gelse bile aynı planı kullanır. Örneğin ilk çağrı tek bir müşteriyi getiriyorsa index seek içeren bir plan üretilir; aynı plan bütün müşterileri getiren bir çağrıda kullanıldığında felakete dönüşür.*
>
> *Çözüm olarak duruma göre şunları kullanırım: OPTION (RECOMPILE) ile her çalıştırmada yeni plan ürettirmek, OPTIMIZE FOR ile tipik bir değere göre plan sabitlemek, parametreyi yerel bir değişkene kopyalayarak ortalama tahmine zorlamak veya prosedürü farklı senaryolar için ayrı prosedürlere bölmek.*
>
> *Bir diğer yaygın performans katili RBAR'dır, yani 'Row By Agonizing Row' işleme. Cursor veya WHILE döngüsüyle satır satır dönmek yerine küme tabanlı (set-based) tek bir UPDATE, MERGE veya INSERT...SELECT yazarım. Aradaki fark genellikle yüzlerce kat hızdır; çünkü SQL Server küme işlemleri için optimize edilmiştir, döngüler için değil."*

2. Detaylı Arka Plan

RBAR: Cursor Yerine Küme Tabanlı Yazım

```T-SQL
-- YANLIS (RBAR): 500.000 satir icin 500.000 ayri UPDATE
DECLARE @id INT, @tutar DECIMAL(12,2);
DECLARE c CURSOR FOR SELECT siparis_id, tutar FROM siparis;
OPEN c;
FETCH NEXT FROM c INTO @id, @tutar;
WHILE @@FETCH_STATUS = 0
BEGIN
    UPDATE siparis SET indirimli_tutar = @tutar * 0.9 WHERE siparis_id = @id;
    FETCH NEXT FROM c INTO @id, @tutar;
END
CLOSE c; DEALLOCATE c;
-- Sure: dakikalar

-- DOGRU (set-based): tek islem
UPDATE siparis
SET indirimli_tutar = tutar * 0.9;
-- Sure: saniyeler
```

Cursor'ün meşru kullanım alanları çok dardır: her satır için farklı bir prosedür çağırmak veya yönetimsel görevler gibi. Veri dönüşümünde neredeyse her zaman küme tabanlı bir karşılığı vardır. Mülakatta "Cursor kullanır mısın?" sorusuna "Önce küme tabanlı bir çözüm ararım, cursor son çaredir" demek doğru cevaptır.

Parameter Sniffing Örneği ve Çözümleri

```T-SQL
CREATE PROCEDURE dbo.sp_SiparisGetir @MusteriId INT
AS
SELECT * FROM siparis WHERE musteri_id = @MusteriId;

-- Ilk cagri: 3 siparisi olan musteri  -> Index Seek plani onbellege alinir
```

```
EXEC dbo.sp_SiparisGetir @MusteriId = 42;

-- Sonraki cagri: 2 milyon siparisi olan kurumsal musteri
-- Ayni Index Seek plani kullanilir -> milyonlarca key lookup -> felaket
EXEC dbo.sp_SiparisGetir @MusteriId = 7;



-- COZUM 1: her calistirmada yeniden derle (kucuk CPU maliyeti)
SELECT * FROM siparis WHERE musteri_id = @MusteriId
OPTION (RECOMPILE);

-- COZUM 2: tipik bir degere gore plan uret
SELECT * FROM siparis WHERE musteri_id = @MusteriId
OPTION (OPTIMIZE FOR (@MusteriId = 42));

-- COZUM 3: ortalama tahmine zorla (parametreyi yerel degiskene kopyala)
DECLARE @Local INT = @MusteriId;
SELECT * FROM siparis WHERE musteri_id = @Local;
```

Yürütme Planında Aranacaklar

- **Tahmini ve gerçek satır farkı:** Aralarında kat kat fark varsa istatistikler bayattır. Çözüm UPDATE STATISTICS çalıştırmak veya otomatik istatistik güncellemesinin yeterli olup olmadığını gözden geçirmektir.
- **Table Scan veya Clustered Index Scan:** Büyük tabloda beklenmiyorsa index eksikliğine veya SARGable olmayan bir koşula işaret eder.
- **Key Lookup:** Index aranan satırı buluyor ama SELECT'teki diğer sütunlar için asıl tabloya gidiliyor demektir. Çok sayıda tekrarlanıyorsa covering index (INCLUDE ile) çözer.
- **Sort ve Hash Warning:** İşlem belleğe sığmayıp diske taşıyor demektir (spill to tempdb). Bellek tahsisi yetersizdir veya kardinalite tahmini yanlıştır.
- **Missing Index önerisi:** Planın önerdiği index'i körü körüne uygulamamak gerekir; öneriler tek sorguya odaklıdır, sütun sırasını optimize etmez ve yazma maliyetini hesaba katmaz. Öneriyi bir ipucu olarak alıp mevcut index'lerle birlikte değerlendiririm.

SARGability, composite index sütun sırası ve covering index detayları için "Index'ler: Clustered ve Non-Clustered" ve "Sorgu Optimizasyonu" bölümlerine bakabilirsin.

Temp Table, Table Variable ve CTE Seçimi

> **SORU:** *#TempTable, @TableVariable ve CTE arasında nasıl seçim yaparsın? Hangisi indexlenebilir?*

1. Mülakat Cevabı

> *"Üçü de ara sonuç tutmaya yarar ama davranışları ve maliyetleri çok farklıdır.*
>
> *CTE fiziksel bir nesne değildir; yalnızca sorguyu okunabilir hale getiren, isimlendirilmiş bir alt sorgudur. Optimizer onu genellikle ana sorguya gömer (inline eder). Bu yüzden sonucu bir kez hesaplayıp saklamaz; aynı CTE'ye sorguda üç kez referans verirsem üç kez hesaplanabilir. Üzerine index konulamaz ve istatistik tutmaz.*
>
> *Temp table (#tablo) tempdb'de gerçek bir tablodur. Üzerine index koyabilirim, istatistik tutar ve bu sayede optimizer sonraki adımlar için doğru plan üretebilir. Ara sonuç büyükse, birden fazla kez kullanılacaksa veya üzerinde JOIN yapılacaksa doğru seçim budur.*
>
> *Table variable (@tablo) da tempdb'de yaşar ancak optimizer geleneksel olarak içinde yalnızca bir satır olduğunu varsayar. Bu yüzden büyük veri kümelerinde çok kötü planlara yol açar. Küçük ve öngörülebilir boyuttaki ara sonuçlar için uygundur; ayrıca transaction'dan etkilenmez ve daha az kilit üretir.*
>
> *Pratik kuralım şudur: okunabilirlik ve tek geçişlik mantık için CTE, büyük ara sonuç ve index gereken durumlar için temp table, birkaç yüz satırlık küçük kümeler için table variable."*

2. Detaylı Arka Plan

Karşılaştırma Tablosu

Özellik	CTE	#TempTable	@TableVariable
Fiziksel nesne	Hayır (inline edilir)	Evet (tempdb)	Evet (tempdb)
Index konulabilir	Hayır	Evet	Sınırlı (PK/UNIQUE)
İstatistik tutar	Hayır	Evet	Hayır
Yeniden kullanım	Her referansta yeniden hesaplanabilir	Bir kez doldurulur, çok kez okunur	Bir kez doldurulur
Kapsam	Yalnızca o sorgu	Oturum / prosedür	Yalnızca o toplu iş (batch)
Uygun kullanım	Okunabilirlik, recursive sorgu	Büyük ara sonuç, tekrar kullanım	Küçük küme, parametre taşıma

Ne Zaman Temp Table Gerçekten Fark Yaratır?

```T-SQL
-- SORUNLU: karmasik CTE uc kez referans veriliyor, uc kez hesaplaniyor
WITH aylik AS (
    SELECT musteri_id, DATEFROMPARTS(YEAR(tarih), MONTH(tarih), 1) AS ay,
           SUM(tutar) AS ciro
    FROM siparis
    GROUP BY musteri_id, DATEFROMPARTS(YEAR(tarih), MONTH(tarih), 1)
)
SELECT a.*, o.ciro AS onceki_ay, y.ciro AS gecen_yil
FROM aylik a
LEFT JOIN aylik o ON ...     -- ikinci hesaplama
LEFT JOIN aylik y ON ...;    -- ucuncu hesaplama

-- COZUM: bir kez hesapla, temp table'a al, index koy
SELECT musteri_id, DATEFROMPARTS(YEAR(tarih), MONTH(tarih), 1) AS ay,
       SUM(tutar) AS ciro
INTO #aylik
FROM siparis
```

```
GROUP BY musteri_id, DATEFROMPARTS(YEAR(tarih), MONTH(tarih), 1);

CREATE CLUSTERED INDEX ix_aylik ON #aylik (musteri_id, ay);

SELECT a.*, o.ciro AS onceki_ay, y.ciro AS gecen_yil
FROM #aylik a
LEFT JOIN #aylik o ON ...
LEFT JOIN #aylik y ON ...;
```

Mülakat Notu: Table Variable Gerçekten Bellekte mi Yaşar?

Yaygın bir yanlış inanış, table variable'ın bellekte, temp table'ın diskte olduğudur. Doğrusu şudur: her ikisi de tempdb'de oluşturulur ve her ikisi de yeterli bellek varsa fiilen bellekte tutulur. Aradaki gerçek fark depolama yeri değil, istatistik ve kardinalite tahminidir. Table variable istatistik tutmadığı için optimizer satır sayısını genellikle 1 varsayar ve büyük kümelerde nested loop gibi yanlış join stratejileri seçer.

SQL Server 2019 ile gelen "table variable deferred compilation" özelliği bu sorunu kısmen hafifletti; artık plan, table variable doldurulduktan sonra gerçek satır sayısıyla derlenebiliyor. Bu detayı bilmek güncel olduğunu gösterir, ancak temel tercih kuralı değişmedi.

CTE'nin materialize edilip edilmediği, recursive CTE ve view/materialized view karşılaştırması için "CTE (Common Table Expressions) ve Recursive CTE" bölümüne bakabilirsin.

Microsoft BI Yığını ve Veri Yönetimi

SSAS Tabular ve VertiPaq Motoru

> **SORU:** *SSAS Tabular nedir? 100 milyon satırlık bir tabloda tek bir sütunun toplamını neden milisaniyeler içinde hesaplayabiliyor?*

1. Mülakat Cevabı

> *"SSAS Tabular, Microsoft BI yığınının semantik katmanıdır. Veri ambarının üzerine kurulur ve iş kullanıcısının anlayacağı dilde tablolar, ilişkiler, hiyerarşiler ve ölçüler (measure) tanımlar. Power BI ve Excel gibi araçlar bu modele bağlanır. İş mantığı tek bir yerde tanımlandığı için her rapor aynı 'net kâr' tanımını kullanır. Eski Multidimensional (küp ve MDX) modellerin yerini büyük ölçüde almıştır.*
>
> *Hızının kaynağı VertiPaq adı verilen bellek içi, kolon bazlı depolama motorudur ve üç şey yapar.*
>
> *Birincisi veriyi satır satır değil kolon kolon saklar. Bir ölçü hesaplanırken yalnızca ilgili sütunun verisi okunur; 200 sütunlu bir modelde diğer 199 sütuna hiç dokunulmaz.*
>
> *İkincisi çok agresif sıkıştırma uygular. Aynı sütundaki değerler aynı tipte ve büyük ölçüde tekrarlı olduğu için sözlük kodlaması, değer kodlaması ve run-length encoding ile veri çoğu zaman onda birine iner. Bu sayede diskte yüz gigabaytlık bir veri modeli belleğe sığar.*
>
> *Üçüncüsü veriyi tamamen RAM'de tutar ve tarama işlemlerini çok çekirdekli olarak paralel çalıştırır.*
>
> *Bu üçünün birleşimi, geleneksel bir satır bazlı veritabanının diskten milyonlarca satır okuyarak dakikalarda yaptığı işi milisaniyelere indirir."*

2. Detaylı Arka Plan

Sözlük Kodlaması (Dictionary Encoding) Nasıl Çalışır?

```
VertiPaq sıkıştırma
Ham sutun (10 milyon satir, ortalama 12 karakter = ~120 MB):
  Sehir: ["Ankara", "Istanbul", "Ankara", "Izmir", "Ankara", "Istanbul", ...]

VertiPaq iki parcaya ayirir:

  1) Sozluk (yalnizca BENZERSIZ degerler - 81 sehir):
     0 -> "Ankara"
     1 -> "Istanbul"
     2 -> "Izmir"
     ...

  2) Kodlanmis sutun (yalnizca tamsayi indeksler):
     [0, 1, 0, 2, 0, 1, ...]
     81 farkli deger icin satir basina 7 bit yeterli -> ~8,7 MB

  Ek olarak Run-Length Encoding: siralanmis veride
     [0, 0, 0, 0, 1, 1, 1] -> (deger 0, adet 4), (deger 1, adet 3)

Sonuc: ~120 MB -> ~9 MB (yaklasik 13 kat sikisma)
```

Kritik Tasarım Sonucu: Kardinalite Her Şeydir

Bu mimarinin doğrudan bir pratik sonucu vardır: sıkıştırma oranını belirleyen şey satır sayısı değil, sütundaki benzersiz değer sayısıdır (kardinalite). Düşük kardinaliteli bir sütun (şehir, ürün kategorisi, durum kodu) mükemmel sıkışır; yüksek kardinaliteli bir sütun (GUID, sipariş numarası, saniye hassasiyetli zaman damgası) neredeyse hiç sıkışmaz ve modeli tek başına şişirir.

Bu yüzden Tabular model tasarlarken şunları yaparım: Modele gerçekten kullanılmayacak sütunları hiç almam; özellikle geniş metin ve benzersiz kimlik sütunlarını. Saniye hassasiyetli datetime sütunlarını tarih ve saat olarak iki sütuna bölerim; böylece tarih sütunu düşük kardinaliteli hale gelir ve çok iyi sıkışır. Detay seviyesinde ihtiyaç yoksa fact tablosunu önceden toplulaştırırım.

Tabular ve Multidimensional Karşılaştırması

Kriter	Tabular (modern)	Multidimensional (eski)
Model yapısı	Tablolar ve ilişkiler (ilişkisel benzeri)	Küp, boyut ve hiyerarşiler
Sorgu dili	DAX (Power BI ile ortak)	MDX (öğrenmesi zor)
Depolama	VertiPaq, bellek içi kolon bazlı	MOLAP, disk tabanlı önceden hesaplanmış
Geliştirme hızı	Yüksek; ilişkisel düşünmeye yakın	Düşük; ayrı bir zihinsel model gerekir
Güçlü olduğu yer	Çoğu senaryo, Power BI entegrasyonu	Çok derin ve karmaşık hiyerarşiler

Kolon bazlı depolamanın genel mantığı ve Parquet ile ilişkisi için "OLTP ve OLAP Sistemleri" ve "Dosya Formatları: Parquet, Avro ve CSV" bölümlerine bakabilirsin; VertiPaq aynı prensibin bellek içi uygulamasıdır.

DAX: Row Context, Filter Context ve CALCULATE

> **SORU:** *DAX'ta row context ve filter context nedir? CALCULATE neden en önemli fonksiyondur? Calculated column ile measure arasındaki fark nedir?*

1. Mülakat Cevabı

> *"DAX'ın tamamı iki bağlam kavramı üzerine kuruludur ve bu ikisini karıştırmak, DAX'ta yapılan hataların büyük çoğunluğunun kaynağıdır.*
>
> *Row context, satır satır ilerleyen bir değerlendirme bağlamıdır. Calculated column yazdığımda otomatik olarak oluşur: her satır için o satırdaki değerlere erişebilirim. Ancak kritik nokta şudur: row context tek başına filtreleme yapmaz.*
>
> *Filter context, modele o an uygulanmış olan filtrelerin kümesidir. Rapordaki dilimleyiciler, satır ve sütun başlıkları ve ilişkiler üzerinden yayılan filtreler bunu oluşturur. Bir measure hesaplandığında, içinde bulunduğu hücrenin filter context'i altında çalışır; bu yüzden aynı measure her hücrede farklı sonuç verir.*
>
> *CALCULATE, DAX'taki en önemli fonksiyondur çünkü iki işi yapabilen tek fonksiyondur: mevcut filter context'i verilen koşullarla değiştirebilir ve var olan bir row context'i filter context'e dönüştürebilir. Bu ikinci davranışa context transition denir ve DAX'ın en çok yanlış anlaşılan konusudur.*
>
> *Calculated column ile measure farkı da bu bağlamlardan gelir. Calculated column, model yenilendiğinde her satır için bir kez hesaplanır ve sonuç bellekte saklanır; yani yer kaplar ve sıkışma oranını düşürür. Measure ise saklanmaz, sorgu anında o hücrenin filter context'ine göre hesaplanır. Bu yüzden mümkün olan her yerde measure tercih ederim; calculated column'u yalnızca satır seviyesinde sabit bir öznitelik gerektiğinde, örneğin dilimleyicide veya ilişkide kullanılacaksa yazarım."*

2. Detaylı Arka Plan

Klasik Tuzak: Calculated Column İçinde SUM

```DAX
-- Satis tablosunda calculated column olarak yazildi:
Yanlis Kolon = SUM(Satis[Tutar])
-- Sonuc: HER satirda tablonun TOPLAM cirosu yazar (orn. 45.000.000)
-- Neden? SUM bir filter context bekler. Row context otomatik olarak
-- filter context'e DONUSMEZ, bu yuzden hicbir filtre uygulanmaz.

-- O satirin degerini istiyorsan zaten sutunu dogrudan kullan:
Dogru Kolon = Satis[Tutar]

-- Ya da o musterinin toplamini istiyorsan CALCULATE ile
-- context transition yaptirmalisin:
Musteri Toplami = CALCULATE(SUM(Satis[Tutar]))
-- Simdi CALCULATE, mevcut satirin tum sutun degerlerini
-- filtreye cevirir ve yalnizca o satira ait toplami dondurur.
```

CALCULATE ile Filtre Değiştirme

```DAX
-- Temel measure
Toplam Satis = SUM(Satis[Tutar])

-- Filtre EKLEME: mevcut context'e yeni kosul ekler
Ankara Satis = CALCULATE([Toplam Satis], Sehir[Ad] = "Ankara")

-- Filtre KALDIRMA: ALL ile o tablonun filtrelerini yok sayar.
-- Bu, yuzde/pay hesabinin klasik kalibidir.
Urun Payi =
DIVIDE(
```

```
    [Toplam Satis],                              -- pay: mevcut filtreyle
    CALCULATE([Toplam Satis], ALL(Urun)),        -- payda: urun filtresi kalkti
    0
)

-- Filtre DEGISTIRME: gecen yilin ayni donemi
Gecen Yil Satis =
CALCULATE(
    [Toplam Satis],
    SAMEPERIODLASTYEAR(Tarih[Tarih])
)

-- Buyume orani
Buyume % =
DIVIDE([Toplam Satis] - [Gecen Yil Satis], [Gecen Yil Satis], BLANK())
```

Bağlam Karşılaştırması

Özellik	Row Context	Filter Context
Nerede oluşur	Calculated column, iterator fonksiyonlar (SUMX, FILTER)	Rapor görselleri, dilimleyiciler, ilişkiler, CALCULATE
Ne yapar	Geçerli satırın değerlerine erişim sağlar	Modeli filtreler, hangi satırların hesaba katılacağını belirler
Filtreler mi?	Hayır	Evet
Dönüşüm	CALCULATE ile filter context'e çevrilir	CALCULATE ile değiştirilir veya kaldırılır

Iterator Fonksiyonlar: SUM ve SUMX Farkı

SUM tek bir sütunu toplar. SUMX ise bir tablo üzerinde satır satır ilerleyerek (row context oluşturarak) her satır için verilen ifadeyi hesaplar ve sonuçları toplar. Satır seviyesinde çarpım gerektiren hesaplarda SUMX zorunludur.

DAX
```
-- YANLIS: once toplamlari carpar, satir bazli dogru sonuc vermez
Yanlis Ciro = SUM(Satis[Adet]) * SUM(Satis[BirimFiyat])

-- DOGRU: her satirda adet x fiyat hesaplanir, sonra toplanir
Ciro = SUMX(Satis, Satis[Adet] * Satis[BirimFiyat])
```

Mülakat Notu: Neden Measure Tercih Edilir?

Üç sebep sayabilirsin. Birincisi bellek: calculated column sonucu modelde saklanır ve genellikle yüksek kardinaliteli olduğu için kötü sıkışır. İkincisi esneklik: measure, kullanıcının uyguladığı her filtre kombinasyonuna göre yeniden hesaplanır; calculated column ise yenileme anındaki değeri sabitler. Üçüncüsü doğruluk: oran ve yüzde gibi toplanamayan (non-additive) ölçüler calculated column olarak yazıldığında ara toplamlar yanlış çıkar, measure olarak yazıldığında her seviyede doğru hesaplanır.

Storage Mode: Import ve DirectQuery

> **SORU:** *Import ve DirectQuery arasında nasıl seçim yaparsın? Composite model ve aggregations ne işe yarar?*

1. Mülakat Cevabı

> *"Import modda veri, kaynak sistemden modele kopyalanır, VertiPaq tarafından sıkıştırılır ve tamamen bellekte tutulur. Sorgular kaynak veritabanına hiç gitmez. En hızlı seçenektir ve tüm DAX yeteneklerini kullanabilirim. Bedeli, verinin bir sonraki yenilemeye kadar bayat kalması ve modelin belleğe sığması gerekliliğidir.*
>
> *DirectQuery modda veri kopyalanmaz. Kullanıcı bir görselle etkileştiğinde model, kaynak veritabanına anlık SQL sorgusu gönderir. Avantajı verinin her zaman güncel olması ve devasa hacimlerde bellek sınırına takılmamaktır. Dezavantajı ise performansın tamamen kaynak veritabanının hızına bağlı olması, bazı DAX fonksiyonlarının desteklenmemesi ve her kullanıcı etkileşiminin kaynak sisteme yük bindirmesidir.*
>
> *Kararımı üç soruyla veririm: Veri ne kadar güncel olmak zorunda? Model belleğe sığıyor mu? Kaynak sistem bu sorgu yükünü kaldırır mı? Cevaplar 'günlük yenileme yeterli' ve 'model sığıyor' ise Import seçerim; gerçek zamanlılık zorunluysa DirectQuery'ye geçerim.*
>
> *Modern çözüm genellikle composite model kurmaktır: büyük ve artık değişmeyen geçmiş veriyi Import modda tutar, yalnızca güncel ve sık değişen kısmı DirectQuery ile bağlarım. Buna aggregations özelliğini eklerim: özet seviyeyi Import'ta bellekte tutarım, kullanıcı detaya indiğinde model otomatik olarak DirectQuery'ye düşer. Böylece sorguların büyük çoğunluğu bellekten milisaniyelerde cevaplanır."*

2. Detaylı Arka Plan

Karşılaştırma Tablosu

Kriter	Import	DirectQuery
Veri nerede	Modelde, bellekte (VertiPaq)	Kaynak veritabanında kalır
Performans	Çok yüksek, tutarlı	Kaynak sistemin hızına bağlı
Güncellik	Son yenileme anına ait	Her zaman anlık
Veri hacmi	Belleğe sığmalı	Pratik olarak sınırsız
DAX desteği	Tam	Kısıtlı (bazı fonksiyonlar çalışmaz)
Kaynak sisteme yük	Yalnızca yenileme sırasında	Her kullanıcı etkileşiminde
Tipik kullanım	Günlük/saatlik yenilenen raporlar	Gerçek zamanlı izleme, çok büyük tablolar

Mülakat Senaryosu: "Rapor çok yavaş, DirectQuery kullanıyoruz. Ne yaparsın?"

Beklenen cevabın yapısı şudur: Önce darboğazın nerede olduğunu ölçerim; DirectQuery'de üretilen SQL sorgularını (Performance Analyzer veya SQL trace ile) yakalayıp kaynak veritabanında ne kadar sürdüğüne bakarım. Sorun genellikle modelde değil kaynak tarafındadır: eksik index, uygun olmayan partition veya toplulaştırılmamış detay tablo.

Çözüm sırası: Önce kaynak tarafta index ve partition optimizasyonu yaparım. Sonra veri ambarında önceden toplulaştırılmış özet tablolar oluşturur ve modeli bunlara bağlarım. Gerçek zamanlılık gerçekten gerekli değilse Import moda geçmeyi öneririm; çoğu iş biriminin "anlık" ihtiyacı sorgulandığında aslında "günde birkaç kez" çıkar. Gerçek zamanlılık zorunluysa composite model ve aggregations ile hibrit bir yapı kurarım.

Master Data Management (MDM) ve Golden Record

> **SORU:** *Master Data Management nedir? Beş farklı kaynak sistemdeki müşteri kayıtlarından tek bir "golden record" nasıl oluşturursun?*

1. Mülakat Cevabı

> *"Master Data Management, bir kurumun kritik iş varlıkları için tek doğru kaynağı (single source of truth) oluşturma ve sürdürme disiplinidir. Bu varlıklar genellikle müşteri, ürün, tedarikçi, şube ve hesap planı gibi birden fazla sistemde ortak kullanılan referans verilerdir. Üretilen tek ve güvenilir kayda golden record denir.*
>
> *Çözdüğü problem şudur: CRM'de, çağrı merkezi sisteminde, kredi başvuru sisteminde ve muhasebede aynı müşteri farklı kimliklerle, farklı yazımlarla ve farklı güncellik seviyelerinde bulunur. Sonuç olarak 'kaç müşterimiz var' sorusunun cevabı sisteme göre değişir ve hiçbir rapora güvenilmez.*
>
> *Golden record üretme sürecini dört adımda kurarım. Önce standartlaştırma: adres ve telefon formatları, büyük-küçük harf, Türkçe karakter normalizasyonu ve kısaltmaların açılması. Sonra eşleştirme (matching): TC kimlik veya vergi numarası gibi güçlü tanımlayıcılar varsa deterministik eşleştirme, yoksa isim, doğum tarihi ve adres benzerliği üzerinden olasılıksal (fuzzy) eşleştirme kullanırım. Ardından birleştirme (survivorship) kurallarını uygularım: hangi kaynağın hangi alan için otoriter olduğunu belirleyen kurallar; örneğin adres için CRM, bakiye için çekirdek bankacılık sistemi otoriterdir.*
>
> *Dördüncü ve en çok atlanan adım yönetişimdir. Otomatik eşleşmeyen veya güven skoru eşiğin altında kalan kayıtlar bir data steward'ın onayına düşer. MDM tamamen otomatik bir süreç değildir; insan onayı sürecin kalıcı bir parçasıdır. Bu yüzden MDM'i bir araç değil, bir disiplin olarak tanımlarım."*

2. Detaylı Arka Plan

Eşleştirme (Matching) Yöntemleri

- **Deterministik eşleştirme:** Güvenilir ve benzersiz bir tanımlayıcı üzerinden birebir eşleşme aranır (TC kimlik no, vergi no, IBAN). Hızlı, kesin ve açıklanabilirdir. Tek koşulu, bu alanın her kaynakta dolu ve doğru olmasıdır.
- **Olasılıksal (fuzzy) eşleştirme:** Güçlü tanımlayıcı yoksa birden fazla alanın benzerliği skorlanır. İsim için Levenshtein veya Jaro-Winkler mesafesi, adres için normalize edilmiş karşılaştırma kullanılır. Her alanın ağırlığı toplanarak bir güven skoru üretilir.
- **Eşik yönetimi:** Skor yüksek eşiğin üzerindeyse otomatik birleştirilir, düşük eşiğin altındaysa farklı kişi kabul edilir, ikisinin arasındaysa manuel incelemeye (data steward kuyruğuna) gönderilir. Bu üçlü ayrım, MDM tasarımının kalbidir.

Survivorship (Hayatta Kalma) Kuralları

```
Örnek kural seti
Ayni musteri, uc farkli kaynakta:

  Kaynak        | Ad Soyad       | Adres             | Telefon      | Guncelleme
  --------------|----------------|-------------------|--------------|------------
  CRM           | Ahmet Yilmaz   | Ankara, Cankaya   | 0532 111 ... | 2026-08-01
  Cagri Merkezi | AHMET YILMAZ   | Ankara            | 0532 111 ... | 2026-09-01
  Muhasebe      | Yilmaz, Ahmet  | Istanbul (eski)   | (bos)        | 2024-03-15

Survivorship kurallari:
  Ad Soyad -> Standartlastirilmis hali, en cok tekrar eden yazim  -> "Ahmet Yilmaz"
  Adres    -> Kaynak onceligi: CRM otoriterdir                    -> "Ankara, Cankaya"
  Telefon  -> En son guncellenen dolu deger                       -> "0532 111 ..."
  Bakiye   -> Yalnizca cekirdek bankacilik sistemi otoriterdir
```

```
GOLDEN RECORD: Ahmet Yilmaz | Ankara, Cankaya | 0532 111 ...
               + her alan icin kaynak ve guven skoru saklanir (soy izlenebilirligi)
```

Kritik tasarım detayı: Golden record üretilirken kaynak kayıtlar silinmez. Her alanın hangi kaynaktan geldiği ve hangi kuralla seçildiği saklanır. Bir yanlış birleştirme fark edildiğinde geri alabilmenin (unmerge) tek yolu budur.

MDM Mimari Yaklaşımları

- **Registry (kayıt defteri):** Veri kaynaklarda kalır, MDM yalnızca eşleştirme haritasını tutar. En az müdahaleci ve en hızlı kurulan yaklaşımdır; ancak golden record fiziksel olarak bir yerde durmaz, sorgu anında birleştirilir.
- **Consolidation (birleştirme):** Kaynaklardan veri toplanıp merkezde golden record üretilir ve analitik kullanım için sunulur. Kaynak sistemler bundan etkilenmez. Veri ambarı projelerinde en yaygın yaklaşımdır.
- **Centralized (merkezi):** Master veri yalnızca MDM sisteminde oluşturulur ve düzenlenir, kaynak sistemler onu tüketir. En temiz sonucu verir ama tüm uygulamaların değişmesini gerektirdiği için en zor uygulanandır.
- **Coexistence (birlikte yaşama):** Golden record merkezde üretilir ve kaynak sistemlere geri yazılır. Güçlüdür ancak iki yönlü senkronizasyon karmaşıklığı getirir.

Veri Yönetişimi ve Row-Level Security (RLS)

> **SORU:** *Row-Level Security nedir ve nasıl uygularsın? Bir şube müdürünün yalnızca kendi şubesinin verisini görmesini nasıl sağlarsın?*

1. Mülakat Cevabı

> *"Row-Level Security, kullanıcının kimliğine göre aynı rapor içinde farklı satırların görünmesini sağlayan yetkilendirme mekanizmasıdır. Rapor tektir, veri kullanıcıya göre süzülür. Her şube için ayrı rapor yayınlamak yerine tek rapor yayınlarım ve güvenliği modelde tanımlarım.*
>
> *SSAS Tabular ve Power BI'da bunu roller üzerinden yaparım. Bir rol tanımlar ve o role bir DAX filtre ifadesi yazarım. Kullanıcının kim olduğunu USERPRINCIPALNAME fonksiyonuyla alırım; bu değer kullanıcının Active Directory oturumundan gelir, dolayısıyla kullanıcı tarafından değiştirilemez.*
>
> *İki uygulama yaklaşımı vardır. Statik RLS'te her şube için ayrı bir rol tanımlanır; az sayıda ve sabit grup varsa işe yarar ama şube sayısı arttıkça yönetimi imkansızlaşır. Dinamik RLS'te ise modele bir yetki tablosu eklerim: hangi kullanıcının hangi şubeye erişebileceğini satır satır tutar. Filtre ifadesi bu tabloyu USERPRINCIPALNAME ile eşleştirir. Yeni bir kullanıcı eklendiğinde modeli hiç değiştirmem gerekmez, sadece yetki tablosuna satır eklerim. Bu yüzden üretimde her zaman dinamik RLS tercih ederim.*
>
> *Kritik bir uyarı da eklerim: RLS yalnızca satırları filtreler, sütunları gizlemez. Maaş gibi hassas bir sütunu belirli kullanıcılardan saklamam gerekiyorsa object-level security kullanmam veya ayrı bir model sunmam gerekir."*

2. Detaylı Arka Plan

Dinamik RLS Uygulaması

```DAX
-- 1) Modele bir yetki tablosu eklenir (veri ambarindan beslenir)
--    Yetki tablosu:
--    +--------------------------+-----------+
--    | kullanici_email          | sube_kodu |
--    +--------------------------+-----------+
--    | ahmet@banka.com          | 101       |
--    | ahmet@banka.com          | 102       |   <- iki subeye yetkili
--    | ayse@banka.com           | 205       |
--    +--------------------------+-----------+

-- 2) "Sube Kullanicisi" rolu icin Sube tablosuna filtre ifadesi:
[sube_kodu] IN
CALCULATETABLE(
    VALUES(Yetki[sube_kodu]),
    Yetki[kullanici_email] = USERPRINCIPALNAME()
)

-- 3) Filtre, iliskiler uzerinden Sube -> fact_islem yonune yayilir.
--    Kullanici raporu actiginda yalnizca yetkili subelerinin
--    satirlarini gorur; toplamlar da otomatik olarak ona gore hesaplanir.
```

RLS Tasarımında Dikkat Edilecekler

- **Filtre yönü:** RLS filtresi ilişkiler üzerinden yayılır. Boyut tablosuna konan filtrenin fact tablosuna ulaşabilmesi için ilişkinin yönünün doğru olması gerekir. Çift yönlü (bidirectional) ilişkiler RLS ile birlikte beklenmedik güvenlik açıklarına yol açabilir; bu yüzden RLS kullanılan modellerde tek yönlü ilişki tercih edilir.

- **Yetki tablosunun kaynağı:** Yetki tablosunu elle yönetmek yerine kurumsal kimlik sisteminden (Active Directory grupları veya İK sistemi) beslemek gerekir. Aksi halde çalışan ayrıldığında yetkisi modelde kalmaya devam eder.
- **Test etmek:** Modeli yayınlamadan önce "View as role" özelliğiyle farklı kullanıcı kimlikleriyle test etmek zorunludur. RLS hataları sessizdir; kullanıcı eksik veri gördüğünü fark etmeyebilir, daha kötüsü görmemesi gereken veriyi görebilir.
- **Performans:** RLS filtresi her sorguda değerlendirilir. Yetki tablosu çok büyükse veya filtre ifadesi karmaşıksa sorgu performansı düşer. Yetki tablosunu olabildiğince küçük ve kullanıcı bazında indekslenebilir tutmak gerekir.

Yönetişimin Diğer Bileşenleri

RLS, veri yönetişiminin yalnızca erişim kontrolü ayağıdır. Mülakatta konu açıldığında diğer bileşenleri de sayabilmek kapsamlı düşündüğünü gösterir:

- **Veri kataloğu ve iş sözlüğü:** Hangi verinin nerede olduğu, ne anlama geldiği ve sahibinin kim olduğu. "Aktif müşteri" tanımının kurumda tek olması bu katmanda sağlanır.
- **Veri soyağacı (lineage):** Bir raporun hangi tablolardan, hangi dönüşümlerden geçerek üretildiğini uçtan uca izleyebilmek. Etki analizi ve hata ayıklamanın temelidir.
- **Sınıflandırma ve maskeleme:** Kişisel veri içeren sütunların etiketlenmesi ve yetkisiz kullanıcılara maskelenmiş gösterilmesi (dynamic data masking). KVKK ve GDPR uyumunun teknik karşılığıdır.
- **Veri sahipliği (ownership) ve stewardship:** Her veri alanının iş tarafında bir sahibi olmalıdır. Teknik ekip veriyi taşır, tanımın doğruluğundan iş sahibi sorumludur.

ETL Sürecinde Veri Kalitesi Yönetimi

> **SORU:** *ETL sürecinde NULL değerleri, tekrar eden kayıtları ve yetim (orphan) yabancı anahtarları nasıl yönetirsin?*

1. Mülakat Cevabı

> *"Üçü de veri ambarında sessizce yanlış rapor üreten klasik problemlerdir ve hepsinin ortak çözüm ilkesi aynıdır: bozuk kaydı sessizce silme; tespit et, işaretle ve izlenebilir kıl.*
>
> *NULL yönetiminde önce NULL'ın ne anlama geldiğini ayırt ederim: veri gerçekten yok mu, bilinmiyor mu, yoksa uygulanamaz mı? Boyut özniteliklerindeki NULL'ları 'Bilinmiyor' gibi açık bir varsayılanla doldururum; çünkü raporda NULL gruplandığında kullanıcı bunu bilgi olarak değil hata olarak algılar. Ölçü sütunlarında ise NULL'ı düşünmeden sıfıra çevirmem; 'satış olmadı' ile 'satış bilgisi gelmedi' farklı şeylerdir ve ortalama hesaplarında bu fark sonucu değiştirir.*
>
> *Tekrar eden kayıtlar için önce iş anahtarını netleştiririm, sonra window fonksiyonuyla her anahtardan yalnızca en güncel kaydı alırım. Sıralamaya mutlaka belirleyici bir ikinci sütun eklerim ki sonuç her çalıştırmada aynı olsun; aksi halde hattın idempotency'si bozulur.*
>
> *Yetim yabancı anahtarlar, fact tablosunda boyutta karşılığı bulunmayan anahtarlardır. Bu satırları asla atmam; atmak ciroyu eksik gösterir ve toplamlar tutmaz. Bunun yerine boyut tablosunda -1 anahtarlı bir 'Bilinmeyen' satırı bulundururum ve eşleşmeyen fact kayıtlarını oraya bağlarım. Böylece toplamlar doğru kalır ve sorun raporda görünür hale gelir.*
>
> *Tüm bu kontrolleri hattın içine test olarak gömerim; dbt kullanıyorsam unique, not_null ve relationships testleriyle, aksi halde her yüklemeden sonra çalışan kontrol sorgularıyla ve eşik aşıldığında alarm üreten bir izleme katmanıyla."*

2. Detaylı Arka Plan

Üç Problem, Üç Kontrol Sorgusu

```sql
-- 1) TEKRAR EDEN KAYIT: is anahtari gercekten benzersiz mi? (bos donmeli)
SELECT musteri_no, COUNT(*) AS adet
FROM   stg_musteri
GROUP BY musteri_no
HAVING COUNT(*) > 1;

-- Temizleme: her anahtardan yalnizca en guncel kayit
WITH sirali AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY musteri_no
               ORDER BY guncelleme_zamani DESC,
                        kaynak_satir_no  DESC   -- belirleyici (deterministik)
           ) AS rn
    FROM stg_musteri
)
SELECT * FROM sirali WHERE rn = 1;

-- 2) YETIM YABANCI ANAHTAR: fact'te olup boyutta olmayan anahtarlar
SELECT DISTINCT f.urun_key
FROM   fact_satis f
WHERE  NOT EXISTS (SELECT 1 FROM dim_urun d WHERE d.urun_key = f.urun_key);

-- 3) NULL KONTROLU: zorunlu alanlarda bosluk var mi?
SELECT
    SUM(CASE WHEN musteri_no IS NULL THEN 1 ELSE 0 END) AS bos_musteri_no,
    SUM(CASE WHEN tutar      IS NULL THEN 1 ELSE 0 END) AS bos_tutar,
    COUNT(*)                                            AS toplam_satir
```

```
FROM stg_satis;
```

Karantina (Quarantine) Deseni

Kalite kontrolünden geçemeyen kayıtları silmek yerine ayrı bir karantina tablosuna almak, veri mühendisliğinde standart pratiktir. Böylece hiçbir veri kaybolmaz, sorunun hacmi ölçülebilir ve düzeltildikten sonra yeniden işlenebilir.

```sql
-- Temiz kayitlar Silver katmanina
INSERT INTO silver_satis
SELECT * FROM bronze_satis
WHERE musteri_no IS NOT NULL
  AND tutar >= 0;

-- Bozuk kayitlar karantinaya, SEBEBIYLE birlikte
INSERT INTO karantina_satis (kayit_id, hata_sebebi, yakalama_zamani, ham_veri)
SELECT
    kayit_id,
    CASE
        WHEN musteri_no IS NULL THEN 'musteri_no bos'
        WHEN tutar < 0          THEN 'negatif tutar'
    END,
    SYSDATETIME(),
    TO_JSON(b)
FROM bronze_satis b
WHERE musteri_no IS NULL OR tutar < 0;
```

Veri Kalitesi Boyutları

Mülakatta "veri kalitesini nasıl ölçersin?" sorusuna yapılandırılmış cevap vermek için bu altı boyut kullanılır:

Boyut	Sorduğu Soru	Örnek Kontrol
Completeness	Zorunlu alanlar dolu mu?	NULL oranı eşiğin altında mı?
Uniqueness	Tekrar eden kayıt var mı?	İş anahtarında COUNT(*) > 1 var mı?
Validity	Değerler kurallara uyuyor mu?	Durum kodu izin verilen kümede mi?
Consistency	Sistemler arası tutarlı mı?	Kaynak ve hedef toplamları eşit mi?
Timeliness	Veri zamanında geldi mi?	En son yükleme beklenen aralıkta mı?
Accuracy	Gerçeği yansıtıyor mu?	Referans veriyle örneklem karşılaştırması

Mülakat Notu: Hattı Durdurmalı mı, Devam Etmeli mi?

Bu, olgunluk gösteren bir sorudur. Cevap kuralın kritikliğine bağlıdır. Birincil anahtarın benzersizliği gibi yapısal bir kural ihlal edilmişse hattı durdururum; çünkü bozuk veri aşağı akışa yayıldığında geri almak çok daha pahalıdır. Buna karşılık "telefon numarası formatı hatalı" gibi tolere edilebilir bir kural için hattı durdurmam; kaydı işaretler, karantinaya alır ve uyarı üretirim.

Bu ayrımı dbt'de test seviyeleriyle (error ve warn) ifade ederim. Her kalite kuralının bir ciddiyet seviyesi olmalıdır; her ihlalde hattı durdurmak, kısa sürede kimsenin bakmadığı bir alarm yığını üretir.

Idempotency, partition overwrite ve backfill stratejileri için "Idempotency, Backfill ve Hata Modları"; Medallion katmanlarındaki kalite sorumlulukları için "Medallion Mimarisi (Bronze / Silver / Gold)" bölümlerine bakabilirsin.

Veritabanı Operasyonu ve Yüksek Erişilebilirlik

Transaction Isolation Seviyeleri ve Eşzamanlılık Anomalileri

> **SORU:** *Transaction isolation seviyeleri nelerdir? Dirty read, non-repeatable read ve phantom read arasındaki fark nedir? Hangi seviyeyi ne zaman seçersin?*

1. Mülakat Cevabı

> *"Isolation, ACID'in I harfidir ve eşzamanlı çalışan transaction'ların birbirinin ara durumunu ne kadar görebileceğini belirler. Standart SQL dört seviye tanımlar; her seviye belirli anomalilere izin verir.*
>
> *Read Uncommitted en gevşek seviyedir ve commit edilmemiş veriyi okumaya izin verir; yani dirty read mümkündür.*
>
> *Read Committed yalnızca commit edilmiş veriyi okur, dirty read'i engeller. Ancak aynı sorguyu aynı transaction içinde iki kez çalıştırdığında farklı sonuç alabilirsin; buna non-repeatable read denir. Çoğu veritabanının varsayılan seviyesidir.*
>
> *Repeatable Read, okunan satırların transaction boyunca değişmemesini garanti eder. Ancak koşula uyan yeni satırlar araya girebilir; buna phantom read denir.*
>
> *Serializable en katı seviyedir. Transaction'lar sanki teker teker sırayla çalışmış gibi bir sonuç garanti eder ve üç anomaliyi de engeller; bedeli eşzamanlılığın ciddi şekilde azalmasıdır.*
>
> *Seçimimi iş gereksinimine göre yaparım: bakiye güncellemesi gibi finansal kritik işlemlerde Serializable veya Snapshot, raporlama sorgularında Read Committed yeterlidir. SQL Server'da ayrıca Snapshot Isolation vardır; okuyucuların yazıcıları bloklamamasını sağladığı için raporlama yükü taşıyan OLTP sistemlerinde çok değerlidir."*

2. Detaylı Arka Plan

Hangi Seviye Hangi Anomaliye İzin Verir?

Isolation Seviyesi	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	İzin verir	İzin verir	İzin verir
Read Committed	Engeller	İzin verir	İzin verir
Repeatable Read	Engeller	Engeller	İzin verir
Serializable	Engeller	Engeller	Engeller
Snapshot (SQL Server)	Engeller	Engeller	Engeller (write skew mümkün)

Üç Anomali Somut Örnekle

```
Eşzamanlılık anomalileri
DIRTY READ  (Read Uncommitted)
  T1: UPDATE hesap SET bakiye = 0 WHERE id = 5;   -- henuz COMMIT edilmedi
  T2: SELECT bakiye FROM hesap WHERE id = 5;      -- 0 okur
  T1: ROLLBACK;                                   -- islem geri alindi
  -> T2 hicbir zaman var olmamis bir veriyi okudu ve ona gore karar verdi.

NON-REPEATABLE READ  (Read Committed)
  T1: SELECT bakiye FROM hesap WHERE id = 5;      -- 1000
  T2: UPDATE hesap SET bakiye = 500 WHERE id = 5; COMMIT;
  T1: SELECT bakiye FROM hesap WHERE id = 5;      -- 500  (ayni sorgu, farkli sonuc!)
  -> Rapor icinde toplamlar tutmaz.

PHANTOM READ  (Repeatable Read)
```

```
T1: SELECT COUNT(*) FROM siparis WHERE tutar > 1000;   -- 42
T2: INSERT INTO siparis (tutar) VALUES (5000); COMMIT;
T1: SELECT COUNT(*) FROM siparis WHERE tutar > 1000;   -- 43  (yeni satir belirdi)
-> Mevcut satirlar degismedi ama kumeye yeni uye katildi.
```

Kilit Tabanlı ve MVCC Yaklaşımı

Isolation iki farklı şekilde uygulanabilir ve bu, performans davranışını tamamen değiştirir.

- **Kilit tabanlı (pessimistic):** SQL Server'ın varsayılan davranışıdır. Okuma işlemi paylaşımlı (shared) kilit alır, yazma işlemi exclusive kilit alır ve bunlar birbiriyle çakışır. Sonuç: uzun süren bir rapor sorgusu, aynı tabloya yazmaya çalışan işlemleri bloklar.
- **MVCC (Multi-Version Concurrency Control):** PostgreSQL ve Oracle'ın varsayılanı, SQL Server'da Snapshot Isolation ile devreye girer. Veri değiştirildiğinde eski versiyon saklanır; okuyucu, transaction başladığı andaki tutarlı anlık görüntüyü okur. Sonuç: okuyucular yazıcıları, yazıcılar okuyucuları bloklamaz.

MVCC bedava değildir: eski versiyonlar bir yerde tutulmalıdır. SQL Server'da bu yük tempdb'ye biner (version store), PostgreSQL'de ise tabloda ölü satırlar birikir ve düzenli VACUUM gerekir. Uzun süren transaction'lar bu birikimi büyütür; bu yüzden "transaction'ı kısa tut" kuralı MVCC sistemlerinde ayrıca önemlidir.

Mülakat Tuzağı: NOLOCK Neden Tehlikelidir?

SQL Server dünyasında yaygın bir alışkanlık, yavaş raporları hızlandırmak için sorgulara WITH (NOLOCK) eklemektir. Bu ipucu aslında Read Uncommitted seviyesine geçmek demektir ve çoğu kişi yalnızca dirty read riskini bilir. Oysa üç ek risk daha vardır:

- Eksik satır: Sorgu tarama yaparken sayfa bölünmesi (page split) olursa, hiç güncellenmemiş bir satır sonuçtan tamamen düşebilir.
- Tekrarlanan satır: Aynı satır sonuç kümesinde iki kez görünebilir.
- Sorgu hatası: Nadiren de olsa "could not continue scan" hatasıyla sorgu tamamen başarısız olabilir.

Yani NOLOCK yalnızca "biraz eski veri okurum" demek değildir; matematiksel olarak yanlış toplam üretebilir. Doğru çözüm, blocking sorununu NOLOCK ile örtmek yerine Snapshot Isolation açmak veya raporlama yükünü okuma replikasına taşımaktır. Mülakatta bu ayrımı yapmak konuya gerçekten hakim olduğunu gösterir.

Veri Ambarında Isolation

Veri ambarında durum OLTP'den farklıdır: yazma işlemleri toplu, kontrollü ve genellikle tek bir ETL sürecine aittir. Bu yüzden çoğu ambarda Read Committed veya Snapshot yeterlidir. Asıl mesele, yükleme sırasında kullanıcıların yarım veri görmemesidir; bunu isolation seviyesiyle değil, staging tabloya yazıp partition switch veya atomik takas ile yayınlayarak çözerim.

Kilitleme, Blocking ve Deadlock

> **SORU:** *Blocking ile deadlock arasındaki fark nedir? Üretimde bir blocking sorununu nasıl teşhis eder ve çözersin?*

1. Mülakat Cevabı

> *"Blocking, bir transaction'ın ihtiyaç duyduğu kaynağın başka bir transaction tarafından kilitlenmiş olması nedeniyle beklemesidir. Normal ve geçici bir durumdur; kilidi tutan işini bitirdiğinde bekleyen devam eder. Sorun, bekleme süresi uzadığında veya zincirleme hale geldiğinde ortaya çıkar.*
>
> *Deadlock ise iki transaction'ın karşılıklı olarak birbirinin tuttuğu kilidi beklemesidir ve kendiliğinden çözülmez. Veritabanı motoru bunu tespit eder, birini kurban (victim) seçer ve geri alır. Yani blocking bir performans sorunu, deadlock ise bir tasarım sorunudur.*
>
> *Teşhise kimin kimi beklettiğine bakarak başlarım. SQL Server'da sys.dm_exec_requests ve blocking_session_id, PostgreSQL'de pg_locks ile pg_stat_activity bunu gösterir. Zincirin başındaki kök oturumu bulup ne yaptığını incelerim.*
>
> *Kök sebepler genellikle şunlardır: gereğinden uzun tutulan transaction'lar, transaction içinde kullanıcı onayı veya dış servis çağrısı beklemek, eksik index yüzünden gereğinden çok satırın kilitlenmesi ve kilitlerin farklı sıralarda alınması.*
>
> *Çözüm olarak transaction'ları mümkün olduğunca kısa tutar, kilitleri her zaman aynı sırayla alır, uygun index ekleyerek kilit kapsamını daraltır ve raporlama yükünü Snapshot Isolation'a veya okuma replikasına taşırım."*

2. Detaylı Arka Plan

Kilit Türleri ve Uyumluluk

- **Shared (S):** Okuma için alınır. Birden fazla okuyucu aynı anda alabilir; ancak exclusive kilitle çakışır.
- **Exclusive (X):** Yazma için alınır. Hiçbir kilitle uyumlu değildir; ne okuyucu ne başka yazıcı geçebilir.
- **Update (U):** Güncellemeye hazırlanırken alınır. Aynı satırı güncellemeye çalışan iki işlemin karşılıklı deadlock'a girmesini önlemek için tasarlanmıştır; yalnızca bir işlem U kilidi tutabilir.
- **Intent (IS, IX):** Alt seviyede kilit alınacağını üst seviyeye bildirir. Motorun "bu tabloda satır kilidi var mı" sorusunu tablo tarayarak değil, tek bakışta cevaplamasını sağlar.

Lock Escalation: Sessiz Blocking Kaynağı

Veritabanı motoru her satır için ayrı kilit tutmanın bellek maliyetini bir yerden sonra kabul edilemez bulur ve çok sayıda satır kilidini tek bir tablo kilidine yükseltir. SQL Server'da bu eşik yaklaşık 5000 kilittir.

Sonuç sürpriz olabilir: 10.000 satır güncelleyen bir toplu işlem, tüm tabloyu kilitler ve o tabloya dokunan her şey durur. Veri ambarı yüklemelerinde en sık karşılaşılan blocking sebeplerinden biri budur. Çözüm, büyük DML işlemlerini parçalara bölmek (batch'ler halinde 2000-3000 satır), partition switch kullanmak veya ilgili tabloda escalation davranışını sınırlamaktır.

Deadlock Senaryosu ve Çözümü

```T-SQL
-- DEADLOCK: iki islem kilitleri TERS sirada aliyor
-- Oturum A
BEGIN TRAN;
UPDATE hesap    SET bakiye = bakiye - 100 WHERE id = 1;   -- A, hesap 1'i kilitledi
UPDATE musteri  SET son_islem = GETDATE() WHERE id = 1;   -- A, musteri 1'i bekliyor

-- Oturum B (ayni anda)
BEGIN TRAN;
```

```
UPDATE musteri  SET son_islem = GETDATE() WHERE id = 1;   -- B, musteri 1'i kilitledi
UPDATE hesap    SET bakiye = bakiye + 100 WHERE id = 1;   -- B, hesap 1'i bekliyor
-- -> Karsilikli bekleme: motor birini kurban secip ROLLBACK eder.

-- COZUM: tum kod yollarinda kilitleri AYNI sirada al (once hesap, sonra musteri)
BEGIN TRAN;
UPDATE hesap    SET ... WHERE id = 1;
UPDATE musteri  SET ... WHERE id = 1;
COMMIT;
```

Deadlock'a karşı ikinci savunma hattı, uygulamanın bunu beklenen bir durum olarak ele almasıdır. Deadlock kurbanı seçilen işlem geri alınır ve hata döner; iyi yazılmış bir veri hattı bu hatayı yakalayıp kısa bir bekleme sonrası yeniden dener. Deadlock'u tamamen ortadan kaldırmak çoğu sistemde mümkün değildir, yönetilebilir hale getirmek mümkündür.

Teşhis Sorgusu

T-SQL
```
-- Kim kimi bekletiyor? (SQL Server)
SELECT
    r.session_id,
    r.blocking_session_id,
    r.wait_type,
    r.wait_time / 1000.0 AS bekleme_sn,
    DB_NAME(r.database_id) AS veritabani,
    t.text                 AS calisan_sorgu
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id <> 0
ORDER BY r.wait_time DESC;

-- Zincirin kokundeki oturumu bul: blocking_session_id'si olan
-- ama kendisi bir sey beklemeyen oturum kok sebeptir.
```

Deadlock'un teorik dört şartı (Coffman koşulları) ve işletim sistemi seviyesindeki karşılığı bu belgenin kapsamı dışındadır; burada yalnızca veritabanı seviyesindeki davranış ele alınmıştır.

Partitioning ve Büyük Tablo Yönetimi

> **SORU:** *Partitioning nedir ve ne zaman uygularsın? Partition switch neden bu kadar hızlıdır? Partition anahtarını nasıl seçersin?*

1. Mülakat Cevabı

> *"Partitioning, mantıksal olarak tek bir tablo görünen yapıyı fiziksel olarak birden fazla parçaya bölmektir. Veri ambarlarında en yaygın partition anahtarı tarihtir.*
>
> *İki temel faydası vardır. Birincisi partition pruning: sorgu WHERE koşuluna göre yalnızca ilgili partition'ları okur, geri kalanına hiç dokunmaz. Beş yıllık bir tabloda son ayı sorguladığında verinin yüzde ikisini tararsın.*
>
> *İkincisi ve veri mühendisliği açısından daha değerlisi, yükleme ve arşivleme işlemlerinin metadata seviyesinde yapılabilmesidir. Partition switch, bir partition'ı tabloya bağlamayı veya koparmayı yalnızca katalog kaydını değiştirerek yapar; veri fiziksel olarak taşınmaz. Bu yüzden milyonlarca satırlık bir partition saniyeler içinde değiştirilebilir.*
>
> *Günlük yükleme desenim şudur: veriyi ayrı bir staging tabloya yaz, kalite kontrollerini çalıştır, sonra tek bir SWITCH komutuyla ana tabloya bağla. Kullanıcılar hiçbir an yarım veri görmez ve tablo boyunca uzun süreli kilit oluşmaz. Arşivlemede aynı mekanizma ters yönde çalışır; en eski partition koparılıp arşivlenir veya silinir. Buna sliding window denir.*
>
> *Anahtar seçerken en kritik kural, çok yüksek kardinaliteli bir sütun seçmemektir. Müşteri numarasına göre partition yapmak binlerce küçük parça üretir ve yönetimi imkansızlaşır; doğru anahtar, sorgularda sürekli filtrelenen ve makul sayıda değer üreten bir sütundur."*

2. Detaylı Arka Plan

Partition Switch ile Kesintisiz Yükleme

```T-SQL
-- 1) Ayni yapida, ayni filegroup'ta bir staging tablo hazirla
--    (index'leri ve kisitlari ana tabloyla birebir ayni olmali)
TRUNCATE TABLE stg_fact_satis;

INSERT INTO stg_fact_satis (islem_tarihi, musteri_key, urun_key, tutar)
SELECT ... FROM kaynak WHERE islem_tarihi = '2026-09-05';

-- 2) Kalite kontrollerini burada calistir (henuz kullaniciya gorunmuyor)
--    Benzersizlik, NULL, yetim anahtar kontrolleri...

-- 3) Eski partition'i kopar, yenisini bagla: METADATA islemi, saniyeler surer
ALTER TABLE fact_satis
    SWITCH PARTITION 42 TO arsiv_fact_satis PARTITION 42;

ALTER TABLE stg_fact_satis
    SWITCH PARTITION 42 TO fact_satis PARTITION 42;

-- Kullanicilar hicbir an yarim veri gormedi, tablo kilitlenmedi.
```

Sliding Window (Kayan Pencere) Deseni

```Arşivleme deseni
Politika: Ambarda son 24 ay sicak tutulur, oncesi arsive tasinir.

  Her ay basinda:
    1) Yeni ay icin bos partition olustur   (SPLIT RANGE)
    2) En eski ayin partition'ini kopar     (SWITCH OUT -> arsiv tablosu)
```

```
    3) Bosalan sinirlari birlestir          (MERGE RANGE)

  Sonuc: Tablo hep 24 partition tutar, boyutu sabit kalir.
         Arsivleme DELETE ile degil metadata ile yapildigi icin
         milyarlarca satir saniyeler icinde tasinir ve log sismez.
```

Bu desenin klasik alternatifi olan DELETE FROM fact WHERE tarih < ... komutu, milyonlarca satır için transaction log'unu şişirir, saatlerce sürer ve tablo boyunca kilit üretir. Partition switch bu üç sorunun üçünü birden ortadan kaldırır.

Partition Anahtarı Seçimi

Anahtar	Uygun mu?	Gerekçe
İşlem tarihi (ay veya gün)	Evet	Sorguların çoğu tarihe göre filtreler; makul sayıda partition üretir
Yıl	Kısmen	Çok kaba; tek partition çok büyür, pruning faydası azalır
Müşteri numarası	Hayır	Çok yüksek kardinalite; binlerce küçük partition, yönetilemez
Bölge veya şube kodu	Duruma göre	Sorgular bölgeye göre filtreleniyorsa ve dağılım dengeliyse uygun
Rastgele hash	Hayır	Pruning sağlamaz; yalnızca dağıtık sistemlerde yük dengeleme amaçlı

Sık Yapılan Hatalar

- **Aşırı bölümleme (over-partitioning):** Binlerce partition, sorgu planlama süresini ve metadata yükünü artırır. Genel kural olarak partition sayısını yüzler mertebesinde tutmak, her partition'ın anlamlı bir boyuta (en az birkaç yüz MB) ulaşmasını sağlamaktır.
- **Hizalanmamış (non-aligned) index:** Index'in partition şeması tabloyla aynı değilse partition switch çalışmaz. Switch işlemi yapacaksan tüm index'lerin aligned olması zorunludur.
- **Partition anahtarını sorguda kullanmamak:** Tablo tarihe göre bölünmüş ama sorgu WHERE'de tarih filtresi içermiyorsa pruning devreye girmez ve tüm partition'lar taranır. Partitioning tek başına sihir yapmaz; sorgunun da uyumlu olması gerekir.

Data Lake Tarafındaki Karşılığı

Aynı prensip nesne depolamada klasör yapısıyla uygulanır: /satis/yil=2026/ay=09/gun=05/ şeklinde Hive tarzı bölümleme. Spark veya sorgu motoru, WHERE koşuluna bakarak yalnızca ilgili klasörleri okur. Buradaki tuzak farklıdır: ilişkisel dünyada aşırı bölümleme metadata yükü yaratırken, Lake tarafında çok sayıda küçük dosya üretir ve okuma performansını felç eder. İki dünyada da çözüm aynıdır: partition seviyesini veri hacmine göre dengelemek.

Yedekleme ve Kurtarma (RPO / RTO)

> **SORU:** *Yedekleme stratejini nasıl kurarsın? RPO ve RTO nedir? Point-in-time restore nasıl çalışır?*

1. Mülakat Cevabı

> *"Yedekleme stratejisi teknik bir tercih değil, iş kararıdır ve iki metrikle tanımlanır.*
>
> *RPO (Recovery Point Objective), ne kadar veri kaybını göze alabildiğimizdir. RPO 15 dakikaysa en fazla 15 dakikalık veriyi kaybedebiliriz demektir ve bu doğrudan transaction log yedeğinin sıklığını belirler.*
>
> *RTO (Recovery Time Objective), sistemin ne kadar sürede tekrar ayağa kalkması gerektiğidir. RTO bir saatse, kurtarma prosedürünün bu süre içinde fiilen tamamlanabiliyor olması gerekir.*
>
> *Klasik strateji üç yedek türünü birleştirir: haftalık tam (full) yedek, günlük fark (differential) yedeği ve on beş dakikada bir transaction log yedeği. Point-in-time restore şu sırayla yapılır: en son full yedek geri yüklenir, üzerine en son differential uygulanır, ardından log yedekleri istenen ana kadar sırayla oynatılır. Bu zincirin çalışabilmesi için veritabanının Full recovery model'de olması şarttır.*
>
> *En kritik nokta şudur: test edilmemiş yedek, yedek değildir. Düzenli restore tatbikatı yapmayan bir kurum, felaket anında yedeğinin bozuk veya eksik olduğunu öğrenir. Bu yüzden yalnızca yedeğin alındığını değil, geri yüklenebildiğini ve geri yükleme süresinin RTO'yu karşıladığını da ölçerim."*

2. Detaylı Arka Plan

Yedek Türleri

Tür	Ne Yedekler	Süre / Boyut	Kullanım
Full	Veritabanının tamamı	Uzun / büyük	Zincirin temeli; haftalık veya günlük
Differential	Son full'den beri değişen sayfalar	Orta	Restore adımını kısaltır
Transaction Log	Son log yedeğinden beri işlemler	Kısa / küçük	RPO'yu belirler; point-in-time restore sağlar
Copy-only	Zinciri bozmadan tam kopya	Uzun	Test ortamına veri almak için

Recovery Model: Log Yedeği Alabilmenin Ön Koşulu

- **Simple:** Transaction log otomatik olarak temizlenir; log yedeği alınamaz. Dolayısıyla point-in-time restore mümkün değildir. Yalnızca yeniden üretilebilir veri ambarı gibi ortamlarda kabul edilebilir.
- **Full:** Log yedeği alınana kadar log korunur. Point-in-time restore mümkündür. Üretim OLTP sistemlerinin standardıdır. Dikkat edilmesi gereken nokta, log yedeği alınmazsa log dosyasının sürekli büyümesi ve diski doldurmasıdır.
- **Bulk-logged:** Toplu yükleme işlemlerini minimum loglar; büyük ETL yüklemelerinde log şişmesini azaltır. Ancak bu dönemde point-in-time restore hassasiyeti kaybolur.

Point-in-Time Restore Akışı

```T-SQL
-- Senaryo: 14:35'te yanlislikla "DELETE FROM musteri" calistirildi.
-- Hedef: veritabanini 14:34'e geri dondurmek.

-- 1) En son full yedek: NORECOVERY ile (daha yuklenecek yedek var)
RESTORE DATABASE Banka FROM DISK = 'F:\yedek\full_pazar.bak'
    WITH NORECOVERY, REPLACE;

-- 2) En son differential
```

```
RESTORE DATABASE Banka FROM DISK = 'F:\yedek\diff_cuma.bak'
    WITH NORECOVERY;

-- 3) Differential'dan sonraki TUM log yedekleri, SIRAYLA
RESTORE LOG Banka FROM DISK = 'F:\yedek\log_1400.trn' WITH NORECOVERY;
RESTORE LOG Banka FROM DISK = 'F:\yedek\log_1415.trn' WITH NORECOVERY;

-- 4) Son log: istenen ana kadar oynat ve veritabanini ac
RESTORE LOG Banka FROM DISK = 'F:\yedek\log_1430.trn'
    WITH STOPAT = '2026-09-05 14:34:00', RECOVERY;
```

3-2-1 Kuralı ve Yedeğin Kendisini Korumak

Yaygın kabul gören kural şudur: verinin en az 3 kopyası olsun, 2 farklı medya türünde tutulsun, 1 kopya farklı bir fiziksel lokasyonda bulunsun. Fidye yazılımı (ransomware) tehdidi bu kurala bir madde daha ekletti: en az bir kopya değiştirilemez (immutable) veya çevrimdışı olmalıdır. Aksi halde saldırgan sistemi şifrelerken yedekleri de şifreler ve yedek hiçbir işe yaramaz.

Ayrıca yedeklerin şifrelenmesi gerekir. Şifrelenmemiş bir yedek dosyası, veritabanının tüm erişim kontrollerini baypas eden bir kopyadır; çalındığında başka bir sunucuda açılabilir.

Veri Ambarında Yedekleme Farklı Düşünülür

Mülakatta değer katan bir nüans şudur: Bir OLTP veritabanındaki veri kaybı geri getirilemez, çünkü kaynağı yoktur. Ancak veri ambarındaki Gold ve Silver katmanları, Bronze katmanından yeniden üretilebilir. Bu yüzden ambar için soru "yedeğim var mı" değil, "yeniden üretebilir miyim ve bu ne kadar sürer" sorusudur.

Bu bakışla ambarda yedekleme maliyetini düşürebilirim: Bronze katmanını ve kaynak sistemleri titizlikle korurum, türetilmiş katmanlar için ise yeniden üretim süresini ölçer ve bunu RTO olarak kabul ederim. Ancak bu yaklaşımın bir şartı vardır: dönüşüm kodunun sürüm kontrolünde olması ve hattın gerçekten idempotent olması.

Yüksek Erişilebilirlik ve Felaket Kurtarma (HA / DR)

> **SORU:** *HA ile DR arasındaki fark nedir? Senkron ve asenkron replikasyon arasında nasıl seçim yaparsın?*

1. Mülakat Cevabı

> *"Yüksek erişilebilirlik (HA), tek bir bileşenin arızasında sistemin çalışmaya devam etmesidir; genellikle aynı veri merkezi içinde ve otomatik failover ile. Felaket kurtarma (DR) ise tüm veri merkezinin kaybı gibi büyük olaylara karşı, coğrafi olarak uzak bir bölgede yapılan hazırlıktır. HA saniyeler ve dakikalar ölçeğinde, DR ise dakikalar ve saatler ölçeğinde konuşulur.*
>
> *SQL Server'da en yaygın HA çözümü Always On Availability Group'tur. Bir birincil (primary) replika yazma trafiğini alır, bir veya daha fazla ikincil (secondary) replika bu değişiklikleri alır. İkincil replikalar okumaya açılabilir; bu, raporlama yükünü üretim veritabanından ayırmanın en etkili yollarından biridir.*
>
> *Senkron replikasyonda birincil, işlemi ikincile yazıldığı teyit edilmeden commit etmez. Veri kaybı sıfırdır, yani RPO sıfırdır; ancak her commit ağ gecikmesini beklediği için yazma performansı düşer ve bu yüzden yalnızca düşük gecikmeli, yakın mesafeli replikalar için uygundur.*
>
> *Asenkron replikasyonda birincil beklemez. Performans korunur ama failover anında son birkaç işlem kaybolabilir. Uzak bölgeye replikasyon pratikte ancak asenkron yapılabilir.*
>
> *Pratikte ikisini birlikte kurarım: aynı veri merkezindeki ikincil replika senkron (otomatik failover ve sıfır veri kaybı için), uzak bölgedeki replika asenkron (felaket senaryosu için)."*

2. Detaylı Arka Plan

HA ve DR Karşılaştırması

Kriter	Yüksek Erişilebilirlik (HA)	Felaket Kurtarma (DR)
Koruduğu senaryo	Sunucu, disk veya servis arızası	Veri merkezi kaybı, doğal afet, siber saldırı
Konum	Aynı veri merkezi veya kampüs	Coğrafi olarak uzak bölge
Devreye girme	Genelde otomatik (failover)	Genelde manuel karar ve prosedür
Hedef süre (RTO)	Saniyeler - dakikalar	Dakikalar - saatler
Replikasyon	Senkron (RPO = 0)	Asenkron (RPO > 0)
Maliyet	Orta	Yüksek (ikinci lokasyon)

Senkron ve Asenkron Seçimi

```
Tipik kurulum
                SENKRON                      ASENKRON
             (RPO = 0)                   (RPO = saniyeler/dakikalar)
                 |                              |
  [Birincil] --> [Ikincil-1]  ..............> [Ikincil-2]
   Ankara DC      Ankara DC                     Istanbul DC
                  |                              |
       otomatik failover              manuel failover (felaket ani)
       okuma replikasi olarak
       raporlamaya acilabilir

 Karar kurali:
   - Ayni sehir, dusuk gecikme, sifir veri kaybi sart  -> SENKRON
   - Uzak bolge, yuksek gecikme, performans korunmali  -> ASENKRON
```

Diğer HA Yöntemleri

- **Failover Cluster Instance (FCI):** Sunucu düzeyinde yedeklilik sağlar, ancak depolama ortaktır. Yani disk arızasına karşı korumaz; paylaşımlı depolamanın kendisi tek hata noktasıdır.
- **Log Shipping:** Log yedeklerinin düzenli olarak ikinci sunucuya taşınıp uygulanmasıdır. Basit ve ucuzdur, otomatik failover yoktur; gecikme dakikalar mertebesindedir. Küçük ölçekli DR için hâlâ makul bir çözümdür.
- **Replication (transactional):** Tablo veya nesne bazında seçici kopyalama sağlar. HA amaçlı değil, veri dağıtımı ve raporlama amaçlı kullanılır.
- **Bulut yönetilen servisler:** Azure SQL ve benzeri servislerde HA altyapı tarafından sağlanır; senin sorumluluğun coğrafi replikasyonu yapılandırmak ve failover senaryosunu test etmektir.

Mülakat Notu: Failover Testi Yapılmamış DR, DR Değildir

Yedeklemede olduğu gibi burada da en kritik nokta tatbikattır. Kağıt üzerinde kurulmuş ama hiç denenmemiş bir DR planı, gerçek olayda genellikle çalışmaz; çünkü unutulan bağlantı dizeleri, DNS kayıtları, uygulama tarafındaki sabit IP'ler veya eksik yetkiler ancak deneyince ortaya çıkar.

Bu yüzden düzenli failover tatbikatı yapar, geçiş süresini ölçer ve RTO ile karşılaştırırım. Ayrıca geri dönüş (failback) prosedürünü de test ederim; birçok kurum felakete hazırlanırken normale dönmeyi planlamayı unutur.

Veri Ambarı İçin HA Gerekli mi?

Genellikle OLTP kadar katı değildir. Ambar birkaç saat erişilemez olduğunda müşteri etkilenmez, yalnızca raporlar gecikir; dolayısıyla SLA daha gevşektir ve senkron replikasyonun maliyeti çoğu zaman gereksizdir. Ancak bir istisna vardır: ambar operasyonel bir karar sürecini besliyorsa (örneğin gerçek zamanlı limit veya risk kontrolü), o zaman OLTP seviyesinde HA gerekir. Bu ayrımı yapabilmek, SLA'yi teknolojiye değil iş etkisine göre belirlediğini gösterir.

Veri Güvenliği ve Uyum

Şifreleme: At-Rest, In-Transit ve TDE

> **SORU:** *Veri hangi katmanlarda şifrelenir? TDE tam olarak neyi korur, neyi korumaz?*

1. Mülakat Cevabı

> *"Şifrelemeyi üç ayrı katmanda düşünürüm ve her biri farklı bir tehdide karşı koruma sağlar.*
>
> *Aktarım halinde şifreleme (in transit): İstemci ile veritabanı arasındaki ağ trafiği TLS ile şifrelenir. Ağı dinleyen birinin veriyi okumasını engeller.*
>
> *Durağan halde şifreleme (at rest): Diskte duran veri dosyaları şifrelenir. SQL Server'daki karşılığı TDE'dir (Transparent Data Encryption); veri dosyaları, log dosyaları ve yedekler şifreli olarak yazılır. Uygulama tarafında hiçbir kod değişikliği gerektirmediği için "transparent" adını alır.*
>
> *Sütun veya alan seviyesinde şifreleme: Yalnızca belirli hassas sütunlar ayrıca şifrelenir. SQL Server'da Always Encrypted bunu sağlar; anahtar istemci tarafında tutulduğu için veritabanı yöneticisi bile o sütunun içeriğini göremez.*
>
> *TDE hakkında en kritik nokta şudur: TDE yalnızca fiziksel medyayı korur. Birisi diski veya yedek dosyasını çalarsa açamaz. Ancak yetkili bir kullanıcı SELECT çektiğinde veri şifresi çözülmüş halde gelir. Yani TDE; yetki ihlaline, çalınmış bir hesaba veya SQL injection'a karşı hiçbir koruma sağlamaz. Bunları karıştırmak çok yaygın bir hatadır.*
>
> *Bu yüzden şifrelemeyi tek başına bir çözüm olarak değil; erişim kontrolü, maskeleme ve denetim ile birlikte katmanlı bir savunmanın parçası olarak kurgularım."*

2. Detaylı Arka Plan

Hangi Katman Neyi Korur?

Katman	Koruduğu Tehdit	Korumadığı Durum
TLS (in transit)	Ağ dinleme, ortadaki adam saldırısı	Uç noktalardaki veri; sunucuya erişen saldırgan
TDE (at rest)	Çalınan disk, kopyalanan yedek dosyası	Yetkili kullanıcı sorgusu, SQL injection, yetki ihlali
Always Encrypted (sütun)	DBA dahil sunucu tarafındaki herkes	İstemci makinesi ele geçirilirse; ayrıca sorgu esnekliği kısıtlanır
Yedek şifreleme	Yedeğin başka sunucuda açılması	Anahtarla birlikte çalınan yedek

Anahtar Yönetimi: Asıl Zor Kısım

Şifrelemenin teknik kısmı kolaydır; zor olan anahtarların yönetimidir. SQL Server'da hiyerarşik bir yapı vardır: Service Master Key, Database Master Key, sertifika ve en altta veriyi fiilen şifreleyen Database Encryption Key. Kurumsal ortamlarda kök anahtar genellikle bir HSM cihazında veya Azure Key Vault gibi bir anahtar kasasında tutulur.

İki kural kritiktir. Birincisi, anahtarlar şifreledikleri veriyle aynı yerde saklanmamalıdır; aynı sunucuda duran anahtar, kapının yanına asılmış anahtar gibidir. İkincisi, anahtarların ayrıca yedeklenmesi ve bu yedeğin güvenli saklanması gerekir.

Mülakat Tuzağı: TDE'li Yedek Başka Sunucuda Açılmaz

Çok sık yaşanan gerçek bir olaydır: TDE açık bir veritabanının yedeği alınır, felaket anında başka bir sunucuya restore edilmek istenir ve işlem başarısız olur. Sebep, yedeğin şifreli olması ve hedef sunucuda şifreyi çözecek sertifikanın bulunmamasıdır.

Doğru pratik, TDE sertifikasını ve özel anahtarını veritabanından ayrı olarak yedeklemek ve bu yedeği güvenli bir yerde saklamaktır. Sertifika kaybedilirse yedekler kalıcı olarak kullanılamaz hale gelir; hiçbir kurtarma yolu yoktur. Bu detayı mülakatta söylemek, konuyu teoriden değil pratikten bildiğini gösterir.

Performans Etkisi

TDE'nin CPU maliyeti modern donanımda genellikle yüzde beş civarındadır ve çoğu sistemde kabul edilebilir. Asıl dikkat edilmesi gereken yan etki, şifrelenmiş verinin sıkışmamasıdır: TDE açıldığında yedek sıkıştırma oranı ciddi şekilde düşer ve yedek boyutları büyür. Always Encrypted'in etkisi ise daha büyüktür; şifreli sütunda aralık sorgusu, LIKE araması veya sıralama yapılamaz, yalnızca eşitlik karşılaştırması mümkündür (deterministik şifreleme kullanılıyorsa). Bu yüzden şifrelenecek sütunları seçerken sorgu ihtiyaçlarını önceden bilmek gerekir.

Veri Maskeleme ve Anonimleştirme

> **SORU:** *Üretim verisini test ortamına nasıl taşırsın? Maskeleme, takma adlaştırma ve anonimleştirme arasındaki fark nedir?*

1. Mülakat Cevabı

> *"Üretim verisini test ortamına olduğu gibi kopyalamak, bankacılıkta ve KVKK kapsamında en ciddi ihlallerden biridir. Veri, ortama inmeden önce dönüştürülmelidir ve burada birbirine karıştırılan üç farklı kavram vardır.*
>
> *Maskeleme (masking), veriyi gösterirken gizlemektir. Dinamik veri maskeleme, yetkisiz kullanıcıya kart numarasını XXXX-XXXX-XXXX-1234 olarak gösterir. Ancak veri diskte hâlâ açık haldedir; değişen yalnızca görüntüdür.*
>
> *Takma adlaştırma (pseudonymization), gerçek değeri tutarlı bir takma değerle değiştirmektir. Aynı müşteri her tabloda aynı takma kimliği alır, dolayısıyla ilişkiler ve JOIN'ler bozulmaz. Ancak eşleştirme tablosu saklandığı için işlem geri döndürülebilir; bu nedenle KVKK açısından veri hâlâ kişisel veri sayılır.*
>
> *Anonimleştirme, kimliği geri döndürülemez şekilde yok etmektir. Doğru yapıldığında veri artık kişisel veri sayılmaz ve mevzuatın kapsamı dışına çıkar. Zor olan kısmı şudur: yalnızca isim ve TC kimlik numarasını silmek yeterli değildir. Doğum tarihi, posta kodu ve cinsiyet birleşimi bir nüfusun büyük bölümünü tekil olarak tanımlayabilir; buna yeniden kimliklendirme (re-identification) riski denir ve k-anonimlik gibi ölçütlerle değerlendirilir.*
>
> *Test ortamı için tercihim, referans bütünlüğünü koruyan tutarlı takma adlaştırmadır; çünkü tamamen rastgele veri üretmek test senaryolarını gerçekçi olmaktan çıkarır ve hatalar üretimde ortaya çıkar."*

2. Detaylı Arka Plan

Üç Kavramın Karşılaştırması

Yöntem	Veri Diskte Nasıl Durur	Geri Döndürülebilir mi?	Tipik Kullanım
Maskeleme (dinamik)	Açık; yalnızca görüntü gizlenir	Gerekmez (veri zaten yerinde)	Üretimde yetkisiz kullanıcıya gösterim
Takma adlaştırma	Takma değerle değiştirilmiş	Evet (eşleştirme tablosuyla)	Test ortamı, analitik
Anonimleştirme	Geri dönülemez şekilde bozulmuş	Hayır	Dış paylaşım, araştırma, uzun süreli arşiv

Tutarlı Takma Adlaştırma: Referans Bütünlüğünü Korumak

Test ortamındaki en yaygın hata, her tabloyu bağımsız olarak rastgele maskelemektir. Sonuçta müşteri tablosundaki kimlik ile sipariş tablosundaki kimlik tutmaz, JOIN'ler boş döner ve test ortamı işe yaramaz hale gelir. Çözüm, deterministik bir dönüşüm kullanmaktır: aynı girdi her zaman aynı çıktıyı üretir.

```SQL
-- Tutarli takma adlastirma: ayni musteri_no her tabloda ayni takma degeri alir.
-- Salt (gizli tuz) kullanmak sart: aksi halde saldirgan hash'i deneyerek geri cozer.

-- musteri tablosu
SELECT
    CONVERT(VARCHAR(64),
        HASHBYTES('SHA2_256', CONCAT(@salt, musteri_no)), 2)  AS musteri_no,
    'Musteri_' + CAST(ROW_NUMBER() OVER (ORDER BY musteri_no)
        AS VARCHAR)                                           AS ad_soyad,
    LEFT(email, 1) + '****@ornek.com'                         AS email,
```

```
    DATEFROMPARTS(YEAR(dogum_tarihi), 1, 1)                  AS dogum_tarihi
FROM prod.musteri;

-- siparis tablosu: AYNI donusum uygulanir -> JOIN'ler calismaya devam eder
SELECT
    siparis_id,
    CONVERT(VARCHAR(64),
        HASHBYTES('SHA2_256', CONCAT(@salt, musteri_no)), 2)  AS musteri_no,
    tutar
FROM prod.siparis;
```

Dikkat edilecek nokta: Tuz (salt) olmadan yapılan hash geri çözülebilir. TC kimlik numarası gibi format ve uzunluğu belli bir alanda saldırgan tüm olasılıkları deneyerek (brute force) eşleşme bulabilir. Tuz gizli tutulmalı ve test ortamında bulunmamalıdır.

Doğum tarihi örneğinde gördüğün gibi, tarihi yılın ilk gününe yuvarlamak da bilinçli bir tercihtir: yaş analizleri hâlâ yapılabilir ama kişi tekilleştirilemez. Buna genelleme (generalization) denir ve k-anonimliğin temel araçlarından biridir.

Mülakat Tuzağı: Dinamik Maskeleme Bir Güvenlik Sınırı Değildir

SQL Server'ın Dynamic Data Masking özelliği kullanışlıdır ama sınırları iyi bilinmelidir: veri diskte açık durur ve maskeleme yalnızca gösterim katmanındadır. Kullanıcı SELECT yetkisine sahipse, WHERE koşullarıyla deneme yaparak maskelenmiş değeri çıkarabilir.

```SQL
-- maas sutunu maskelenmis; kullanici yalnizca "0" goruyor.
-- Ancak WHERE ile deneyerek gercek degeri bulabilir:
SELECT COUNT(*) FROM calisan WHERE maas > 50000;   -- 12
SELECT COUNT(*) FROM calisan WHERE maas > 75000;   -- 3
SELECT COUNT(*) FROM calisan WHERE maas > 90000;   -- 1  -> ikili arama ile daralt
-- Maskeleme goruntuyu gizler, BILGIYI gizlemez.
```

Bu yüzden dinamik maskeleme, gerçek yetkilendirmenin yerine değil, tamamlayıcısı olarak kullanılmalıdır. Hassas sütuna erişimin gerçekten engellenmesi gerekiyorsa çözüm sütun seviyesinde yetki (DENY SELECT ON sütun) veya ayrı bir görünüm sunmaktır.

Alternatif: Sentetik Veri

Bazı durumlarda üretim verisini hiç taşımamak en temiz çözümdür. Sentetik veri üretimi, üretim verisinin istatistiksel dağılımını taklit eden ama gerçek kişilere ait olmayan kayıtlar üretir. Avantajı KVKK riskini tamamen ortadan kaldırmasıdır; dezavantajı, üretimdeki gerçek veri kirliliğini (beklenmedik karakterler, bozuk formatlar, uç değerler) yansıtmamasıdır. Pratikte yaygın yaklaşım ikisini birleştirmektir: hacim testleri için sentetik veri, davranış testleri için takma adlaştırılmış gerçek veri.

Erişim Yönetimi ve Audit (Denetim İzi)

> **SORU:** *Veritabanında yetkilendirmeyi nasıl kurgularsın? Audit log'da ne tutulmalı ve neden değiştirilemez olması gerekir?*

1. Mülakat Cevabı

> *"Yetkilendirmede üç ilkeye bağlı kalırım.*
>
> *En az yetki (least privilege): Kimseye işini yapmak için gereğinden fazla yetki verilmez. ETL servis hesabının yalnızca ihtiyaç duyduğu şemalara, yalnızca ihtiyaç duyduğu işlemler için yetkisi olur; kimse rutin işini db_owner ile yapmaz.*
>
> *Rol tabanlı erişim (RBAC): Yetki kişiye değil role verilir, kişi role atanır. Böylece bir çalışan ayrıldığında veya görev değiştirdiğinde tek yerden yönetilir ve kimin neye erişebildiği denetlenebilir kalır.*
>
> *Görevler ayrılığı (segregation of duties): Veriyi değiştirebilen ile denetim kayıtlarını yönetebilen aynı kişi olmamalıdır. Bankacılıkta bu yalnızca iyi bir pratik değil, çoğu zaman mevzuat gereğidir.*
>
> *Uygulamada kullanıcıya doğrudan tablo yetkisi vermek yerine görünüm (view) veya stored procedure üzerinden erişim veririm; böylece hangi satırın ve sütunun görüleceğini kontrol ederim.*
>
> *Audit tarafında minimum şunları kaydederim: kim (kullanıcı ve kaynak IP), ne zaman, hangi nesneye, hangi işlemi yaptı ve başarılı oldu mu. Özellikle yetki değişiklikleri, şema değişiklikleri, hassas tablolara erişim ve başarısız giriş denemeleri mutlaka kayıt altına alınır.*
>
> *En kritik nokta audit kaydının değiştirilemez (immutable) olmasıdır. Denetim kaydı, denetlediği sistemin yöneticisinin silebileceği bir yerde duruyorsa hiçbir kanıt değeri taşımaz."*

2. Detaylı Arka Plan

Rol Tabanlı Yetkilendirme Örneği

```T-SQL
-- Roller olusturulur (kisi bazli yetki VERILMEZ)
CREATE ROLE rol_etl_yazar;
CREATE ROLE rol_analist_okur;
CREATE ROLE rol_rapor_gelistirici;

-- ETL servis hesabi: yalnizca staging ve ambar semalarina yazabilir
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::staging TO rol_etl_yazar;
GRANT SELECT, INSERT, UPDATE          ON SCHEMA::dwh     TO rol_etl_yazar;

-- Analist: yalnizca sunum katmanini okur, ham katmani goremez
GRANT SELECT ON SCHEMA::mart TO rol_analist_okur;
DENY  SELECT ON SCHEMA::bronze TO rol_analist_okur;

-- Hassas sutun: rol duzeyinde acikca reddedilir
DENY SELECT ON dwh.dim_musteri(tc_kimlik_no) TO rol_analist_okur;

-- Kisiler role atanir
ALTER ROLE rol_analist_okur ADD MEMBER [KURUM\ayse.demir];
ALTER ROLE rol_etl_yazar    ADD MEMBER [KURUM\svc_etl_prod];
```

Audit'te Ne Kaydedilmeli?

Olay Türü	Neden Kritik
Yetki değişiklikleri (GRANT / ALTER ROLE)	Yetki yükseltme saldırısının ilk adımıdır; en yüksek öncelikli kayıttır

Olay Türü	Neden Kritik
Şema değişiklikleri (DDL)	Beklenmedik tablo veya sütun değişikliği hem güvenlik hem veri bütünlüğü sinyalidir
Hassas tabloya erişim (SELECT)	Kişisel veriye kimin, ne zaman eriştiğinin kaydı KVKK açısından zorunludur
Başarısız giriş denemeleri	Kaba kuvvet saldırısının erken göstergesidir
Toplu veri okuma / dışa aktarma	Veri sızdırma (exfiltration) tespitinin temel sinyalidir
Ayrıcalıklı hesap kullanımı	Break-glass hesapların her kullanımı gerekçelendirilmelidir

Audit Log Neden Ayrı Yerde Durmalı?

Bir saldırganın veya kötü niyetli bir iç kullanıcının ilk yapacağı şey izini silmektir. Audit kayıtları denetledikleri veritabanının içinde ve DBA'in silme yetkisine sahip olduğu bir tabloda duruyorsa, kayıtların kanıt değeri yoktur.

Doğru kurgu şudur: audit kayıtları ayrı bir sunucuya, yalnızca ekleme yapılabilen (append-only) veya değiştirilemez (WORM) bir depoya yazılır; veritabanı yöneticisinin bu depoda silme yetkisi bulunmaz ve erişimi ayrı bir ekip denetler. Bulut ortamlarında bunun karşılığı, nesne depolamada immutability politikası tanımlanmış ayrı bir hesaptır.

Bir başka önemli detay: audit kayıtlarının kendisi de kişisel veri içerebilir. Sorgu metinlerini olduğu gibi loglarsan, WHERE koşullarında TC kimlik numaraları veya hesap numaraları log dosyasına düşer. Bu yüzden audit kayıtları da erişim kontrolüne ve saklama süresi politikasına tabidir.

Ayrıcalıklı Hesaplar ve Break-Glass

Üretim ortamında kimsenin sürekli yönetici yetkisiyle dolaşmaması gerekir. Standart yaklaşım, olağanüstü durumlar için "break-glass" adı verilen özel bir hesap tanımlamaktır: şifresi kasada tutulur, kullanımı otomatik alarm üretir, her kullanım sonrası şifresi değiştirilir ve gerekçesi kayıt altına alınır. Böylece acil müdahale mümkün kalır ama rutin hale gelmez.

KVKK Uyumu, Veri Saklama ve İmha

> **SORU:** *Veri ambarında KVKK açısından hangi sorumlulukların var? Bir kişi "verilerimi silin" dediğinde teknik olarak ne yaparsın?*

1. Mülakat Cevabı

> *"Veri mühendisi olarak KVKK'nın teknik karşılığını dört başlıkta ele alırım.*
>
> *Veri envanteri ve sınıflandırma: Hangi tabloda hangi kişisel veri var, nereden geldi ve nereye akıyor bilinmelidir. Bunu veri kataloğu ve soyağacı (lineage) ile sağlarım; çünkü varlığından haberdar olmadığın veriyi koruyamaz, silemez ve raporlayamazsın.*
>
> *Amaçla sınırlılık ve veri minimizasyonu: Ambara ihtiyaç duyulmayan kişisel veriyi hiç almam. En güvenli kişisel veri, hiç toplanmamış olandır. Bu, mühendislik tarafında en çok atlanan ama en etkili kontroldür.*
>
> *Saklama süresi ve imha: Her veri kümesinin tanımlı bir saklama süresi olmalı ve süre dolduğunda otomatik olarak imha edilmelidir. Bunu partition bazlı arşivleme ve silme işleriyle otomatikleştiririm; elle yürütülen imha süreçleri er ya da geç unutulur.*
>
> *Silme ve düzeltme talepleri: Veri ambarında teknik olarak en zorlu kısım budur, çünkü aynı kişinin verisi ham katmanda, ara katmanlarda, yedeklerde ve BI önbelleklerinde kopyalanmış haldedir.*
>
> *Pratik yaklaşımım şudur: Silme talebini merkezi bir talep tablosuna kaydeder ve hattın her katmanında bu listeyi uygulayan bir adım bulundururum. Ham katmanda satır seviyesinde silme yapabilmek için Delta Lake veya Iceberg gibi bir tablo formatı kullanırım; klasik Parquet dosyalarında satır silmek mümkün olmadığı için bu mimari tercih doğrudan bir uyum gereksinimidir."*

2. Detaylı Arka Plan

KVKK Gereksinimlerinin Teknik Karşılıkları

Mevzuat Gereksinimi	Teknik Karşılığı
Veri envanteri tutma	Veri kataloğu, sütun seviyesinde sınıflandırma etiketleri, lineage
Amaçla sınırlılık	Kaynaktan yalnızca gerekli alanları çekme, minimizasyon
Veri güvenliği	Şifreleme (TDE, TLS), RBAC, Row-Level Security, maskeleme
Saklama süresi ve imha	Partition bazlı otomatik arşivleme ve silme işleri
Silme ve düzeltme hakkı	Merkezi talep tablosu + her katmanda uygulama adımı; Delta/Iceberg
Hesap verebilirlik	Değiştirilemez audit log, erişim kayıtları
İhlal bildirimi	Anomali tespiti ve alarm; olay müdahale prosedürü

Silme Talebinin Katmanlar Boyunca Uygulanması

```
Silme akışı
[1] Talep alinir  ->  silme_talebi tablosuna yazilir
                      (musteri_no, talep_tarihi, durum, tamamlanma_tarihi)
                             |
                             v
[2] GOLD / MART katmani   -> Kisi bazli satirlar silinir veya anonimlestirilir.
                             Toplulastirilmis metriklerde birey yoksa dokunulmaz.
                             |
                             v
[3] SILVER katmani        -> Kisiye ait satirlar silinir (Delta/Iceberg DELETE).
                             |
                             v
[4] BRONZE (ham) katmani  -> Tablo formati sayesinde satir seviyesinde silinir.
                             Klasik Parquet ise: ilgili partition yeniden yazilir.
```

```
                              |
                              v
[5] YEDEKLER              -> Aninda silme genelde mumkun degildir.
                             Saklama suresi dolunca yedegin tamamen imha
                             edildigi belgelenir. (Regulator bu yaklasimi kabul eder.)
                              |
                              v
[6] BI onbellegi / SSAS   -> Model yeniden islenir (full refresh).
                              |
                              v
[7] Talep kapatilir       -> Tamamlanma tarihi ve uygulanan katmanlar loglanir.
```

Bu akıştaki en kritik tasarım kararı, silmeyi tek seferlik manuel bir iş olarak değil, hattın kalıcı bir adımı olarak kurgulamaktır. Aksi halde silinen kişinin verisi bir sonraki tam yükleme (full refresh) ile kaynaktan geri gelir ve talep fiilen uygulanmamış olur. Mülakatta bu detayı söylemek, konuyu gerçekten uçtan uca düşündüğünü gösterir.

İleri Teknik: Crypto-Shredding

Yedeklerden ve değiştirilemez arşivlerden veri silmenin pratik olarak imkansız olduğu durumlarda kullanılan zarif bir yöntemdir. Her kişinin verisi, o kişiye özel bir anahtarla şifrelenerek saklanır. Silme talebi geldiğinde verinin kendisine dokunulmaz; yalnızca o kişinin anahtarı imha edilir.

Anahtar yok edildiği anda veri matematiksel olarak geri getirilemez hale gelir ve pratikte silinmiş sayılır. Yedeklerde şifreli bloklar durmaya devam eder ama hiçbir şekilde çözülemez. Bu yaklaşımın maliyeti, kişi başına anahtar yönetiminin karmaşıklığıdır; bu yüzden genellikle yalnızca en hassas veri kümelerinde tercih edilir.

Veri İhlali Anında Sorumluluk

KVKK, veri ihlalinin öğrenilmesinden itibaren en kısa sürede ve 72 saat içinde Kurul'a bildirilmesini gerektirir. Mühendislik tarafında bunun anlamı şudur: ihlali fark edecek mekanizmaya sahip olmak. Kimsenin bakmadığı bir log dosyası, ihlali 72 saat içinde tespit etmeni sağlamaz.

Bu yüzden pratikte şu kontrolleri kurarım: alışılmadık hacimde veri okuma veya dışa aktarma alarmı, mesai dışı erişim tespiti, hassas tablolara yetkisiz erişim denemeleri ve yetki yükseltme olaylarında anlık bildirim. Ayrıca hangi kişisel verinin etkilendiğini hızlıca çıkarabilmek için veri envanterinin güncel olması şarttır; ihlal anında "hangi veriler sızdı" sorusuna saatler içinde cevap verebilmek gerekir.

Row-Level Security, veri yönetişimi, veri kataloğu ve soyağacı konuları için "Veri Yönetişimi ve Row-Level Security (RLS)"; veri kalitesi kontrolleri için "ETL Sürecinde Veri Kalitesi Yönetimi" bölümlerine bakabilirsin.

Ek: Mülakat Öncesi Hızlı Tekrar Listesi

Mülakattan hemen önce göz gezdirmek için hazırlanmış özet. Her madde, dokümanda detaylandırılan bir kavramın tek cümlelik hatırlatıcısıdır.

SQL: Mutlaka Ezberde Olması Gerekenler

- Çalıştırma sırası: FROM > WHERE > GROUP BY > HAVING > SELECT > DISTINCT > ORDER BY > LIMIT.
- WHERE satırları gruplamadan önce, HAVING grupları gruplamadan sonra filtreler. Mümkünse WHERE kullan.
- LEFT JOIN'de sağ tabloya ait koşulu WHERE'e yazarsan JOIN sessizce INNER JOIN'e döner; koşul ON içine yazılmalı.
- NOT IN listesinde tek bir NULL varsa sorgu hiç satır döndürmez. NOT EXISTS kullan.
- COUNT(*) tüm satırları, COUNT(sütun) NULL olmayanları sayar. AVG de NULL'ları yok sayar.
- UNION tekrarları eler ve maliyetlidir; gerekmiyorsa UNION ALL kullan.
- ROW_NUMBER her satıra benzersiz numara verir (1,2,3,4); RANK boşluk bırakır (1,2,2,4); DENSE_RANK bırakmaz (1,2,2,3).
- Deduplication kalıbı: ROW_NUMBER() OVER (PARTITION BY anahtar ORDER BY güncellik DESC) = 1, mutlaka tie-breaker ile.
- Window fonksiyonu WHERE içinde kullanılamaz; CTE'ye sar veya QUALIFY kullan.
- CTE bir performans aracı değil, okunabilirlik aracıdır; çoğu motor onu inline eder.
- TRUNCATE bir DDL'dir, hızlıdır, WHERE almaz, identity'yi sıfırlar; DELETE bir DML'dir, seçicidir, rollback edilir.
- Sütuna fonksiyon uygulamak (YEAR(tarih) = 2024) index'i devre dışı bırakır; aralık koşulu yaz.
- LIKE 'abc%' index kullanabilir, LIKE '%abc' kullanamaz.

Python: Kilit Kavramlar

- Python önce bytecode'a derlenir, sonra PVM tarafından yorumlanır; saf yorumlanan bir dil değildir.
- GIL aynı anda tek thread'in bytecode çalıştırmasına izin verir: CPU-bound'da thread fayda etmez, I/O-bound'da eder; gerçek paralellik için multiprocessing.
- Bellek yönetimi referans sayımıdır; döngüsel referansları nesil bazlı çöp toplayıcı temizler.
- Mutable varsayılan argüman tuzağı: def f(x, liste=[]) yanlıştır, liste=None kullan.
- Fonksiyona geçen mutable nesneyi yerinde değiştirirsen dışarısı etkilenir; parametreye yeniden atarsan etkilenmez.
- list içinde arama O(n), set ve dict içinde O(1); tekrarlı üyelik kontrolünde listeyi set'e çevir.
- dict anahtarı hashable, yani pratikte immutable olmalıdır.
- Generator tembeldir ve sabit bellek kullanır; ancak tek geçişliktir, tükendikten sonra tekrar gezilemez.
- Decorator yazarken functools.wraps kullan; yoksa fonksiyonun __name__ ve docstring bilgisi kaybolur.
- __str__ kullanıcıya, __repr__ geliştiriciye yöneliktir; yalnızca birini yazacaksan __repr__ yaz.
- __eq__ tanımlarsan __hash__ da tanımla; aksi halde nesne hashable olmaktan çıkar.
- super() 'ebeveyn sınıf' değil, MRO listesindeki 'bir sonraki sınıf' demektir.

- finally her durumda çalışır; içine return yazma, try'in dönüş değerini ve istisnayı ezer.
- Çıplak except kullanma, spesifik istisna yakala. Python felsefesi EAFP'dir.
- Dosya ve bağlantıları with ile aç; __enter__/__exit__ temizliği her durumda garanti eder.
- İzolasyon için venv, stil için black + ruff, test için pytest (fixture, parametrize) kullan.

Veri Mühendisliği: Kilit Kavramlar

- ETL dönüşümü ayrı sunucuda, ELT ise hedef ambarın içinde yapar. Modern yığın ELT'dir.
- Idempotent hat: aynı girdiyle kaç kez çalışırsa çalışsın aynı sonucu üretir. Yöntem: partition overwrite, MERGE veya deterministik anahtar.
- Exactly-once pratikte = at-least-once teslimat + idempotent hedef.
- Streaming'de olay zamanı (event time) ile işlenme zamanını ayır; geç veriyi watermark ile yönet, aşanları dead-letter'a yaz.
- Medallion: Bronze ham ve dokunulmaz, Silver temiz ve uyumlanmış, Gold iş seviyesinde toplulaştırılmış.
- Boyutsal modellemede ilk adım grain'i (bir satır neyi temsil ediyor) ilan etmektir.
- SCD Type 1 üzerine yazar, Type 2 yeni satır ekleyip tarihçe tutar (surrogate key şart), Type 3 tek adım geçmiş için sütun ekler.
- Fact tablosu ölçüleri ve boyut anahtarlarını, dimension tablosu betimleyici öznitelikleri tutar.
- Spark'ta transformation tembeldir, action tetikler. Shuffle'ı wide transformation'lar üretir ve asıl maliyet oradadır.
- Skew çözümü: broadcast join, salting veya Adaptive Query Execution.
- Parquet kolon bazlıdır: kolon budama, predicate pushdown ve yüksek sıkıştırma sağlar. Avro satır bazlıdır ve şema evriminde güçlüdür.

Orkestrasyon ve Doküman Zekası

- DAG yönlü ve döngüsüz çizgedir; bağımlılık ve sırayı tanımlar.
- Airflow orkestratördür, işleme motoru değildir. Ağır veriyi worker içinde işleme.
- Airflow görevleri mantıksal tarihe ({{ ds }}) göre yazılmalı; CURRENT_DATE kullanmak backfill'i bozar.
- Sensor'lerde mode="reschedule" kullan, aksi halde worker slotlarını kilitlersin.
- SSIS ETL aracıdır ve GUI tabanlıdır; Airflow orkestratördür ve kod tabanlıdır (git, test, CI/CD avantajı).
- dbt, ELT'nin dönüşüm katmanıdır; ref() ile DAG üretir, test ve dokümantasyonu yerleşik sunar.
- Tesseract metni okur ama düzeni anlamaz (layout blind); LayoutLM metin + konum + görsel bilgiyi birlikte işler.
- unstructured.io farklı dosya tiplerini ortak, tipli elemanlara çevirir ve anlamsal sınırlara saygılı chunking yapar.
- RAG hattı da bir ETL hattıdır: idempotency, tekilleştirme ve metadata yönetimi aynen geçerlidir.

Veri Ambarı Tasarımı ve BI Katmanı

- ER modelleme bütünlük için normalize eder (karmaşık ağ); boyutsal modelleme okuma için denormalize eder (yıldız şema).
- M:N ilişki doğrudan kurulamaz; bağlantı (junction) tablosuyla iki adet 1:N ilişkiye çevrilir.

- Veri ambarı boyutlarında her zaman surrogate key kullan; SCD Type 2'yi mümkün kılan tek şey budur.
- Eşleşmeyen fact kaydını atma; boyuttaki -1 'Bilinmeyen' satırına bağla ki toplamlar doğru kalsın.
- Stored procedure iskeleti: SET XACT_ABORT ON + TRY/CATCH + XACT_STATE kontrolü + logla + THROW.
- CATCH içinde THROW etmezsen hata sessiz kalır; çağıran hat yeşil görünür ve eksik veriyle rapor üretilir.
- 'Bazen hızlı bazen yavaş' tarifi parameter sniffing'e işaret eder. Çözüm: OPTION (RECOMPILE), OPTIMIZE FOR veya yerel değişken.
- Cursor ve WHILE döngüsü RBAR'dır; küme tabanlı tek bir UPDATE veya MERGE yaz.
- CTE indexlenemez ve her referansta yeniden hesaplanabilir; büyük ara sonuç için #TempTable kullan (index + istatistik).
- VertiPaq kolon bazlı, sözlük kodlamalı ve bellek içidir; sıkışmayı belirleyen satır sayısı değil kardinalitedir.
- Row context filtrelemez; onu filter context'e çeviren tek fonksiyon CALCULATE'tir (context transition).
- Calculated column bellekte saklanır, measure sorgu anında hesaplanır; mümkün olan her yerde measure kullan.
- Import hızlıdır ama veri bayattır; DirectQuery günceldir ama kaynağa bağımlıdır. Hibrit çözüm: composite model + aggregations.
- Golden record süreci: standartlaştır, eşleştir (deterministik veya fuzzy), survivorship kurallarını uygula, data steward onayına düşür.
- RLS yalnızca satırları filtreler, sütunları gizlemez. Üretimde dinamik RLS kullan (yetki tablosu + USERPRINCIPALNAME).
- Veri kalitesinde bozuk kaydı silme: karantinaya al, sebebini yaz, hacmini ölç ve eşik aşılınca alarm üret.

Veritabanı Operasyonu, Güvenlik ve Uyum

- Isolation: Read Uncommitted dirty read'e, Read Committed non-repeatable read'e, Repeatable Read phantom read'e izin verir; Serializable hiçbirine.
- NOLOCK sadece dirty read değildir: eksik satır, tekrarlı satır ve sorgu hatası da üretir. Doğru çözüm Snapshot Isolation'dır.
- MVCC'de okuyucu yazıcıyı bloklamaz; bedeli version store (tempdb) veya VACUUM yüküdür. Uzun transaction bu yükü büyütür.
- Blocking geçicidir ve bir performans sorunudur; deadlock kendiliğinden çözülmez ve bir tasarım sorunudur.
- Lock escalation yaklaşık 5000 kilitte tablo kilidine yükselir; büyük DML işlemlerini batch'lere böl.
- Deadlock çözümü: kilitleri her zaman aynı sırada al, ayrıca uygulamada yeniden deneme (retry) kur.
- Partition switch metadata işlemidir, saniyeler sürer. Arşivlemede DELETE yerine sliding window kullan.
- Partition anahtarı yüksek kardinaliteli olmamalı; tarih tipik olarak doğru seçimdir. Sorgu da o anahtarı filtrelemeli.

- RPO = göze alınabilir veri kaybı (log yedeği sıklığını belirler); RTO = göze alınabilir kesinti süresi.
- Point-in-time restore sırası: full, sonra differential, sonra log yedekleri sırayla. Full recovery model şarttır.
- Test edilmemiş yedek, yedek değildir. Restore tatbikatı yap ve süreyi RTO ile karşılaştır.
- TDE yalnızca fiziksel medyayı korur; yetkili sorguya, çalınmış hesaba veya SQL injection'a karşı koruma sağlamaz.
- TDE'li yedeği başka sunucuda açmak için sertifika yedeği şarttır; sertifika kaybolursa kurtarma yolu yoktur.
- HA aynı veri merkezi ve otomatik failover; DR uzak bölge ve manuel karar. Senkron RPO=0 ama yavaş, asenkron hızlı ama kayıp riskli.
- Dinamik maskeleme görüntüyü gizler, bilgiyi değil; WHERE denemeleriyle gerçek değer çıkarılabilir.
- Takma adlaştırma geri döndürülebilir (hâlâ kişisel veridir), anonimleştirme geri döndürülemez.
- Test ortamında tuzlu ve deterministik takma adlaştırma kullan ki JOIN'ler ve referans bütünlüğü bozulmasın.
- Audit log değiştirilemez ve ayrı bir yerde olmalı; DBA silebiliyorsa kanıt değeri yoktur.
- Silme talebini hattın kalıcı bir adımı yap; aksi halde bir sonraki full refresh'te veri kaynaktan geri gelir.
- Crypto-shredding: kişiye özel anahtarı imha ederek yedeklerdeki veriyi geri getirilemez hale getirme tekniği.